

Replication Certification at Publication— Checkable by Anyone, Anywhere, in Seconds, Forever

Gary King* Yiqing Xu† Leo Y. Yang‡

September 19, 2026

Abstract

Authors increasingly deposit replication datasets—the data and code behind published results—in archives such as Dataverse, and journals increasingly require it. But a deposit alone proves little, so authors keep paying: months while a data editor reruns analyses, years answering readers’ questions, and rancor as failed replications become accusations because no record establishes what actually ran. We introduce a *certificate*: a cryptographically signed record of one run of the deposited files. It establishes what the data editor’s rerun establishes, without the rerun, on evidence anyone can check and which no one, including the archive, can backdate or feasibly fake. Certification asks little of anyone and benefits everyone: The author deposits, then reruns the analysis under software that records the run; the archive hashes, signs, timestamps it in Bitcoin’s blockchain, and stores the record—confidential data included, under restricted access—but never runs the analysis; the journal need only confirm that the certificate exists, and may require the manuscript’s tables and figures to match it; and anyone can verify the record in seconds, or replay any step of it, even decades later.

*Albert J. Weatherhead III University Professor, Institute for Quantitative Social Science, Harvard University; GaryKing.org, King@Harvard.edu.

†Associate Professor, Department of Political Science, Stanford University; yiqingxu.org, yiqingxu@stanford.edu.

‡Research Assistant Professor, Department of Accountancy, Economics and Finance, Hong Kong Baptist University; LeoYang.org, leoyang@hkbu.edu.hk. Contributions to this work by this author were made prior to and outside of the author’s current affiliation with HKBU.

Contents

1	The problem and a proposed solution	3
1.1	The problem	3
1.2	The solution: a certificate, and what it certifies	6
1.3	The toolkit, in brief	11
2	Help for authors preparing replication datasets	13
2.1	The replication dataset and its closed copy	13
2.2	Closing and grading	17
2.3	What closing records	22
3	Producing certificates	24
3.1	Stage 1: deposit the replication dataset, receive the seed	24
3.2	Stage 2: run once under the archive’s seed	27
3.3	Stage 3: sign, timestamp, and store the certificate	30
3.4	Revising a certified replication dataset	35
3.5	Confidential data: certified in full, locked in storage	35
3.6	The author’s choices, the protocol’s requirements, and its refusals	37
4	Checking certificates	38
4.1	Part 1: recompute the certificate (static checks)	40
4.2	Part 2: re-execute sampled steps	43
4.3	Part 3: what the journal checks (existence, not replication)	46
5	What certification guarantees	47
5.1	What is not claimed	47
5.2	The attacker’s calculus	52
5.3	Prior work and our contributions	56
A	Cryptographic tools used throughout	61
A.1	Naming bytes: hash and canonical encoding	63
A.2	Binding a person or a time: signatures, nonce, seed, timestamp	64
A.3	Folding a list into one hash: the Merkle root	68
B	Requirements of the archive	69
C	Byte-level rules of closing and deposit	73
	References	79

1. The problem and a proposed solution

Journals, archives, authors, and readers have spent four decades trying to make a deposited replication dataset mean something, and every policy change has moved the cost of verifying it from one of them to another. We offer a way to pay this cost once and have it return dividends forever. A specially designed certificate, issued by the archive when the replication dataset is deposited and sealed after one recorded run, carries proof, in the sense Section 1.2 fixes, that the analysis ran on exactly those files and produced exactly those numbers. A journal that requires it therefore need not replicate anything prior to final acceptance, and a later reader can verify the certified record in seconds instead of re-running all the analyses. A failed effort at replication in the future has a certified record to be argued from rather than a reputation to be argued about.

Figure 1 puts the idea on one page, in three cards—how certification works, why it is proof, and how everyone gains—and is meant to be read alongside this section. Section 1.1 explains the problem, Section 1.2 the solution and what it certifies, and Section 1.3 the tools every stage shares, in brief (Appendix A defines them exactly). Then the machinery, in the order it is used. Section 2 explains how an author prepares a replication dataset, with the software’s help, to a standard met before our proposed protocol begins; Section 3 how the protocol produces a certificate from a ready one; and Section 4 how anyone checks one. Section 5 states what certification guarantees, through what it does not claim and what evading it costs an attacker. Appendix B collects the archive’s duties for whoever builds one. We use well-tested, standard cryptographic tools wherever possible; Section 5.3 separates what the protocol borrows from prior work from what it adds, and lists our contributions.

1.1. The problem

The problem we address (the TODAY column of Figure 1, card 3) is not that replication materials sometimes go undeposited¹; it is that no rung of enforcement yet reached makes a deposit verify itself, so someone pays to verify the same deposit, repeatedly. This subsection has three parts: the four rungs of the escalation, from lightest to strictest; the evidence that each rung catches more than the one below without closing the gap; and finally, the two costs the escalation leaves standing. The larger comes first: the time it takes from science, then the rancor it breeds.

First, the escalation’s four rungs. The norm that authors deposit the code and data behind a published result is long-established in the social sciences (Dewald, Thursby and Anderson, 1986; King, 1995), and professional archives now supply the storage, versioning, and persistent identifiers this protocol builds on (King, 2007). The first rung, and the lightest, is a footnote. Under the original proposal, authors would “indicate in their first footnote in which public archive they will deposit the information necessary to replicate their numerical results,” and journals would police that footnote and nothing more. Editors would be “responsible only for the existence of the footnote, not for confirming that the data

¹We define “replication” as most users and archives do (King, 1995): running the same code on the same data. Some call this “reproduction” and reserve “replication” for a study redone with new data (National Academies of Sciences, Engineering, and Medicine, 2019).



Figure 1. Certification at a glance. Card 1: the author gets a dataset ready (blue), the protocol certifies it (amber), anyone checks it (teal); the timeline pins the run's seeded draws between two block times: a day before the receipt's proof, and the certificate's own proof. Card 2: why the record is proof. Card 3: what each party pays today (red) and gains once certified (green).

set has been submitted nor for checking whether the results actually can be replicated.” Any verification “can and should be left to future researchers” (King, 1995).

The second rung is data on request. Enforcement that light—*Science*’s rule from 2011 that authors supply data and code to anyone who asks—proved better than none but insufficient alone. Asked for the materials behind 204 papers published under it, authors supplied them for 44%, and 26% of the papers’ results could be reproduced (Stodden, Seiler and Ma, 2018).

Journals have escalated enforcement tactics since. The third rung makes deposit of the data and code a condition of publication, as many leading outlets now do (Brodeur et al., 2026; Ferguson et al., 2023; Miske et al., 2026). In economics the requirement reached the *American Economic Review* (*AER*) in 2005 and the *Quarterly Journal of Economics* only in 2016 (Christensen and Miguel, 2018). In political science the editors of twenty-seven journals committed, in the joint editors’ statement of the Data Access and Research Transparency (DA-RT) initiative, to require from 2016 that cited data be available in a trusted repository at publication (Lupia and Elman, 2014; Data Access and Research Transparency (DA-RT), 2015). The Transparency and Openness Promotion (TOP) guidelines generalize the ladder across the sciences, with verified re-execution as their top level (Nosek et al., 2015). Practice has followed: among quantitative political-science articles, the share with a working link to their data and code was rare before about 2008 and has climbed steeply since, from 12% in 2014 to 31% in 2022 (Rainey et al., 2025).

The fourth and last rung, the strictest, is re-execution by the journal. It was first offered as an option: from July 2009, *Biostatistics* let authors request that its associate editor for reproducibility run their code, marking the article *R* if it reproduced, and in two years five of 125 articles earned the mark (Peng, 2011). It is now required at the top of the ladder: human-verified re-execution, in which a *data editor* (journal staff assigned to the task) or an assigned code-checker runs the analysis as a final stage before publication (Vilhuber, 2020; Nüst and Eglen, 2021). In political science, the *American Journal of Political Science* has had the Odum Institute (a university data archive and verification service) verify every accepted quantitative article since 2015, at a mean verification time near seventy-five days (Jacoby, 2018).

Next, the evidence that each of the four rungs catches more than the one below, and none closes the gap, in three categories: whether deposited materials run, whether they reproduce, and whether checking settles it. First, deposit alone does not establish that the deposited materials run: of more than nine thousand R scripts deposited in Harvard Dataverse, 74% did not run to completion without error (Trisovic et al., 2022). Of 15,817 Python notebooks from biomedical publications with declared dependencies, 10,388 could be installed and executed, and 1,203 of those ran through without error (Samuel and Mietchen, 2024), in line with earlier mass executions of notebooks (Pimentel et al., 2019). When two teams each tried to redo one analysis from each of 18 microarray papers, ten could not be reproduced, chiefly because the data were unavailable (Ioannidis et al., 2009). Second, running does not establish that they reproduce the article’s numbers. Working only from deposited files, and without contacting the authors, one audit reproduced the key result of just 22 of 67 published economics papers (Chang and Li, 2022). At *Cognition*, mandated deposit raised usable data availability sharply, while 13 of 35 audited articles still had at least one value that could not be reproduced even with the authors’ help (Hardwicke et al., 2018).

In the largest journal-run audit, of nearly five hundred management-science articles, over 95% largely reproduced when data and environment posed no obstacle, and 68% overall; the gap is exactly those obstacles (Fišar et al., 2024). Finally, checking raises the floor without closing the gap: of the twenty-four papers given in-house replication review at the *Quarterly Journal of Political Science*, four needed no modification, and fourteen produced results that differed from those in the manuscript (Eubank, 2016). Across 254 papers from the three leading political-science journals whose materials could be obtained (two of the three, the *American Journal of Political Science* and the *Journal of Politics*, verify every accepted paper), 61 did not reproduce every coefficient exactly. That is the exact-match count; the paper’s headline figure, 234 of 254 fully or largely reproduced, uses a looser standard (Xu and Yang, 2026).

Finally, we discuss the two costs escalation leaves standing; the larger is time, and it is paid twice. Once at the journal: median verification times at the American Economic Association’s journals have run near ninety days per round, and some manuscripts need several (Vilhuber, 2024). The average manuscript the *AER* published in 2025 spent thirteen weeks in the data editor’s hands, up from nine in 2022 (Luttmer, 2026). And again, indefinitely, by everyone the escalation shifted it to. Journals now staff data editors and adjudicate their disputes with authors. Authors assemble replication datasets to a data editor’s standard and field questions about them for years afterward. For later readers, re-running a deposited replication dataset is real work: teams in one mass-reproduction project averaged thirteen days per paper—reproduction plus robustness re-analysis and correspondence, not the bare re-run—and about one in twelve spent more than a hundred (Brodeur et al., 2024). None of that work accumulates; every reader starts over.

The second and final cost is rancor. When a replication fails, the absence of any shared record of what was actually run turns a technical question into a personal one. The dispute over Reinhart and Rogoff’s debt-and-growth result turned on that question—which spreadsheet, which exclusions, which weighting—and was settled only when the original file changed hands (Herndon, Ash and Pollin, 2014). The LaCour affair turned on whether the deposited data had been collected at all (Broockman, Kalla and Aronow, 2015). Had the first case’s computation been a script, a certificate would have fixed its identity and date on the day it ran (as a spreadsheet it lies outside version 1, Section 5.1). In the second it would have bound the deposited file’s provenance and date without catching the fabrication itself, which replicates perfectly (Section 5.1). The rancor runs deep enough that Daniel Kahneman proposed etiquette rules to govern how a reader attempting a replication may approach its authors (Kahneman, 2014). What no side in such disputes can produce today—a checkable record of the original computation—is what the certificate proposed here supplies, and Section 1.2 describes it.

1.2. The solution: a certificate, and what it certifies

A certified paper carries its own proof. One recorded run, certified before publication, can be checked by anyone, anywhere, in seconds, forever—taking no one’s word for what was done: not the author’s, not the journal’s, not even the archive’s. *Proof* here means what it means in *proof of work* (Dwork and Naor, 1993): evidence anyone can check and no one can feasibly fake at a stated price—not deductive certainty; Section 1.2.1 states what is

proved and Section 5.1 what is not. The claim of this paper is that a certificate is as good as independent reproduction, in this sense. It establishes the same fact a data editor’s re-run establishes: that the deposited code on the deposited data yields the certified numbers, from which the article’s must be taken (Section 4.3). It does so by a record any reader can replay rather than by an eyewitness. “As good as” is a statistical claim, not a mathematical one. Exceptions exist; an author who fabricates a record that no one ever replays is the largest. Section 5.1 names each and Section 5.2 prices it. The design’s bet is that every exception short of fabrication written into the code costs more, and is caught more surely, than writing an honest paper and depositing a correct replication dataset. Fabrication, which the check does not catch, is dated and fixed by the record rather than deterred by it (Section 5.2). Six parts follow. First, the certificate and the archive that issues it; second, what certification saves; third, why no one’s word is needed; fourth, why the check is self-contained; fifth, the four consequences that drive adoption; and finally, what certification costs. Then two named units: the two claims (Section 1.2.1) and the four roles (Section 1.2.2).

First, the certificate (card 1 of Figure 1). A *certificate* (which we label ρ) is a filled-in form for one finished analysis: a document with named fields recording what data was used, what analysis was run, and what output was produced. A *replication dataset* is the folder of files behind the paper—the data, code, and documentation. The author deposits it in an *archive*, a professional research-data repository (such as Dataverse, with widespread community support, institutional guarantees of indefinite storage, archival best practices, disaster recovery, etc.; see dataverse.org). The archive adds metadata and a DOI (digital object identifier: a permanent citable address), and publishes it. The certificate is not a file inside the replication dataset: a metadata field on the published replication dataset links to ρ .

Second, what it saves: a journal that requires authors to certify their replication datasets no longer needs to replicate anything, because the certified run already did, on the terms of Section 1.2.1. Acceptance reduces to an *existence check*: that the deposit, the citation, and the certificate exist (Section 4.3).

Third, why no one’s word is needed (card 2 of Figure 1). The archive hashes the deposit and only then issues the seed, derived from a secret of its own. So the random draws, and the numbers that depend on them, could not have been produced before the deposit. Work that consumes no draw could, and for it the window dates the record rather than the computation (Section 3.3.2). A public pulse recorded in the receipt (the newest six-confirmation-deep Bitcoin block at arrival, Section 3.1) dates the deposit from below, and the timestamp on the certificate closes the window from above. That lower bound is what no signature, log, or timestamp supplies on its own, since each attests only that bytes existed *by* some moment. It is why an author cannot run the analysis under seed after seed and submit the one whose numbers they prefer, *seed shopping* (the residuals are in Section 5.1); Section 3.3.2 makes the argument. Everything else is arithmetic anyone can recompute. The archive’s word is taken for one thing the check cannot recompute—that it issued the seed as specified, without shopping the pulse—and for the account facts it alone holds (who deposited, and whose deposit a predecessor was, Section 3.4). It is never taken for a date: the window’s edges are both block times (Appendix A and section 3.3.2).

Fourth, the check is self-contained (the CHECK band of Figure 1): a local calculation over the replication dataset’s files, plus two values pinned in advance, the archive’s trust root (the

offline key that vouches for its signing keys, Section 1.2.2) and one Bitcoin block hash. The certificate software carries both and runs the check, and both are also deposited under DOIs (Appendix A.2). It takes seconds to at most a few minutes (Section 4.1). It needs no one's permission and no service that must still be answering, so, as long as the mathematics stays unbroken, it works the same in fifty years as today, for anyone (Section 5.1.4). Confidential data change who may run which half of that check, not whether a certificate can be issued and displayed (Section 3.5).

Fifth, adoption; four consequences drive it (the CERTIFIED column of Figure 1, card 3). (1) For authors, the re-run they already do before submission becomes evidence that travels with the deposit, where today it convinces no one but themselves and must be repeated at every revision. (2) For everyone else, the cost of producing the evidence is separated from the cost of checking it. Producing it costs one run of the analysis, however long that run takes, while checking it costs time proportional to the size of the files rather than to the computation. So a certificate can carry evidence for papers no journal would ever re-execute. (3) A reader who wants more than the check can re-run any recorded step whose pre-state, the hashed list of every file present just before the step ran, was stored; the coverage fraction of Section 3.2 says how many. Where a container image was deposited, one command suffices, because a ready replication dataset carries its own software environment (as a container image, wherever licensing allows) and the certificate records the exact files each step started from (Section 4.2). The thirteen days per paper of Section 1.1 included rebuilding what the certificate now carries. (4) Every certified paper gains a measurement no paper reports today, at no extra cost. Because the certified seed replaces the author's, the certificate records for each captured estimate how far it moved between the two seeds, a per-estimate seed sensitivity. It reports the observed change for each estimate, one paired difference rather than an estimate of Monte Carlo error (Section 2.2).

What certification costs. The price is the READY band of Figure 1: the protocol adds one run of the analysis, the certified run, which replaces the re-run a data editor would otherwise do (Section 3.2). Everything before it is readiness, plus two preparations the protocol asks for. Readiness is the two closure runs: two runs of the closed dataset from fresh copies, whose agreement is measured (Section 2.2). An author already does that work, though the protocol asks that it be done single-threaded, which for a many-core analysis is the largest runtime cost (Section 2.2). The two preparations: a Stata analysis routes its seed statements through the capture command before deposit or carries an exception mark, and routes any estimation Stata runs quietly through the capture command if its estimates are to be bound (Sections 2.3, 3.2 and 3.6). And the environment travels as a container image where licensing allows, with add-on libraries vendored inside the deposit otherwise (Sections 2.1 and 2.3). Readiness is what authors already aim at and now have a test for: a replication dataset is ready when, closed under the frozen settings, two runs from fresh copies agree at the determinism tier the protocol requires (Section 2.2). The certificate software runs that test, names any file that came out unstable, and leaves a report that is deposited with the dataset; the goal is byte-identical agreement. A revision that changes the files repeats the readiness test and the one certified run (Section 3.4). For the author, that one run is close to pushing a button: the hardware, the software, every dependency, and every

particular of the code and data are already in place on the machine the analysis was built on. A data editor or a later reader must do the same work from nothing. They must obtain the hardware and software, install the dependencies, work out the particulars of someone else’s code and data, and then hope the machine has the memory and the code handles every edge case. That is the work certification removes. Because the certified seed replaces the author’s, the final manuscript’s simulation-based numbers come from the certified run, so a journal can require certification only at final acceptance, sparing each revision round the cost. What a changed number means for review is settled by Section 4.3’s default. The one run and the two preparations are the author’s price at certification time. The disclosure review for confidential deposits (Section 3.5) and the archive’s storage exposure (Appendix B) are priced where they arise, and everything a reader or journal does instead got cheaper.

1.2.1. The two claims: record consistency and replication

The certificate makes two claims, the two halves of the CHECK band in Figure 1. The first can be checked without running the analysis—it is only hashing and signature arithmetic, so it finishes in seconds even if the analysis itself takes days; the second requires running recorded steps again, and costs what those steps cost. In these claims, three concepts recur. A hash H computes a fingerprint of a file’s *bytes*: a short identifying code that changes dramatically for any change in the file. A seed is an integer that starts the random draws in an analysis (the role of `set.seed` in R). The environment is the computer setup the analysis ran in (operating system and programs). These three are working definitions, enough to follow this section; Section 1.3 collects the toolkit, Appendix A defines it exactly, and Section 2.1 defines the third. The two claims:

1. **Record consistency.** The certificate still matches the files and has not been rewritten: the archive issued the run’s seed only after hashing the deposited replication dataset, as its signed receipt attests. The recorded run started from that replication dataset’s code, data, and environment and reported these outputs, as a record the certificate software wrote and any reader recomputes. The files and outputs now stored with ρ still match those hashes. Whether that record describes a computation that happened as written is the second claim’s business: a fabricated record survives only until one of its steps is re-run.
2. **Replication.** Running the same steps again would produce the same results. A *pre-state* is a list of every filename in the replication dataset, with its hash taken just before a script runs. A *determinism tier* records how closely two runs of the same *closed replication dataset*—a copy in which the usual sources of run-to-run differences are frozen (defined in Section 2.1)—agreed (byte-identical, close enough numerically, or a real difference; Table 1). The claim: re-executing recorded steps from their stored pre-states produces outputs that agree with the certified outputs at the given determinism tier, adjudicated only in an environment that matches the recorded one (Section 4.2).

Neither claim implies that the analysis answers the paper’s question, that the scripts whose hashes are in the certificate are honest, or that the printed article reports the certified numbers. The printed page is tied to the certified run by policy, not cryptography: authors

must take reported numbers from the certified run, and anyone who doubts them can compare (Section 4.3). Section 5.1 lists everything else not claimed.

1.2.2. The four roles: author, archive, journal, reader

Four roles take part: author, archive, journal, and reader (Figure 1 names three beside the band where each acts and lists all four, with science, in card 3). They differ in what they do, not in what they can check: for the deposits journals accept, no role holds evidence the check needs that the others cannot obtain; for a restricted deposit, any approved reader can (Section 3.5). So the archive that signs a certificate and a reader who doubts it can run the same verification. Systems that instead assign checking to a designated person, agency, or platform (discussed in Section 5.3) hand the reader a verdict; we let any reader run the check. A role is a job rather than an institution: one organization can hold two, as when a university press both publishes the article and archives the files.

1. **Author.** The researcher who did the analysis. The author prepares the replication dataset on a machine they control, obtains a closed replication dataset, deposits it, performs the certified run, and submits ρ .
2. **Archive.** Stores the replication dataset and signs the finished certificate. A professional archive already does institutional work—custody, versions and lasting addresses, backups and succession, metadata and terms of use, restricted access—that the later check of ρ never inspects and that a passing certificate does not attest. For certification the archive hashes deposits and issues a *receipt* D (one signed, timestamped document, issued only after that hash, that includes the run seed; Section 3.1). It signs the finished certificate (the *witness* W , Section 3.3), adds a metadata field on the published replication dataset that links to ρ , and keeps a *ledger* (an append-only public list, Section 3.3). Who operates the archive is never part of what a passing check attests.
3. **Journal.** The publisher of the article. The journal keeps the two duties it has always had: tentative acceptance conditional on the replication dataset being deposited and cited, then a check that both happened. It gains one look-up of the same kind, that the certificate exists; it runs no code and chooses nothing technical (Section 4.3 states the duties). The certified run establishes what a data editor’s re-run would (Section 1.1). That substitution is the point of the design.²
4. **Reader.** Anyone who, after publication, wants to check the certificate, including anyone attempting a replication (this paper has no separate word for a replicator: a replicator is a reader who re-runs). The reader downloads the replication dataset from the cited version, follows that version’s metadata link to ρ , and pins the archive’s *trust root* (an offline root key that vouches for the archive’s signing keys through a public key log) and one Bitcoin block hash. Both are settled in advance and obtained away

²What the certificate’s check does not cover, and a data editor’s re-run does, is checking that the printed article’s numbers match the record. That becomes a mechanical reading task against the captured estimates. Section 4.3 offers it with the rest of what survives of the data editor’s review.

from the files being checked, through the recognized certificate software or the archive’s deposits (Appendix A.2 defines both exactly).

Figure 2 compresses everything that follows onto one page: readiness and the three stages of production, the three-part check, and every symbol beside its meaning. It is meant to be returned to; each entry names the section that defines it.

1.3. The toolkit, in brief

Every stage of the protocol uses the same small set of cryptographic tools, and two conventions govern all of them. This subsection names each in a sentence, enough to read the protocol; Appendix A defines each exactly. In order: the two conventions; the three tools that name bytes; the four that bind a person or a time to bytes; and finally the one that folds a list into one hash.

First, the conventions. A schema is the machine-readable half of this specification: one small file tree fixing every byte-level spelling—key names, field order, grammars—whose meaning this paper explains (Appendix A). The protocol version v names which edition of the rules, and which schema, every signed object was built under; this paper is version 1, and version 1 is closed: its constants are rules and its lists are complete (Appendix A).

Second, naming bytes. The hash H (SHA-256) computes a 32-byte fingerprint of any run of bytes, and no one can find two inputs that share one fingerprint; every file, list, and signed object in the protocol is named by its hash (Appendix A.1). The canonical encoding $\text{enc}(\cdot)$ turns a record with named fields into one exact run of bytes—one JSON spelling, fields sorted by key, no floating-point values—so that every implementation hashes the same record to the same value (Appendix A.1). $\text{pack}(\cdot)$ lays a whole file tree end to end into one run of bytes, in a form every implementation reproduces, so that a tree too has one hash (Section 2.1).

Third, binding a person or a time to bytes. A signature $\text{Sign}(k_{\text{priv}}, m)$ is a mark on the bytes m that only the holder of the secret key can make and anyone holding the matching public key k_{pub} can check (Ed25519; Appendix A.2). The nonce ν is a number the author could not have known. The archive computes it with a keyed hash, $\text{HMAC}(k_{\text{arch}}, \cdot)$ under a secret key k_{arch} of its own used for nothing else, over the nonce context. That context includes the hash of the deposited files and the freshness pulse (the newest six-confirmation-deep Bitcoin block at arrival, Section 3.1), so the nonce can exist neither before the bytes it is bound to nor before a public moment. The seed s , which starts the analysis’s random draws, is derived from ν by a public rule, so the author cannot choose it and anyone can check it. An author’s own reseeds are re-mapped through s where the language allows and disclosed where it does not (Section 2.3). The timestamp proof π places the certificate’s hash in the public Bitcoin record, which closes the window around the run from above—the receipt’s pulse closes it from below (Section 3.3.2)—and no party, the archive included, can backdate it.

Finally, folding a list into one hash. A Merkle root folds an ordered list into one hash so that changing any item, or the order of items, changes the root; the certified run’s list of step records is named this way, as R_T (Appendix A.3).

TOOLKIT (§A) <i>version 1 is closed: constants and lists are rules; spellings live in the schema</i>			USED THROUGHOUT
v	protocol version; readers apply the declared H edition only	SHA-256: any bytes in, 32-byte fingerprint out	
$\text{enc}(\cdot)$	one exact JSON spelling, so equal records hash equal	a tree as one byte run; $H(P^*)=H(\text{pack}(P^*))$	
Sign, Vrfy	Ed25519 under k_{priv} ; checked under the π pinned trust root	timestamp proof: bytes existed by this moment	
Merkle root	one hash over an ordered list (RFC 6962 tree); ν, s any change shows	nonce: a keyed hash only the archive computes; seed s from ν by public rule (band 1)	
READY (§2) <i>the author's work, before the protocol: freeze, measure, name by hash</i>			AUTHOR'S MACHINE
P	the replication dataset: code c , data d , other P^* inputs u , environment e (κ = version list)	closed copy: frozen settings + capture code, at most C_{size} (1 GB); outputs o arrive at the run	
capture	added files copying every estimate out of the statistical runtime (read and print only); record h_c^- keeps the author's code recoverable		
h_G	group hash: one fingerprint per file group x, h_x	commitment $x = (h_c, h_d, h_u, h_e, h_o)$; $h_x = H(\text{enc}(x))$ names the certified dataset	
s_0, τ	two closure runs under author's s_0 , compared tiers after normalization, earn τ	T0 bytes; T0-meta; T1 10^{-9} ; T1-reported 4 significant figures; T2 disclosed; TF failed	
audits	dependency audit (no undeclared reads); evidence source state; entry scripts = targets	P^* + evidence set stored with ρ , or no certificate	
$\downarrow P^*$ uploaded; $\text{Run}(e, c, d, u, s) = o$ is now well defined			
1 DEPOSIT (§3.1) <i>hash first, then seed: the seed cannot precede the bytes it is bound to</i>			ARCHIVE
id	deposit identifier $H(\text{enc}(H(P^*), v))$; same γ, β bytes replay it	nonce context: $id, H(P^*)$, key id, and pulse β , a six-deep Bitcoin block D cannot predate signed receipt: $id, H(P^*)$, group hashes, closure hash, depositor, predecessor, t_{arr} , ledger position, β, ν, s , and more; timestamped	
$\nu \rightarrow s$	$\nu = \text{HMAC}(k_{\text{arch}}, \text{enc}(\gamma))$; the public rule D gives s		
$\downarrow$ author holds D and a seed no one could have known earlier			
2 RUN (§3.2) <i>the one certified run, every step written down</i>			AUTHOR'S MACHINE, ONCE, UNDER s
i, M_i	per step: pre-state manifest, every file hashed L_i, Δ (root R_{M_i})	console transcript; processes sampled every Δ (50 ms)	
r_i	step record: script, exit, times, written-file Q hashes, $H(L_i)$, processes	result capture: every listed-command estimate, saved to a file or not	
T, R_T	trace $(r_0, \dots, r_{n-1})$, Merkle root R_T C_{blob}	blobs kept to 200 MB; re-execution coverage n_c/n	
$\downarrow$ one run $\rightarrow$ one certificate			
3 SEAL (§3.3) <i>sign, timestamp, list, link; then freeze the published version</i>			ARCHIVE
ρ	the certificate: $x, h_x, \kappa, \tau, D, \nu, s, T, R_T, W, \pi$ $H(Q)$, audits, coverage, v	witness: archive signs $H(\text{enc}(\rho))$; timestamp on those bytes	
E_i	ledger: a Merkle log of D, W, P , and key-log DOI lines, its head signed and anchored hourly; ρ stores its own inclusion proofs	stored by hash, linked from the published version; cite it, "Certified."	
id^-	a revision is a new deposit naming its predecessor; per-author version chain		
$\downarrow$ published; from here on, anyone can check			
CHECK (§4) $\text{Verify}(\rho, P^*, \theta, k, \omega)$: one mode; every yes-or-no finding that runs must pass			ANYONE
Part 1	$k = 0$: recompute every hash, root, seed, tier, audit; check signatures, dates, and that D names the tree; ten findings, seconds	re-execute k random steps from stored pre-states at τ ; ω : re-query pending timestamps	
odds	$1 - \binom{n_c - m}{k} / \binom{n_c}{k}$: chance k steps hit one of m faked among n_c replayable	Part 3 journal looks up: P^* present, one link to ρ , no exception mark	
not claimed	(§5.1) fabricated or honest-but-wrong code, p -hacking, a privileged machine; seed shopping made visible		

Figure 2. The protocol on one page. Each band's chip names who acts (slate = author's machine, red = archive, teal = anyone; gray = no actor); arrows carry the object that moves; each band's section defines its bold symbols.

Four standard assumptions, on SHA-256, HMAC, Ed25519, and Bitcoin’s blockchain, and one trust assumption in the archive carry every security claim; Appendix A states them.

2. Help for authors preparing replication datasets

A replication dataset is *ready* when a second run of it gives the same results; this section makes that precise and helps the author get there. Readiness is the author’s own work, on the author’s machine, at the author’s pace, with the certificate software as the test, and it is finished before the protocol of Section 3 begins. Closing—the procedure that gets a dataset ready—freezes anything that could make a second run differ, then measures how close two runs of the closed replication dataset come. Byte-level spellings that only an implementer needs are collected in Appendix C. Three parts follow: first, the objects being closed, from one file to the whole tree; second, the closing procedure and the determinism tier it earns; and finally, what closing records about the dataset for the certificate to carry.

2.1. The replication dataset and its closed copy

Closing exists to make one function well defined. This subsection states that function first, then the objects it takes, built up from one file to the whole tree. In order: the dataset and its four groups; how a file is named; the record of the software environment, and the container image that carries the environment; and the closed copy. Then the hashing: how a group of files gets a single hash, and five such hashes one name; and how the copy is laid out as one byte run (pack).

Run. The purpose of closing a replication dataset is to make one function well defined: the closed replication dataset, its environment, and the seed together determine the outputs, the way a well-defined estimator maps data to a number. Write that function as

$$\text{Run}(e, c, d, u, s) = o, \tag{1}$$

where e , c , d , and u are the environment, code, data, and other-input groups the next unit defines (Section 2.1), s is the seed, and o is the outputs. Calling it *well defined* means that the same closed replication dataset, environment, and seed must produce outputs that agree at the determinism tier τ of Table 1.

Replication dataset P . A replication dataset P is a *file tree*: the folder and every file inside it, including nested folders. *Partition* its files (split them with no overlap and no remainder) into four groups, by a rule an implementation applies without judgment. Code c is the *executable scripts*: files the computer runs, recognized by the version-1 schema’s extension list—.do, .R, .py, and kin. The list is complete, so a replication dataset whose analysis lives in a language off it cannot be certified under version 1, a scope limit restated in Section 5.1. Data d is the non-code files the dependency audit, Section 2.3, finds the scripts reading. Other inputs u are all remaining files except the environment capsule, the group that follows. They include, whatever their extensions, the files under the schema-named library root, since a vendored library is not the author’s analysis (Section 2.3). Environment

e is exactly one file: the *environment capsule* κ , which records the operating system, language runtimes, and add-on software versions, analogous to R’s `sessionInfo()` dump.

Write o for the *outputs*—the files a run writes, such as tables, figures, logs, and derived data. These are not the article: a paper is typeset separately, often with numbers moved by hand, and the certificate binds the files without binding what the article prints (Section 4.3). Outputs are not a fifth group, because they are not something the author deposits: they do not exist until the certified run makes them (Section 3.2), which happens only after the deposit has been hashed (Section 3.1). The certificate records their hashes beside the other four, which is why the commitment x of this subsection lists five hashes over a tree partitioned into four. If the author’s tree holds output files left over from a run of their own, those are files in one of the four groups; the certified o is whatever the certified run wrote.

Paths. A path is the file’s name inside the replication dataset, written in one fixed style so two computers spell it the same way. *Relative* paths start from the replication dataset root, not from the whole disk (so `code/estimate.R`, not `/Users/smith/paper/code/estimate.R`). They use `/` separators, no leading `./`, and no trailing slash. The remaining rules are byte-level: UTF-8, Unicode normalization form C, byte comparison so that letter case matters, no empty or dot components, no control characters, no symbolic links; Appendix C states them. They are part of the protocol rather than implementation taste, because a path goes into a hash: two implementations that spell one file differently compute different group hashes for the same replication dataset. A symbolic link whose target lies outside the replication dataset is an undeclared external reference and stops certification through the dependency audit (Section 2.3).

Environment capsule κ . The environment capsule is the written answer to “what software was this run made with,” and its field list is fixed here so that every implementation records the same things. Encode κ with `enc` and store it as the single file of the environment group e , at the fixed path `environment.json`. Its fields are the operating system and processor architecture; each language runtime the targets will run (Section 2.2) and its exact version; and each add-on library the dependency audit (Section 2.3) finds the scripts loading, and its version. Next come the numerical library and the C mathematics library the runtime links, the two sources of last-digit disagreement between machines. The rest are the frozen settings in force (Section 2.2); the container image’s digest when one is deposited (Section 2.1); and the certificate software’s own version and source hash, the pair $(v_{\text{tool}}, h_{\text{tool}})$ of Section 3.1, *tool* abbreviating the certificate software in every subscript. Appendix C lists them exactly. κ is assembled at closing, before the closure runs (Section 2.2), from the machine’s stated versions; no analysis execution is consulted. Reading a runtime’s version or its linked numerical library does run that runtime’s own probe commands, which the schema names so two implementations fill κ identically.

What κ is worth is limited to *declared evidence*, a recorded claim the check keeps but cannot re-derive, like the source state of Section 2.3. The check recomputes h_e from the deposited bytes and so establishes that the capsule in hand is the capsule that was certified, but nothing in the check confirms that the author’s machine held that software. Re-execution does not rebuild from κ either—it restores files from a recorded pre-state and runs under

the frozen settings (Section 4.2). κ is there to tell a later reader what to install to attempt a fresh replication of their own.

Container image. A ready replication dataset carries its software environment, not only a description of it. Wherever licensing allows, the author deposits a *container image* (a packaged copy of the whole software environment); the closure runs and the certified run execute inside it. κ records the image’s content digest; an OCI digest is a hash of the image’s bytes, so the binding is cryptographic. The image is not a file of P^* : the archive stores it beside P^* , content-addressed by that digest and outside the size ceiling, and serves it to readers who re-execute (Appendix B). The image is what lets a later reader re-run a step with one command (Section 4.2); where a license forbids it (a proprietary runtime), κ ’s exact versions are the recipe the reader restores from instead. Add-on libraries live in the image when there is one, and otherwise are vendored inside P^* (Section 2.3).

Closed replication dataset P^* . A closed replication dataset is a copy of P with two changes: the usual sources of “it came out different this time” are frozen (clock, locale, extra processor cores), and the capture code (Section 2.2) is added. The author’s original is never modified; every change lands in the copy.

Because the copy carries the capture code, the code group’s hash h_c (Section 2.1) covers that code, and a reader who re-runs the deposit runs it too—so the deposit no longer matches the author’s own scripts byte for byte. The certificate therefore also carries a *capture-code record*: the code group hash as the author supplied it, written h_c^- , together with the before-and-after hashes of every file the capture code touched and the hashes of any files it added. A reader can then check the author’s original code against a code repository or a working paper, and see exactly which files grew. Like the source state (Section 2.3), this is declared evidence: the check recomputes h_c from what was deposited, but h_c^- names something that was not deposited and so cannot be re-derived from it.

Group hash h_G . A group hash is one hash standing for a whole set of files—all the code, say, or all the data. It is not taken over the files’ bytes directly. Each file instead contributes one line giving its name and the hash of its contents; those lines are sorted and joined into a short text, and the group hash is the hash of that text. Because every line carries a content hash, altering one byte of any file changes h_G . Because the lines carry the paths and are sorted by them, renaming, adding, or removing a file changes h_G as well—so the group hash covers what the files are called and which files are present, not only what is inside them. For a group G with files at relative paths $p_1, \dots, p_m$, write one line per file: the path, then the hexadecimal hash of that file’s bytes. Sort the lines by path as bytes, join them, and set $h_G = H(\text{those lines})$; the separators, the join, and the empty group’s value are spelled in the schema (Appendix A). The subscript names which group was hashed, so the four groups of a replication dataset have hashes h_c for code, h_d for data, h_u for other inputs, and h_e for environment.

Commitment x and commitment hash h_x . The hashes for code, data, other inputs, environment, and outputs are published together as a commitment (Appendix A.1), so none

of them can be swapped later. The public commitment is

$$x = (h_c, h_d, h_u, h_e, h_o), \quad (2)$$

where h_c , h_d , h_u , and h_e are the group hashes just defined, one for each of the four deposited groups, and h_o is the group hash of the certified outputs over their *raw* bytes. A second value of h_o , computed after normalization (stripping leftover date marks such as dates inside a PDF, Section 2.2), is stored in ρ beside the raw one, but only the raw value enters x . The normalized value covers only outputs stored as blobs: a file above C_{blob} , Section 3.2, is hashed raw only, since no reader could renormalize bytes it does not hold. Encode x with `enc` as a JSON array of those five hashes in that order, each as 64-character hexadecimal. The *commitment hash*

$$h_x = H(\text{enc}(x)) \quad (3)$$

names the *certified replication dataset*, which the author never assembles as a folder. Instead, it is the closed replication dataset P^* (Section 2.1) together with the outputs o that the certified run wrote from it: everything the run consumed and everything it produced. It carries no letter of its own, because h_x is its name: one 32-byte value standing for the pair, which is what a certificate needs to refer to it. P stays the author’s untouched original and P^* the deposited copy; neither of them contains o .

x is a commitment in the sense of Appendix A.1: the receipt D binds the four input hashes at deposit, and the witness W binds all five through h_x , so none can be swapped later.

Packed replication dataset $\text{pack}(P^*)$. Packing lays a whole file tree end to end into a single run of bytes, so that the tree has one hash, the way a zip file has one hash. Unlike a zip file, the packed form is defined so every implementation produces the same bytes. Three parts: first, the packed form; second, why the archive serves the packed bytes themselves; and finally, the two hashes that now name a replication dataset. First, the form. Every file is a run of bytes already; a tree is not, being many such runs plus the names and nesting that tell them apart. Packing must therefore carry those names and lengths, or two different trees could flatten to the same bytes and share a hash. $\text{pack}(P^*)$ writes files in path-sorted order, each as a short header line followed by the file’s raw bytes. The header line holds the path, the byte count, and the permission bits, the last written as the constant 644 for every file: an interpreter, not the operating system, runs every recognized script, so an executable bit would do no work. A constant also makes `pack` a pure function of paths and bytes, so two closings of one tree, on any filesystem, write identical headers. No owner, group, or modification time appears, and the header’s exact spelling is fixed in the schema (Appendix A). Two identical trees must produce identical packed bytes on every implementation.

Second, transport. An archive’s ordinary handling of uploads can change bytes: a tabular ingest, line-ending or filename normalization, a re-assembled download. So the archive serves P^* as the packed byte run itself, one opaque file exempt from ingest, stored under its *content-addressed* name $H(\text{pack}(P^*))$, the object’s name being its hash (Appendix B). Unpacking is defined as `pack`’s inverse, so the bytes round-trip exactly. Throughout this paper $H(P^*)$ abbreviates $H(\text{pack}(P^*))$, since H applies to a run of bytes and `pack` is what supplies one.

Finally, the two hashes. Two now stand for a replication dataset, and they are not the same thing. $H(P^*)$ is taken over the closed copy’s own bytes, and is the value the archive computes when the files arrive; it names what was sent. h_x is taken over the commitment, which is a list of hashes rather than a tree of files, and so it also covers the outputs, which did not exist at the moment of deposit; it names what was certified.

2.2. Closing and grading

Closing proceeds in order. Check that the dataset can run at all; freeze the settings that would otherwise vary between runs; decide which scripts to run; add the capture code; then run twice and record how closely the two runs agreed. Outputs are compared only after normalizing away generation noise, the rule that closes this part.

Preflight. Preflight is a cheap look before any real work: is this a replication dataset that can be run, and can the archive be reached? Before closure, reject on three grounds. First, a tree that cannot be run or packed: no code file, a missing runtime, two paths differing only in letter case, an empty directory, no writable scratch space. Second, a tree larger than the size ceiling C_{size} , fixed at one *gigabyte* (10^9 bytes) before encoding; the ceiling keeps the static check’s hashing to about a second (Section 4.1) and bounds the archive’s storage exposure; larger materials enter as declared external dependencies (Section 2.3). Finally, an unreachable archive. Appendix C gives the list with each reason.

Frozen settings. *Frozen settings* standardize the timezone, language formatting, build date, and processor-core count so that none of them changes the numbers. Three parts: first, clocks and formatting; second, processor cores; and finally, hash randomization. First, clocks and formatting. The settings do not stop the clock: code that writes the current time into an output is the author’s to remove at closure, where the closure runs expose it. The reproducible-builds community interposes on the clock itself ([Reproducible Builds Project, 2026a](#)), a step this design declines because no such tool runs without privileges on every desktop system. On P^* set the process timezone to UTC and the *locale* (the settings that choose date format and decimal mark) to the plain C-language default: a period for decimals, dates in one fixed spelling. Freeze the “date of this build” to the schema’s fixed value, so output files do not embed today’s date. Build tools honor the setting; most statistical software ignores it, and there the normalization rules of Section 2.2 do the same work after the fact. Appendix C names each setting.

Second, cores. Numerical libraries read process settings to decide how many processor cores to use when adding numbers. Set each setting on the schema’s threading list to 1 and, where a numerical library dispatches different code by instruction set, its reproducibility switch to its fixed branch (Appendix C); the sums these libraries take then run on one core in one order. Two machines that differ only in core count can otherwise disagree in the last digits, since adding a vector of numbers in a different order is a different floating-point expression. Single-threading is the protocol’s largest runtime cost: an analysis tuned for many cores runs correspondingly longer, and it does so three times (Section 1.2; a frozen thread count above one, measured by the closure runs like everything else, is a natural

version-2 relaxation). Code run on graphics processors adds instability of its own: the same container can fail to give identical results on other hardware or drivers (Heil et al., 2021). Code that parallelizes by other means is untouched by these settings; the closure runs measure whatever instability remains.

Finally, hash randomization. A language runtime *randomizes hashing*, in an unrelated sense of the word, when it shuffles the order of items in a table so two runs walk them differently; Python dictionaries are the usual case. Freeze any such randomization to the run’s step seed. In the closure runs that is the seed derived from the *closure seed* s_0 , the author’s own starting number, used only there (Section 2.2), and never the certified run’s seed s ; in the certified run and any re-execution it is s_i (Section 3.2). The seed rather than a constant is chosen so that code whose output order depends on the runtime’s table order shows the dependence as seed sensitivity rather than hiding it.

Entry scripts. Entry scripts are the starting files: the ones nothing else calls, run in a fixed order—the master `.do` file, not the files it `dos`. Build a *directed graph* of scripts: an arrow from A to B means script A calls script B . What counts as a call is not left to judgment: the arrows are exactly the matches of the schema’s per-language invocation patterns, the same pattern family the dependency audit runs (Section 2.3). The patterns are `source` in R, `do` in Stata, `import` in Python, the launch of a separate process naming a deposited code file, and `kin`. The capture code’s added files are never targets and contribute no arrows; they are loaded only through the line inserted at each target’s top (Section 2.2). A script’s *in-degree* is how many arrows point at it. Scripts with in-degree zero, listed in path order, are the default *targets*, the run’s name for this entry-script list—*target* is the name used from here on. If every script is called by some other script, so none has in-degree zero, every code script is a target; files under the library root contribute no arrows and are never targets (Section 2.3). Closing always writes the target list, in execution order, to a schema-named file inside P^* . The list is proposed from the in-degree rule and shown to the author to confirm or edit, since real trees hold stale scripts nothing calls, and path order can put an analysis before its cleaning step. Because the confirmed list is a file inside P^* , the order is inside h_u , and finding 2 (Section 4.1) checks the trace T (the run’s list of step records, Section 3.2) against that file rather than against a declared value. Some project structures’ top level is not a script the patterns see: a Makefile, an RMarkdown or Quarto document, a pipeline framework’s configuration. Their authors edit that file, which exists for this reason; the in-degree rule is the default, not a claim that it fits every project.

Capture code. Closing adds *capture code* to the copy it works on: a small amount of code whose only job is to copy estimates out of the statistical runtime as it runs. The certificate can then bind the numbers a paper reports and not only the files that produced them. Three parts follow: first, why files alone are not enough; second, what capture produces—the result capture Q —and how Q differs from the outputs o ; and finally, how the code arrives, in brief.

First, why files are not enough. A paper’s coefficients and standard errors live inside the runtime’s memory while it runs, and only what the author’s code chose to write ever reaches a file. A certificate built from files alone would therefore bind code and data to output files while saying nothing about the estimates a reader wants to compare against the

article. The files leave two gaps. Numbers a paper states in its running text—a coefficient quoted in a sentence, a p -value in a footnote—often reach no file. And the files that do exist are frequently in no form anyone can compare: a figure is a picture, and a table may be a typeset fragment from which a number has to be read back, which Section 4.3 notes can be misread.

Second, what capture produces. Capture code forces those numbers into a per-step *capture file* the run writes beside its other outputs. Its records become the *result capture* Q : one record for every estimation the analysis performs, in a fixed exact form. The certificate binds Q , and a later check rebuilds it from the capture files and matches it byte for byte. Q and the outputs o are both products of the certified run, but they are not the same thing and $H(Q)$ is not h_o . The outputs are the author’s selection: whatever their code chose to write to disk and nothing else. Q is the protocol’s extraction: every estimation run through a listed command (Section 3.2) yields a record, wanted or not. A figure appears only in o , an estimate the author discarded only in Q , and a check tests each in its own way. Q ’s exact form is given with the certified run in Section 3.2, since that is when Q comes into being; closing only puts in place the code that will produce it.

Finally, how the code arrives, in brief. Capture code arrives as files added to the code group plus a single *loading line* at the top of each target (Section 2.2). The loading line *seats* the step’s seed (sets the runtime’s random-number generator to it), masks the language’s set-seed call, sets the determinism state no process setting reaches, and prints a *seed banner* into the step’s transcript. The files and the line are fixed in the version-1 schema, and after its duties capture code may only read and print. The mechanism by language, the four duties in full, the protections on the channel, and the author-callable capture command are in Appendix C.

The two closure runs and tier τ . The two *closure runs* are a pair: run the closed replication dataset twice, each run from a fresh scratch copy (a temporary working copy, Section 3.2), under the same s_0 and frozen settings, then see whether the files come out the same. The tier records how closely they agreed. Three parts follow: first, the pair and the tier it earns; second, the seed sensitivity the pair also measures; and finally, what tier T2 needs and where the numeric tiers’ edges lie. First, the pair. This pair is the readiness test of Section 1.2: the author’s own work, done privately and repeated freely, of which only the kept pair’s report is deposited. Compare every file both runs wrote, after normalization (stripping leftover date marks, Section 2.2), by hash. Which formats are parsed for numbers, and how the two runs’ numbers are located and paired for Table 1’s numeric tiers, are fixed in the schema (Appendix A); *unparseable* structure, a file no rule of the schema reads, is treated as different. The raw hash of what was actually written is stored in every case. Closing stops, naming the path, when a run wrote a path that spells the current date or time in the schema’s formats (refusal vii, Section 3.6). A re-execution on a later day would write another name and fail the write-set rule of Section 4.2; the fix is one line of the author’s. The replication dataset’s tier τ is the most stringent class in Table 1 that holds for all pairs; a single unstable figure sets the tier for the whole replication dataset. Two runs are a floor, not a guarantee: nondeterminism that strikes intermittently can agree twice by chance, so the tier records what was observed. The tier also measures agreement *under one seed*. It says

nothing about how much an estimate moves *across* seeds, which is Monte Carlo variation, a property of the analysis rather than of its determinism. In principle even the tier could be seed-dependent (a degenerate bootstrap resample under one seed and not another); that is rare, but one more reason the tier is read as measurement, not guarantee.

Second, seed sensitivity. The certified run’s seed s will differ from s_0 , so simulation-based numbers in the final tables can differ, legitimately, from any computed before deposit (Section 1.2). The certificate records that movement at no extra cost. For each uniquely paired captured coefficient, the record gives its value under the first closure run’s seed s_0 , its value under the archive seed s , the absolute movement between them, the movement divided by the positive finite baseline standard error when that scale exists, the two standard errors, and the absolute change in standard error. This is an observed two-seed contrast between the author’s baseline seed and the archive’s seed. It is not an estimate of Monte Carlo error and carries no cross-seed refusal threshold.

The accompanying *sensitivity summary* reports coverage before it reports magnitude: its state and reason, its scope, the baseline and comparison seeds, the numbers of baseline and comparison records, the numbers of paired and standard-error-scaled coefficients, and the counts of ambiguous records, unpaired records, and invalid coefficients. Its largest movement is the maximum baseline-standard-error-scaled movement among coefficients for which that scale is positive and finite. An invalid coefficient, an unpaired record, an ambiguous repeated identity, an absent baseline, or a missing standard-error scale is not recoded as zero movement; it is disclosed as unavailable or incomplete coverage. Which records are paired is fixed by the grouping rules of Appendix C. What the pair measured is written into the *closure report*, a document in the schema’s format (Appendix A). It holds the two runs’ per-file agreement: each file’s two raw hashes, two normalized hashes, and, for a parsed format, its relative gap, from which the file’s own tier is a function. It also holds both runs’ captured values, the deposit’s $H(P^*)$ and s_0 , and the count of pairs the software saw attempted, a self-report an author can reset. The certificate software computes the differences against the certified run’s Q and writes them, with the summary, into ρ (Section 3.3), and the check re-derives τ and the sensitivity record from the report and Q (Section 4.1). The report is deposited beside P^* , never inside it, since it names $H(P^*)$ and so cannot be among the bytes that hash covers. The archive signs its hash into the receipt (Section 3.1), so the kept pair was fixed before the seed existed: an author cannot run further pairs after seeing the certified numbers and submit the one whose tier or baseline suits.

The tier and seed sensitivity can point in different directions. A fixed-seed Markov chain Monte Carlo run can be byte-identical across the closure runs (T0) while its posterior summaries move in the third significant figure across seeds; another certificate can have little cross-seed movement but incomplete sensitivity coverage because the estimates could not be paired. That is why the metadata field a journal reads reports the sensitivity state, coverage counts, and largest standard-error-scaled movement beside the tier (Appendix B). Without the closure runs no pair is measured, so no τ is earned, the later check could not re-derive one (Section 4.1), and sampled re-execution would have no measured allowance. No certificate is issued without them, since a certificate means a verifiable record. A deposit that lacks the closure report receives no receipt, since D signs the report’s hash (Section 3.1), and one that lacks the rest of the evidence set receives the receipt and nothing more (Section 3.6).

Finally, tier T2. It needs one extra record. From T0 through T1-reported, one allowance

Table 1. Determinism tiers. The tier is earned by measurement rather than declared by the author, and is the tolerance later re-execution is judged against.

Tier τ	What held for every output
T0	byte-for-byte identical (the two files have the same hash)
T0-meta	identical after removing generation noise (Section 2.2)
T1	numerically equal within relative tolerance 10^{-9} : for each pair of numbers a, b , $ a - b / \max(a , b) \leq 10^{-9}$; both zero match; one zero and one nonzero do not; non-finite and missing values match only themselves
T1-reported	equal to the four significant figures a paper reports: $ a - b \leq \frac{1}{2} \cdot 10^{e-3}$ where $10^e \leq \max(a , b) < 10^{e+1}$, half a unit in the fourth significant place (both zero match; non-finite and missing values match only themselves), so that every pair within 10^{-9} passes and the tiers nest
T2	a numeric or structural difference, not cosmetic
TF	execution did not complete; nothing is certified

covers every output; at T2 a single allowance would either fail every honest T2 replication dataset or mean nothing. A T2 certificate therefore carries the *per-output disclosure*: for each file, how the two closure runs differed. That disclosure is what the closure report records for the file: its two hashes or, for a parsed format, its relative numerical gap (the largest relative difference over the file’s paired numbers). How re-execution reads that disclosure, and why an output the pair disagreed on gets no verdict, is Section 4.2’s. TF admits no re-execution. The four significant figures of T1-reported are a version-1 constant chosen to cover every digit a paper ordinarily prints; published tables commonly report two to four. Agreement at that tier therefore holds every printed value to within one unit in its last place. The tier uses significant figures rather than decimal places because they are scale-free, holding a coefficient near 10^{-3} and one near 10^3 to the same relative standard. Section 3.6 makes agreement at T1-reported the protocol’s own requirement. One edge of the numeric tiers is deliberate: T1’s zero rule makes an exact zero against a rounding residue like 10^{-17} a T2 event. It surfaces at closure, where the pair is measured and the tier records what held, rather than surprising a later reader.

Normalization. Normalization peels off leftover date marks so two honest runs are not called different for cosmetic reasons. Before comparison, strip *generation noise* (marks that do not change scientific content): creation dates, producer strings, log timestamps, and meta-data padding that file formats embed, following the per-format handlers of the reproducible-builds tool strip-nondeterminism (Reproducible Builds Project, 2026b); Appendix C lists every category and how each is stripped.

That list is exhaustive for version 1, and it has to be: normalized bytes are hashed, so an implementation that stripped one mark more than another would compute a different h_o for the same outputs and reject an honest certificate. Every byte not on that list is left exactly as written, whatever it looks like. Normalization decides only whether two runs agreed; h_o is

stored for both raw and normalized bytes, so a disagreement about normalization is visible rather than silent.

2.3. What closing records

Closing also writes what it found, for the certificate to carry: what the scripts reach for, and what state the author’s files were in. In order: the dependency audit; the source state. Which choices belong to the author and which to the protocol, and when certification refuses, are collected in Section 3.6.

Dependency audit. A dependency audit is a search of the scripts for any file they try to read that is not in the replication dataset—the way one searches a `.do` file for `use` of a file that never made it into the zip. Four parts follow: first, what the audit classifies and refuses; second, how a declared dependency is bound; third, the reseeding and capture mimicry it discloses; and finally, why its patterns are the schema’s to fix. Appendix C gives the fetch layer, the library root, and the reseed masks, and says where each pattern family runs; Appendix C gives the capture-path family.

First, classification. Before deposit, classify every file-read reference and every extension of a language *search path* (the list of places the language looks for add-on files: `.libPaths()` in R, `adopath` in Stata, `sys.path` in Python) against the replication dataset tree, into three classes. They are a read that resolves inside P^* , which places that file in the data group d (Section 2.1); a read that matches a declared external dependency (next); and an undeclared reference outside P^* , which stops certification. A search-path extension that names anything but the schema-named library root, or the deposited image’s own, also stops certification (Section 2.1). Reading an undeclared file at run time is an error the protocol *defines* rather than detects: nothing in the run watches file opens (confinement is out of scope, Section 5.1), so a dynamic read the patterns miss is a residual risk, not a caught event. The deployed alternative—tracing every system call, as ReproZip does (Chirigati et al., 2016)—detects such reads, at the price of administrator rights and per-platform machinery, the same price that rules out sandboxes (Section 5.1); version 1 keeps the static audit and declares the gap.

Second, declared dependencies. External dependencies are allowed only if declared as a list of (locator, $H(\text{bytes})$) pairs, where a *locator* is an address that says where to fetch the file (a URL, or a library name and version). Each pair carries the path under the schema-named dependency root at which the author’s code reads the file. The declaration is exhaustive at the level of bytes: every external file the run will read is its own entry. The list lives in the *dependency declaration file*, a schema-named file inside P^* beside the target list, so that h_u and the receipt bind it before the seed exists (Section 2.2). The certificate software itself fetches each entry before the run, requires the bytes to match the declared hash, places them at the declared path inside the scratch copy, and stores them as blobs of the evidence set (Section 3.2). Placed there, they appear in every pre-state manifest, and re-execution rebuilds them like any other file. The author’s code reads the file at that path and never at the locator, since a script that itself reads from the locator reaches outside P^* and is an undeclared reference; the run needs no network. A ready replication dataset also carries its own add-on libraries. In version 1 they are *vendored*, installed inside P^* under the schema-named library root as a project-local library, or they live in a deposited container image

(Section 2.1); in either case the language’s search path points only there. A locator can stop resolving even when its hash is recorded (a library version withdrawn, a repository branch deleted), so a ready replication dataset preserves each declared dependency with itself, in the image where one is deposited. For source code, a locator into Software Heritage, the content-addressed public archive of software, outlives the repository it came from (Di Cosmo, 2020). The fetch layer serves data files, whose declared hash names exactly the bytes the run reads. Fetching and installing a library from a package repository is deferred to version 2, since a library’s locator names its source archive while the run loads the installed files built from it, which the declared hash would not cover.

Third, reseeding and mimicry. The audit reports every match of the schema’s per-language *reseed patterns*, by file and line, recorded in ρ as declared evidence. The patterns are `set.seed` in R, `set seed` in Stata, `numpy.random.seed` and `kin` in Python, and the constructors that draw fresh operating-system entropy rather than reseeding, which bypass the certified seed instead of overriding it; Python’s unseeded `default_rng()` is the common case. Legitimate code reseeds: nearly every stochastic script opens with a literal seed, and per-bootstrap streams are ordinary, so a match is disclosure, not refusal. But such lines are near-universal, so disclosure alone would tell a reader little, and an unconditional reseed after the loading line would make the outputs independent of the archive’s s , reopening the seed shopping that Section 5.3’s contribution 8 exists to make visible (Section 5.1). In R and Python the capture code therefore *re-maps* the author’s reseeds through s . The loading line masks the language’s set-seed call, so that a request for seed k seats the seed derived from (s, k) by the derived-seed rule of Appendix A.2, under a schema-fixed domain string. The derivation starts from the run’s seed s rather than the step’s s_i , so that one literal seed in two scripts yields one stream, as it does in the author’s own runs. That preserves the author’s stream structure while keeping every draw dependent on the archive’s challenge; the audit still records each match. Stata’s built-in `set seed` cannot be masked, so the capture files provide a command, `certified_seed k`, that seats the same derived seed, and closure names the one-line replacement. A raw reseed left in place after the loading line is disclosed and marks the certificate (Section 3.6), a language-specific limit stated as such. The rule covers `set seed` and the `seed()` and `rseed()` options of Stata’s resampling commands, spelled as a literal or a macro, at top level or inside a program, in a target or in any file the run reaches from one. The reseed and capture-path families (the third family, below) run over every text file of P^* and of the image, whatever its extension: a shell script, a Julia file, an `.ado` program, not only the scripts the extension list names. So no text file inside P^* or the image that the run executes carries a reseed the audit did not read. The author’s machine-level startup files (Python’s user site, R’s user environment file, Stata’s personal profile) are outside both, so every target and every launched process runs with the interpreter’s flags that skip them (Appendix C); one those flags cannot reach is the in-run memory edit of Section 5.1, at its stated price. A compiled artifact the audit cannot read is unaudited; ρ carries instead a manifest naming, for each such file, the public registry digest it matches or none, which finding 5 (Section 4.1) recomputes and whose count of unmatched artifacts sits on the metadata field. Code a data file carries into a run (a serialized object a script loads) is a residual Section 5.1 names. A third pattern family covers the capture files. A deposited file that names a capture file’s path (Section 2.2) is disclosed the same way and closing stops (Section 3.6), so a counterfeit Q record cannot come from text the audit read.

A read, by author code, of the process setting through which the run’s seed reaches each step (Section 3.2) is disclosed too, since it would let code tell the certified run from the closure runs.

Finally, the patterns. The reference patterns and search-path expressions this audit looks for are fixed in the version-1 schema (Appendix A), not chosen by each implementation. They have to be: the check re-runs the audit on P^* and requires the same verdict (Section 4.1), so an implementation searching for one pattern more than another would refuse a replication dataset the other had certified. A version 1 reader uses the version 1 set, and ρ names both the protocol version v (Appendix A) and the certificate-software build ($v_{\text{tool}}, h_{\text{tool}}$) that produced it.

Source state. Source state is a snapshot of which version of the files the author had recorded, if they use a version-control system such as Git. If P is under *version control* (a system that records a history of file changes), record the *commit* (the saved snapshot, named by its own hash) and the *branch* (which line of history). Record also the count of *dirty* files (tracked files the author has edited since the last save) and the count of *untracked* files (files version control is not watching), and bind that record by hash. The record is declared evidence, not *re-derived* (recomputed from the tree alone): the later check does not reconstruct Git history from the files.

3. Producing certificates

Once a replication dataset is ready (Section 2), the protocol produces a certificate in three stages, and this section is that protocol. In order: *deposit* the closed dataset and receive a seed the author could not have known (Section 3.1); *run* once under that seed, recording every step (Section 3.2); and *seal*: sign, timestamp, store ρ by its hash, and link it from the published replication dataset (Section 3.3). Three further subsections follow the stages: what happens when the author later changes the files (Section 3.4); how the same stages serve a deposit whose files are restricted (Section 3.5); and finally, what the author may set, what the protocol fixes, and when it refuses (Section 3.6).

3.1. Stage 1: deposit the replication dataset, receive the seed

Depositing hands the closed replication dataset P^* to the archive. Only after the archive has hashed those bytes does it hand back a signed, timestamped receipt that includes the seed s . In order: how the deposit is named; the nonce context; the freshness pulse; the receipt; the required order of events; the software-build check.

Deposit identifier id . A deposit identifier names this exact replication dataset at this edition of the rules: $id = H(\text{enc}(H(P^*), v))$, spelled as the schema fixes (a *renderer*—a program that displays ρ , Appendix B—may display a prefix). The identifier carries no timestamp: the same bytes at the same v *replay* the same id , and with the same declared predecessor the same signed receipt. Whoever re-deposits the same bytes receives the stored D , which keeps naming the first uploader’s account identifier, a pseudonym (Appendix C). The stored

D is served, not recomputed (unless it was superseded for cause, when its successor is, Section 3.1): a fresh computation would draw a new arrival time and pulse, and so a different D . And id names the bytes at a rule edition, not the deposit event: declaring a different predecessor is a new event under the same id (Section 3.4), and the receipt, which signs id^- , is what names the event.

Nonce context γ . The context γ is the four facts the nonce ν is tied to, so that nonce cannot be reused for a different replication dataset. Four fields, and only these: id ; $H(P^*)$; the *archive key* id (a short name for which archive key was used, $H(k_{\text{pub}})$); and the *freshness pulse* β (Section 3.1, which fixes its three fields). The arrival time t_{arr} is deliberately not among them: it stays in D , where finding 9 (Section 4.1) bounds it. The archive’s one free choice in γ is therefore the pulse β , which the same finding confines to the window: about 144 blocks in a day, where an arrival time in γ would have offered every second of it. The trust assumption of Appendix A is that the archive does not shop even those (Section 5.2). Encode γ with enc as an array of those four values in that order, each spelled as the schema fixes (Appendix A). Then

$$\nu = \text{HMAC}(k_{\text{arch}}, \text{enc}(\gamma)) \quad (4)$$

and s is computed from ν as in Appendix A.2.

Freshness pulse β . The freshness pulse takes the archive’s clock out of the story. Every other field of γ says what arrived; the receipt’s t_{arr} says when, but only as the archive’s own assertion, which leaves backdating open; the timestamp proof closes that opening from the other side (Section 3.3.2). The pulse replaces that assertion with a value no one could have known before the public moment it appeared. Its three fields are the name of a public unpredictable source, the identifier of that source’s most recent pulse when the files arrived, and the pulse’s value. In version 1 that value is the height (the block’s position number in the chain) and hash of the newest Bitcoin block that is six confirmations deep (Appendix A.2). The depth keeps a later reorganization of the chain’s newest blocks from orphaning the one named, at the price of a floor about an hour coarser, nothing against the day-wide window of Section 3.1. Because that value did not exist until the block was mined, a receipt containing it cannot have been built any earlier, whoever built it. The run in turn cannot predate the receipt, since the seed comes from it, so the certified run inherits a floor the author cannot touch (the window, with its two limits, is Section 3.3.2). The archive chooses which recent block to name, so the floor the protocol guarantees is not the pulse’s own time but the coherence window C_{time} before the receipt’s timestamp proof (defined with the receipt, Section 3.1). The check requires the pulse to fall inside that window and rejects a receipt whose pulse does not (Section 4.1).

The pulse marks a time; it is not a source of randomness, and that distinction is what makes it safe. A block hash is not neutral—whoever mines a block may discard it and try again, nudging the value—so a seed drawn from one is open to steering. Here ν still comes from $\text{HMAC}(k_{\text{arch}}, \text{enc}(\gamma))$, so the randomness is the archive’s secret and steering the block buys nothing: whoever mined it cannot compute ν without k_{arch} (the economics of manipulating block-hash beacons are quantified by [Bonneau, Clark and Goldfeder, 2015](#); here manipulation buys nothing to spend them on). Reading a pulse writes nothing to any

public record, so no fee attaches; the one cost is that the source must answer at deposit time. As with the timestamp proof, the source sits behind an interface: any service that publishes unpredictable values on a schedule with stable identifiers will serve. Version 1 names one, Bitcoin: an instantiation, not a permanent choice. A *randomness beacon*, a public service that publishes unpredictable values on a schedule, in the style of the National Institute of Standards and Technology satisfies the same interface (Section 5.3). An unreachable source is a refusal (Section 3.6), not a deposit with the field left empty.

Receipt D . The receipt is the archive’s one signed document for this deposit: these files arrived, from this account, under this edition of the rules, and this is the seed. It is issued only after the archive has hashed what it received. The archive signs $\text{enc}(D)$, whose fields carry its statement type (**deposit**) and name the deposit: id , $H(P^*)$, the four group hashes, and the predecessor id^- or its recorded absence (Section 3.4). Then the depositor: uploader account and *identity assurance*, which kind of login sent the files (Appendix C). Then the archive and the protocol: archive name and key id, v , the schema hash h_{schema} of Appendix A, the recognized certificate-software build $(v_{\text{tool}}, h_{\text{tool}})$, the ledger position the receipt’s line takes, and the arrival time t_{arr} , the archive’s own assertion and never evidence on its own. Then the seed $(\beta, \nu, \text{ and } s)$, the closure hash, H of the closure report deposited with P^* (Sections 2.2 and 3.6), and, for a receipt reissued for cause, the supersedes field (Appendix C). Appendix C lists them in order.

D predates every run: it names the seed, not which later run used it. The archive submits $H(\text{enc}(D))$ to the timestamp service (Appendix A.2) the moment it signs, keeps the proof, and appends D ’s ledger line (Section 3.3). The deposit is thus bounded from both sides: after the pulse named in β , and no later than the moment that proof confirms. The check compares both bounds against the arrival time they sandwich (finding 9, Section 4.1). The whole span must fit inside the *coherence window* C_{time} , fixed at 24 hours, which is why the guaranteed floor is stated as a day before the receipt’s timestamp rather than as the pulse’s own time. The window also obliges the archive to act in time. The public calendars (the servers that batch hashes into Bitcoin, Appendix A.2) commit on their own schedule, and an honest deposit whose proof confirmed late would fail finding 9 permanently. So a receipt whose proof would confirm outside the window must be re-submitted under the archive’s own transaction fee rather than left to lag. By rule the archive broadcasts a Bitcoin transaction of its own, or a purchased one, if no Bitcoin attestation exists twelve hours after issuance. Those twelve hours are the *attestation deadline*, a version-1 constant that leaves the confirmation time to fit the window. Should a proof nonetheless confirm outside the window, the archive *reissues for cause*: a fresh receipt for the same bytes under a fresh pulse. Its signed *supersedes* field names the dead receipt by its ledger document hash ($H(\text{enc}(\text{doc}))$) of its ledger entry, Section 3.3) and its confirmed time, and its bundle carries the dead receipt with its proof, so that finding 9 checks the cause (Section 4.1). The author re-runs under the new seed, and the abandoned receipt counts on the *landing page* (the archive’s page for a published version, which carries the certification metadata field, Appendix B) as the archive’s fault, not as a receipt the author walked away from. The submission is one more leaf in the calendar’s Merkle tree (Appendix A.2), whose root is published regardless, so it adds nothing to the fee.

Order. Receipt, then the run; anything else fails. The archive hashes, then signs D and appends D 's ledger line (Section 3.3), then the run uses s from that D . The run needs no archive: once the author holds D , the archive need not answer again until the certificate is submitted, and no deadline runs between the two. A receipt from another deposit, or a seed that ν does not yield, fails the later check, not as a matter of duty but of arithmetic. Finding 6 (Section 4.1) recomputes $H(P^*)$ and the deposit identifier from the files being checked and requires both to appear in D , and recomputes s from the ν that D signs. So a seed issued for one tree cannot be used for another.

Certificate-software build. The archive will not sign a build of the certificate software it does not recognize. The software that produces ρ hashes its own installed source the same way a group hash is computed: sorted relative paths and file contents, excluding generated *intermediate* files of the language runtime (scratch files the language writes while running). The exclusion list is the schema's, not the implementation's, for the same reason as every other pattern family. The software sends the pair $(v_{\text{tool}}, h_{\text{tool}})$, tool version and source hash, with the deposit; κ inside ρ carries the same pair. The archive compares the pair to the builds it publishes, refuses an unrecognized hash or a build whose recognition has lapsed, and signs a recognized pair into D . A build stays recognized for a stated overlap after its successor is published—version 1: ninety days—so a run that straddles a release is not refused for finishing. Which builds are recognized is governance rather than protocol, and it is the closest thing in this design to a central authority. Version 1 requires that the list be carried in the archive's signed key log (Appendix B), so it is public and dated by the anchors, and that a recognized build be a reproducible build whose hash at least two independent builders have published. Archives may maintain the list jointly. This comparison stops a stale, damaged, or foreign build. It cannot stop a modified certificate software that reports the expected hash, because a program cannot establish its own integrity when the modification can also change the reporting code.

3.2. Stage 2: run once under the archive's seed

The certified run is the one official execution: once, under the archive's seed s , with every step written down. Numbers a paper reports must be taken from this run. P^* runs in a *scratch* copy (a temporary working copy, not the stored original). The certificate software does not select among repeated attempts, and the build that runs is the build closing recorded in κ ; a later build re-closes first. In order: the pre-state taken before each step; the record each step leaves; the capture of estimates; the trace that binds it all.

Step i and pre-state M_i . A *step* is one target (Section 2.2), run as its own *process* (one launched program). The certified run executes the targets in order, one process each, and any script a target calls runs inside its caller's step; there is no finer interception. A script a target launches as a separate process runs inside the caller's step too, under the caller's s_i , seeded, masked, and captured like its caller. The means is a *startup hook*, a routine the language runs when a process starts, which the certificate software installs in the run's environment (Appendix C gives the mechanism). A launch whose flags or cleared

environment would skip that routine (R’s `-vanilla`, say) is a refusal at closure (refusal viii, Section 3.6). A process the hook cannot reach, a compiled program, marks the certificate when the step’s process sampler (below) observes it (Section 3.6); one that exits between two samples is the sampling residual of Section 5.1. Step granularity is therefore exactly the author’s top-level structure: a replication dataset driven by one master script is one step, re-executable only whole, while one split into several top-level scripts can be spot-checked script by script. Write i for the step’s *index* (its place in the order), starting at 0. Before the script starts, the certificate software records the pre-state *manifest* M_i : the written list of (path, $H(\text{bytes})$) for the file tree, one pair per file, a directory listing with hashes rather than a camera image. The manifest is summarized by a Merkle root R_{M_i} over those entries, computed by the Merkle rule of Appendix A.3 under the domain string `certified-manifest-v1`, entries path-sorted, each pair a leaf spelled as the schema fixes (Appendix A). Each M_i is kept in full in the evidence set (Section 3.6), paths and hashes only, so the manifests are small. Each is stored under its own root R_{M_i} , so the root is also the lookup key; that is what later re-execution rebuilds a step’s tree from (Section 4.2). M_0 must equal the manifest of a fresh scratch copy of P^* with the declared dependencies placed at their schema-named paths (Section 2.3) and nothing else, no extra file and no missing file. So the recorded run began from the deposited bytes, and the check recomputes that starting point rather than trusting it (finding 2, Section 4.1).

The software then launches the script’s process under the frozen settings (Section 2.2), with the scratch copy’s root as its working directory so relative paths resolve from the tree root. It hands the process the step’s seed $s_i = \text{derive}(s, i)$, the seed derived from s and the step’s index under the schema’s per-step domain string (Appendix A.2). So two steps that both simulate draw independent streams while every draw still depends on s , unless both request one literal seed, when they share a stream as the author’s runs do (Section 2.3). The seed travels as two schema-named process settings, the run’s seed and the step’s index. From them the loading line derives s_i and seats it by the language’s own set-seed call, and the mask derives every re-mapped reseed; the closure runs do the same from s_0 . While the step runs the software records the *console transcript* L_i (everything the script printed, the way a log file does). Every $\Delta = 50\text{ ms}$ (50 milliseconds; twenty times a second) it samples the list of running programs descended from the step’s process, never the whole machine, whose other command lines are nobody’s business. And it hashes every file the step wrote. A failed step aborts certification: a non-zero *exit* (the script’s report that it failed: 0 means success in Unix convention) or, for a runtime whose batch mode returns zero regardless (Stata), the schema’s per-language error test on the log.

Step record r_i . A step record is the written log of that one script: what ran, what it wrote, and what programs were seen running. Fields: index i ; script path; language; exit code; start time, end time, duration in milliseconds; R_{M_i} ; before-and-after hashes of every written file (read off the adjacent manifests); $H(L_i)$; and the hash of the step’s capture file (Section 2.2). The other fields: the list of *observed* processes, programs seen running, each with process id, parent id (the program that started it), executable path and $H(\text{executable})$, command line, first and last seen; and (sampler, Δ) (which sampler was used, and how often). Process records are labeled *observed*: a process that starts and exits between samples is missed.

And executable hashes do not cover *dynamic libraries* (code loaded while a program runs) or *interpreted libraries* (add-on files a language reads at run time rather than compiling into the program: R packages, Python modules, Stata .ado files).

Result capture Q . Result capture copies the numbers out of the statistical runtime itself, through the capture code added at closing (Section 2.2), so they do not live only in a table someone could retype. After each *estimation* command (a command that computes a statistical result: `regress` in Stata, `lm()` in R), read the runtime’s stored results and write them to the step’s capture file in one fixed form. The results read are the *coefficients* (the estimated numbers), the *standard errors* (how uncertain those numbers are), the sample size, and the command line. The capture file is a schema-named path inside the scratch copy, hashed and stored like any other written file, and so a member of o (Section 2.2). The fixed form is a single JSON object per estimation, each number spelled exactly rather than rounded by a formatter (Appendix C) wherever the runtime’s stored results are read from memory. Stata offers no hook after a command, so there the capture parses the log the run writes. It reaches only estimations whose results the log shows, at the digits the log prints; a *quietly* estimation, or one whose stored results a later command overwrote before the parse, is uncaptured. Closure names each such command so the author can route it through the author-callable capture command (Section 2.2), which reads the stored results exactly. Two Stata particulars follow. A record inside a loop body carries the loop’s echoed line, so its command line does not distinguish iterations; its dependent variable and coefficient names do (Section 2.2). A *bootstrap* prefix hides its replicates from the log while an R `boot` call fires the hook once per replicate, so the record count in ρ is not comparable across the two languages. From the capture files write a sequence Q of those objects, in run order, in the sequence form the schema fixes; an empty Q is the empty byte run. The later check rebuilds Q from the stored capture files and requires a byte-for-byte match. Capture is required for every language the replication dataset uses. Which commands count as estimation commands is fixed per language by the version-1 schema (Appendix A), since ρ records how many estimation records Q holds and two implementations working from different lists would disagree on that count. The lists cover the standard post-estimation families (marginal effects, predictions, linear and nonlinear combinations), since those are the numbers tables print; a quantity outside them reaches Q only through the author-callable capture command (Section 2.2). Figures left in memory are exported as files and enter o .

Trace T . The trace is the ordered list of all step records,

$$T = (r_0, \dots, r_{n-1}), \tag{5}$$

with Merkle root R_T (the Merkle rule of Appendix A.3): one hash that stands for the whole run log. Altering any record, any recorded output, or the tree shape (how many leaves, and which pairs were hashed together) changes R_T . After the last step the software records a final manifest M_n by the same rule as the pre-states, with its root R_{M_n} . The outputs and each step’s *write-set* (the files it wrote) are then derived from the manifests rather than reported by the run. o is the set of files in M_n that are new or changed relative to M_0 ; step i ’s written files are the differences between M_i and M_{i+1} ; and a write that leaves a file’s

bytes identical counts as no write. A file written then deleted is absent from o , recorded in the intermediate manifests if it survived a step boundary, invisible if it came and went within one step. Written files are stored as content-addressed *blobs* (raw stored files named by hash), up to a per-file size ceiling C_{blob} , fixed at 200 *megabytes* (a megabyte being 10^6 bytes, so 2×10^8 bytes per file), a fifth of C_{size} . Above it only the hash is kept, capture files excepted, since finding 4 (Section 4.1) rebuilds Q from them. So a later check can reconstruct a recorded pre-state and re-execute that step only when every file in M_i was stored as bytes. *Re-execution coverage* is the fraction of the n steps with a complete pre-state: every file in M_i available as bytes, not only as a hash, whether as a stored blob or, for files unchanged since deposit, in P^* itself (Section 4.2). The fetch layer stores each fetched dependency at or below C_{blob} as a blob of the evidence set, and a declared dependency counts only then, since a locator can die (Section 2.3). If n_c of the n steps qualify, re-execution coverage is n_c/n , stored as the two integers n_c and n . It is computed at issue from what was stored; it does not name which steps a later reader re-runs.

3.3. Stage 3: sign, timestamp, and store the certificate

This stage turns the finished run into a certificate ρ , the witness W (the archive’s post-run signature), a timestamp proof π (Appendix A.2), and the ledger’s W line (D ’s line was appended at deposit, Section 3.1). First the documents stored with the run; second, the window those documents pin around the run; and finally the public ledger and the anchor that fixes its head (storage and crash recovery are the archive’s, Appendix B).

3.3.1. The documents

The certified run yields three documents—the certificate itself, the archive’s signature over it, and a public time mark on its exact bytes. The unit that closes this part states what the seed and the time mark together establish: the window around the certified run.

Certificate ρ . The certificate is the finished record: every hash and list from the certified run, in one document, each field defined where its object was introduced (Appendix A and sections 2, 3.1 and 3.2), and Figure 3 is its one full statement, an example with every field annotated. Three fields are new here. First, the full receipt D exactly as signed, with ν and s ; its timestamp proof is stored beside ρ (Section 3.6 and Appendix B). Second, the output *inventory*, counts of output files by kind under the schema’s kind map (its rule from a file’s extension to a kind: figure, table, log, data). Finally, the list of written files that exceeded C_{blob} and are kept only as hashes. ρ cannot prove on its own that it is genuine: genuineness is the witness W ’s job, and π fixes only when the bytes existed.

Witness W . The witness W is the archive’s later signature saying: this finished certificate is the run we saw. After receiving ρ , the archive signs: statement type **witness**; $H(\text{enc}(\rho))$, which binds everything inside ρ , the receipt included; archive identity; protocol context (the pair (v, h_{schema}) and the key id, Appendix A); the ledger position W ’s line takes; the account that submitted ρ (Appendix B); and the signature time. Every signed object carries its type, the same discipline as the domain strings. Nothing ρ already holds is restated. W does not

```

{
  "protocol_version": "1",
  "schema": "9e4c...77",
  "commitment": [
    "ab40...9e",
    "17c9...0d",
    "e803...77",
    "9c41...5b",
    "6f2d...c4"],
  "commitment_hash": "20fa...91",
  "final_manifest_root": "aa31...9d",
  "predecessor": null,
  "environment_capsule": {...},
  "source_state": {...},
  "capture_code": {
    "code_hash_pre": "3d18...7a",
    "touched": {"code/clean.R":
      ["9b02...c1", "4e77...30"], ...},
    "added":
      ["code/certified_capture.R"]},
  "closure": {
    "report_hash": "5e21...aa",

    "tier": "T0",
    "exception_mark": [],

    "seed_sensitivity": {...},
    "sensitivity_summary": {...},
    "deposit_receipt": {...},

    "nonce": "77b0...13",
    "seed": "1583296741",
    "targets": ["code/clean.R",
      "code/estimate.R", "code/figs.R"],
    "timeouts_ms": {"code/clean.R": 3600000, ...},
    "trace": {
      "n": 3,
      "root": "c9d4...3e",
      "steps": [
        {"index": 0,
          "script": "code/clean.R",
          "language": "R",
          "exit": 0,
          "start": 1789000000,
          "end": 1789000041,
          "duration_ms": 41250,
          "pre_state_root": "b2ac...77",
          "wrote": {"data/clean.csv":
            ["9f31...c8", "40cc...19"]},
          "transcript_hash": "88d1...b5",
          "capture_file_hash": "1c0a...e2"},
        {...}, {...}],
      "outputs": {
        "normalized_hash": "b7aa...02",
        "files": {"figs/fig1.pdf": "d0e7...4f", ...},
        "inventory": {"figure": 2,
          "table": 3}},
      "estimates": {
        "count": 4,
        "hash": "3af2...96"},
      "dependency_audit": {
        "external": [],
        "reached": [],
        "undeclared": 0,
        "reseeds": [],
        "artifacts": []},
      "reexecution_coverage": [3, 3],
      "oversize": []
    }
  }
}

```

<- line breaks and readable key order
 for display; enc(ρ) has neither
 <- v: which edition of the rules
 <- h_schema: the v1 schema file
 <- x: the five group hashes (readiness)
 <- h_c: code
 <- h_d: data
 <- h_u: other inputs
 <- h_e: environment capsule kappa
 <- h_o: outputs, raw bytes
 <- h_x = H(enc(x)): one name for it all
 <- R_{M_n} (Stage 2)
 <- id~ as signed in D; null = none
 <- kappa; h_e is the group hash over it
 <- commit, branch, dirty/untracked counts
 <- h_c~: code as the author supplied it
 <- each target: before, after
 <- files the capture code created
 <- H of the closure report (two
 closure runs); always present
 <- tau, measured; the tier table
 <- the marks that hold, in schema order:
 disclosed-T2, unmasked-reseed, unhooked-process
 <- continuous two-seed movement rows
 <- state, coverage, max SE-scaled movement
 <- D in full, archive-signed (Stage 1):
 id, H(P*), group hashes, predecessor,
 depositor, t_arr, ledger position,
 closure hash, pulse, nonce, seed, ...
 <- nu, as issued in D
 <- s, derived from nu by the public rule
 <- targets, in execution order
 <- per-script cap (sec. 3.6)
 <- T: one record per step (Stage 2)
 <- step count
 <- R_T: Merkle root of the records
 <- 0 = success; anything else aborts
 <- Unix seconds
 <- R_{M_0}: every file, hashed, before
 <- before- and after-hash
 <- H(L_0): console output
 <- H of this step's capture file
 <- observed programs; sampler, Delta
 <- h_o after stripping generation noise
 <- per-file output hashes
 <- counts of outputs by kind
 <- estimation records in Q
 <- H(Q); Q is rebuilt at check time
 <- declared (locator, hash) pairs
 <- what the fetch layer reached (App. C)
 <- must be 0
 <- matched reseed lines (finding 5)
 <- image/library files: registry digest
 <- n_c and n: every step replayable
 <- files kept only as hashes (> C_blob)

Figure 3. A certificate ρ , annotated. Hashes are truncated at each end; {...} elides contents covered in the named sections. Field names are the version-1 schema's (Appendix A); the display adds line breaks and readable key order that $\text{enc}(\rho)$ does not have; a T2 certificate adds the per-output disclosure (Section 2.2). W and π are issued over $\text{enc}(\rho)$ after ρ is fixed, so they cannot appear inside it.

bind that the outputs are scientifically correct, that the code is honest, or what happened between process samples. W is timestamped indirectly: its ledger line, whose document is W with its signature, falls under the next anchored head (Section 3.3.3).

Timestamp proof π . The timestamp proof puts a public mark on the certificate’s exact bytes, so that nobody—the archive included—need be trusted about when ρ came into existence. Compute π (Appendix A.2) over $\text{enc}(\rho)$ at issue. The archive’s later pass (Appendix B) gathers confirmation on the public Bitcoin record and attaches it. A ρ too new for the Bitcoin record to have confirmed it may still be **pending**, which is not a failure.

Who signed. One signature carries the certificate: the archive’s witness W , checked under the archive’s trust root (Appendix A.2). Who the depositor is comes from the archive’s lookup of which login sent the files, recorded in D as *identity_assurance* (Appendix C). The value is a shared login, a person registered with the archive, or a person who signed in through ORCID, the Open Researcher and Contributor ID, a researcher’s persistent public identifier. No author signature is part of version 1: a key that arrives with the files binds nothing beyond what the deposit already binds, and one published out of band is a step nobody takes (a version-2 candidate). An author who wants to sign may place a signature file in the tree, where it is a file like any other, hashed and bound; the protocol reads nothing into it.

3.3.2. The window around the certified run

The seed and the timestamp proof together pin the certified run between two moments the author controls neither of, and this is the argument the whole design rests on. Two parts follow: first, why fixing a date takes a public record no party controls; and finally, the window itself and its two limits.

First, the record no party controls. The timestamp proof settles when the certificate existed without anyone’s clock having to be trusted: not the author’s, not the archive’s, not the journal’s, not a later reader’s. The time a reader obtains is instead a Bitcoin block’s, taken from a public record that no party to the certificate controls. Some such outside source is needed, because the one date the archive writes into a signed document, the arrival time on the receipt, is its own assertion. So a stolen signing key, or an archive willing to oblige, could produce a complete and internally consistent certificate at any later date and give it whatever dates suited. The proof forecloses that in the one direction that matters. To hold a proof confirmed in a 2027 block, the certificate’s exact bytes must have existed in 2027. No amount of cooperation between an author and an archive produces such a proof in 2029, because it would mean inserting data into a block that was mined two years earlier. A certificate can therefore be shown to be at least as old as it claims to be.

Finally, the window. The proof matters when a question arises years later: whether the analysis predates the objection, the failed replication, or the competing paper it stands accused of having followed. What it establishes is a deadline—the certificate existed no later than the moment the proof names—and that deadline is the later edge of a window around the certified run.

The earlier edge comes from the seed, and it is not a clock reading at all. The archive does not produce the nonce until it has received the deposit, and the seed is derived from that nonce, so an analysis that ran under that seed must have run after the deposit. What the seed fixes is therefore an order of events rather than a date. The date is supplied by the receipt’s freshness pulse (Section 3.1): the archive records inside γ a recent pulse of a public unpredictable source, and since that value did not exist beforehand, the receipt—and so the run—cannot have been made earlier. Which recent pulse is the archive’s choice, within the coherence window the check enforces (Sections 3.1 and 4.1), so the earlier edge the protocol guarantees is one day before the receipt’s own timestamp proof—a block time, like the later edge. Taken together, the certified run happened no earlier than a day before one public moment and no later than another, and neither moment can be revised. Two qualifiers. The earlier edge binds the run’s outputs (Section 3.2) only insofar as they depend on the seed. An analysis with no random draws could have been run before deposit and replayed under the certificate software. An author who computed every deterministic part before deposit and ran only the seeded remainder after it has a certificate that dates the record and the seeded draws, not the substantive work (Section 5.1 restates this). And the later edge dates the certificate’s bytes, not the moment the analysis finished. Neither the author nor the archive controls either edge: both are the times of blocks in the public record. A confirmed proof also outlives the archive: if the archive’s signing key is later compromised, or the archive itself disappears, the proof still shows that this certificate existed while that key was good.

3.3.3. The public ledger and its anchor

How the archive stores, serves, and survives crashes—and what happens when another archive takes over—is collected in its checklist (Appendix B). Defined here is the public record those duties feed: the append-only Merkle log the archive maintains, the signed head that names it, and the anchor that places that head where the archive cannot revise it.

Ledger entry E_i . A ledger is an *append-only* public list (it can only grow) whose entries are the leaves of one Merkle tree (Appendix A.3, under its own domain string `certified-ledger-v1`). So the tree’s root at any size names the whole list up to that point, and a later root that does not extend an earlier one shows. Write E_i for entry number i (starting at 0) and t_i for its integer time. Write kind for which issuance this line is: D ; W ; a key-log entry K ; or P , the publication of a version carrying a certification field, whose document is that field’s contents (Appendix B). Write doc for the document on that line, which the archive stores content-addressed by its hash: for every kind, the statement together with its signature, so that the anchored head dates the signature and not only the statement. Entry i is

$$(i, \text{kind}, H(\text{enc}(\text{doc})), t_i), \tag{6}$$

encoded with enc as an array of those four fields in that order, spelled as the schema fixes; it carries the document’s hash, not the document, so an entry is a few hundred bytes. Each certification appends three entries: D ’s at deposit, W ’s at seal, and P ’s at publication. Appends are taken under a *lock* (only one writer at a time) that binds the position a signed document names to the position its line actually takes; D and W name theirs, K and P do

not. So no other issuance can slip between the read and the append. The ledger makes the archive’s history *tamper-evident* (an edit changes every later root). Its tree gives two short proofs, both RFC 6962’s and each a handful of hashes: an *inclusion proof* that a named entry is in the tree of a given root, and a *consistency proof* that one root’s tree is a prefix of another’s. The ledger dates ρ only indirectly, through the head covering W ’s line, which finding 8 falls back on when a calendar strands π ; π is the direct mark.

Anchoring the ledger head. A hash tree shows that the archive has not edited its own past. It does not show that everyone is being shown the same past. Three parts follow: first, the split view a tree alone cannot catch; second, what the signed, anchored head and the stored proofs add; and finally, the two limits that remain. First, the split view. Nothing so far stops an archive from keeping two ledgers, each internally flawless, and serving one to some readers and the other to others—a *split view*. No reader detects it alone, because each sees a tree whose every proof checks out and neither tree was ever edited. Catching it would mean comparing one’s own copy of the head against somebody else’s, which is what it means for observers to *gossip* the latest head (copy it among themselves the way independent newspapers reprint the same last headline). This protocol does not require anyone to do that.

Second, the head and the proofs. At a fixed interval (version 1: one hour) the archive signs a *head*: a statement of type **head** naming the tree’s size, its root, the time, the archive’s identity, and the protocol context. It sends H of the head’s encoding to the same timestamp service that marks certificates (Appendix A.2), one more leaf in a tree whose root is published regardless, so this adds nothing to the fee, and keeps the proof. And each certificate’s stored documents carry its own place on that record: for each of its D and W entries, an inclusion proof under the first signed, anchored head at or after it, with that head and the head’s timestamp proof. The archive attaches them in its later pass (Appendix B); until then finding 10 reports pending, as π does. What this buys is evidence that travels. A reader checks the certificate’s membership from the stored documents alone, with no archive to ask. Two signed heads of the same size with different roots convict from the two documents alone. For two of different sizes the archive must serve a consistency proof that the larger extends the smaller; a proof that fails, or a refusal to produce one, is the demonstration. An archive can still anchor two forks, since the timestamp service takes any hash from anyone. But any two certificates whose stored heads conflict now document the split whenever they are compared. And an archive’s refusal to produce a consistency proof between two of its own heads is attributable misbehavior of whoever holds its key.

Finally, two limits remain. A reader alone detects a split only against a head from a source the archive cannot later revise. In version 1 that source is the ledger snapshots the archive deposits under their own DOIs, and the mirrors other archives keep of them (Appendix B). A head may also carry cosignatures from independent parties, cosigners rather than the witness W ; a cosignature under a key the reader has not pinned is ignored, never failed. The cosignature slot is optional, costs nothing when empty, and lets a consortium add the standard defense against split views without a version bump. And anchoring is periodic, so a fork that opens and is abandoned inside one interval is caught only when a certificate issued on it is later checked against the surviving tree, which it then fails. The anchor itself is evidence about the archive’s history rather than about any one certificate; what the findings

consult is the certificate’s own proofs. Finding 10 (Section 4.1) requires both entries to be present under signed, anchored heads and, when a reference head is available, consistent with it. That requirement is what makes a certificate the ledger never issued fail rather than float free.

3.4. Revising a certified replication dataset

A revision is a later, different replication dataset that the author wants linked to an earlier certificate. Unchanged files are not a revision: re-certifying unchanged bytes with the same declared predecessor replays the same id , D , and s (a D superseded for cause replays its successor), because a deposit is named by a hash of its contents. Declaring a different predecessor is a new deposit event and draws a new receipt, whose seed is new only if the pulse has moved on, since γ omits the predecessor (Section 3.1). Re-certifying changed bytes produces a new id and a new s . No stable identifier survives a revision, so the author declares the link and the archive confirms it.

Predecessor id^- . The author names the earlier deposit id^- their new certification revises. Before signing D , the archive checks that id^- exists here and belongs to the same uploader, or, for a predecessor another archive holds, signs it as an unresolved link the landing page shows as unconfirmed (below). The certificate ρ carries the archive-signed predecessor, not the value the author sent. A certificate whose predecessor disagrees with D fails finding 6 (Section 4.1); that the predecessor belongs to the same uploader is the archive’s duty before signing, not something a reader can recompute.

Version chain. A version chain is the author’s own growing list of deposits, each line tied to the line before it. Per uploader, an append-only version chain records

$$(H(P^*), id, H(\text{previous entry})). \quad (7)$$

It is a display the archive derives from the D documents on the ledger, not a signed object of its own; its length is displayed on the landing page as an integer certified-version count. Published history is derived only from archive-signed predecessor links; titles and display names do not determine which certificates belong to the same work. Two certificates that name the same predecessor are both listed as current. A predecessor held by another archive is an unresolved link (named but not yet confirmed here). Fetching history at view time leaves an already-published page byte-identical, so a paper that cited that page still cites the same bytes.

3.5. Confidential data: certified in full, locked in storage

Confidentiality restricts who may read the files, not whether the run can be certified. A replication dataset the archive holds under *restricted access* (stored and hashed by the archive, served only to readers it has approved) is a deposit in every respect, and its certificate is public. Four parts follow. First, what is published and what stays locked; second, two rules

that protect the restricted bytes; third, who can run which part of the check; and finally, the author’s disclosure duty and the one kind of data outside certification.

First, what is published. The archive receives the bytes, hashes them, issues the seed, and signs and timestamps the receipt and witness exactly as for any deposit (Sections 3.1 and 3.3). The certificate carries no data, only file names and hashes, the environment capsule, the step records, and the captured estimates. So ρ , D , W , π , and the ledger entries are published and linked from the version’s metadata as usual, while P^* and the evidence set stay behind the access control. Transcripts and blobs can contain data, so they are restricted with the files, never published with ρ .

Second, two rules protect the restricted bytes themselves. One: every per-file hash a public document carries is *salted* for restricted deposits, so that an approved reader still verifies while an outsider cannot test a guess—an unsalted hash of a guessable file is a confirmation oracle, the side channel long known from storage deduplication (Harnik, Pinkas and Shulman-Peleg, 2010). The salts derive from one random file R that the deposit adds to the other-inputs group; because R is inside the tree, it also covers the whole-object hashes the protocol’s own equations cannot salt (Appendix C). The other: the captured estimates are published as a standalone Q stored beside ρ and bound by the $H(Q)$ already inside it, released only after the disclosure review required of the author (the fourth and last part of this subsection). Where output-checking rules forbid an exhaustive release, they are published as a *redacted* Q in which each unreleased value is replaced by its salted hash, the full Q staying in the restricted evidence set. The standalone Q file is salted like any other, so $H(Q)$ confirms no guess. For a restricted deposit ρ likewise publishes only the sensitivity summary: state, coverage counts, and the largest movement in baseline-standard-error units where it is defined (Section 2.2). The per-estimate movement rows, which would republish withheld values, stay in the restricted evidence set, where finding 5 recomputes them for an approved reader. Without the second rule, the comparison of the article’s numbers with the certified estimates, promised to any reader by the optional comparison of Section 4.3, would require access to the restricted capture files.

Third, the check: what changes is who can run which part of it. Findings 7–10 (Section 4.1) and the signature and seed checks of finding 6, which read only the signed documents and the public record, run for anyone, as does the comparison of the article’s numbers with the certified estimates (Section 4.3). The findings that recompute hashes over the files (1–5 and the rest of finding 6), and any re-execution, run only for a reader with access. A reader without it reports them as unevaluated, never as passed (Section 4.1). The journal’s existence check (Section 4.3) needs no files and is unchanged.

Finally, one duty falls on the author: the certificate is a publication, and every estimate it captures, including estimates from listed commands the article never reports, gets the same disclosure review as the article’s tables. The author must anticipate that review at closure, since capture creates records (an intermediate regression on restricted microdata, say) that the article never contemplated. The file paths and dependency locators ρ publishes are part of that review, since a path or a locator can name a dataset; a restricted deposit may use schema-named pseudonymous paths. Data the archive may not hold at all has no route in version 1: data confined to a provider’s enclave (a federal research data center, a national statistics office) is outside certification entirely. Certification as specified requires an archive that may hold the bytes, and enclave-held work remains the province of accredited

re-execution (Pérignon et al., 2019). How much of a discipline that excludes is a fact about the discipline. At the American Economic Association’s journals a majority of recent papers use restricted data held under agreements that forbid deposit with a third party, most of it licensed rather than enclave-held, so for those papers the data editor’s re-execution survives (Vilhuber, 2024). At the leading political-science journals most quantitative articles use public data, and there certification covers the field.

3.6. The author’s choices, the protocol’s requirements, and its refusals

Every choice in production belongs either to the author or to the protocol, and the protocol says in advance when it will stop. In order: what the author may set; what the author may not set; the evidence set, which every certificate must carry. Then the exception mark, which names the three departures from the protocol’s default, and finally the refusals, each with a named cause.

What the author may set. The author controls the following, and every one of these choices is written into the certificate: dataset contents; s_0 ; per-script *timeout* (how long a script may run before it is stopped); and the target order (Section 2.2). A stopped script counts as a non-zero exit and aborts certification (Section 3.2); the default timeout is three times the longer of the two closure runs, and never under an hour. Where the statistical runtime searches for add-on libraries is not a choice: the schema-named library root inside P^* , or the deposited image’s (Section 2.1). The size ceilings are not on this list: C_{size} and C_{blob} are fixed by the protocol version, so that one author’s tree cannot be judged by a looser rule than another’s.

What the author may not set. The following are set by the protocol, not by the author, which is what makes them worth anything as evidence. They are the receipt and therefore s ; the archive’s arrival time; τ (earned by the two closure runs; the author does not pick the tier); the dependency-audit result; whether the archive signs the finished run; the trace T (Section 3.2), once the run has started.

The evidence set. A certificate is issued only when the *evidence set*, everything the check reads beyond the signed documents and the closed replication dataset itself, is stored with ρ alongside P^* . It comprises the closure report, every transcript L_i , every manifest $M_0, \dots, M_n$ (the pre-states and the final state), the blobs recorded as stored, the capture files and fetched dependencies among them (Section 3.2), the container image when κ names one (Section 2.1), and, for a restricted deposit, the salts (Appendix C). The requirement is not the determinism tier τ (Table 1): τ records how closely the two closure runs agreed, while the evidence set is what lets a later check re-derive τ and the *closure hash* (H of the closure report) at all. Without it the check as specified could not run, and signatures and a timestamp on a record nobody can check would not be a certificate. So the archive stores the evidence set with P^* and signs the witness W (Section 3.3) only when it holds both. A deposit without them

receives no certificate, and, lacking the closure report, no receipt (Section 3.1). A journal therefore has nothing to require here: every certificate carries its evidence.

Exception mark. A certificate attests agreement at T1-reported or better, so that any four-figure value a paper prints differs from a re-run’s by at most one unit in its last place; the tier recorded in ρ says how much better (Table 1). The protocol fixes that requirement in advance so that no journal has to choose a tier. Three departures from it exist, each disclosed in ρ and marked on the metadata field as an *exception mark*, so that a journal’s default look-up refuses it (Section 4.3). First, a disclosed-T2 certificate (refusal ii below), the one route that bypasses the ordinary stop. Second, a certificate whose audit matched a reseed after the loading line that the capture code could not re-map, in any code the certified run executes (Section 2.3), the one disclosed way to disconnect the outputs from s . Finally, a certificate whose certified run launched a process the startup hook could not reach (Section 3.2), the one way computation escapes the audit’s reading. The mark is the schema’s function of the tier, the audit record, and the process observations (a certificate meeting more than one departure carries the list). The certificate software writes it into ρ , the archive recomputes it before signing W and writes its own value onto the metadata field at publish (Appendix B), and finding 5 (Section 4.1) recomputes it again. So what a journal reads is a value any reader re-derives. A journal accepts a marked certificate only by an exception stated in its policy in advance.

Refusals. Certification stops, with a named cause, in these cases. (i) An undeclared external reference exists. (ii) The two closure runs reach T2 on the *ordinary route* (close the replication dataset, then deposit and certify; the author is better placed to fix the cause than a reader is to discount it later). A deliberate *disclosed-T2 route* (certify a replication dataset that was closed earlier, without treating T2 as a stop) may issue a T2 certificate, which then carries an exception mark (Section 3.6). (iii) A deposited script names a capture file’s path (Section 2.2). (iv) The archive is unreachable at deposit or at submission of ρ (never during the run, which needs only D). (v) The public source of the freshness pulse is unreachable (Section 3.1). (vi) The closure report, P^* , the rest of the evidence set, or the container image κ names is not deposited (Section 3.6). (vii) A closure run wrote a path spelling the current date or time (Section 2.2). (viii) A target launches a process with a flag or environment that drops the runtime’s startup hook (Section 3.2). There is no *offline* mode (issuing a seed without the archive): a local seed would give the certified run no more assurance than the closure runs. Certification can also stop elsewhere, each named where it occurs: at preflight, at a non-zero exit during the run, at an unrecognized software build, at a storage mismatch, or at the archive’s gate before it signs W . That gate stops a second ρ for one receipt, a ρ from an account the depositor’s did not authorize, or a ρ that fails any finding the archive can evaluate then (Appendix B).

4. Checking certificates

Checking has three parts, in order: recompute the certificate from the files without running the analysis (Section 4.1); optionally re-run some recorded steps (Section 4.2); and the

journal’s own checks, which are existence checks, not computations (Section 4.3). The first two are the same for everyone, and any reader can run them; the third is why the journal needs no one to re-run anything. A reader’s procedure is short: fetch the files from the version-specific DOI, follow the metadata link to ρ , hold the archive’s trust root and one Bitcoin block hash (Appendix A.2), and run Verify, re-running steps if wanted; a timestamp still pending is not a failure. Two units precede the three parts: the check Verify itself, and what a pass means.

The check Verify. The check anyone can run recomputes the hashes from the files and sees whether they match the certificate—the way one recomputes a table from the data rather than trusting the printed numbers. Write $\text{Verify}(\rho, P^*, \theta, k, \omega)$ for that check. The five arguments are the certificate, the closed replication dataset, the pinned pair θ (the archive’s trust root and one Bitcoin block hash, Appendix A.2), how many steps to re-run, and whether to consult live sources. $k \in \mathbb{N}_0$ is the number of steps to re-execute, where $\mathbb{N}_0$ is the non-negative integers $0, 1, 2, \dots$; bare k always means this count, while keys stay subscripted: $k_{\text{pub}}, k_{\text{priv}}$. $\omega \in \{\text{offline}, \text{online}\}$ controls whether live sources are consulted. Offline, every finding evaluates from stored material: the stored Bitcoin header chain (Section 5.1.4), the stored proofs, the certificate’s own ledger proofs, and the newest ledger snapshot the reader holds. Findings report unevaluated only for what is not stored. Online, pending proofs are looked up again and the archive’s current head serves as the ledger reference. Throughout, ρ stands for the certificate together with its documents (D and its timestamp proof, W , π , its ledger proofs, and the governing key-log entry) and the evidence set of Section 3.6, all stored beside it (Section 3.3).

There is one mode and no lenient setting: every *finding* (one yes-or-no check) that runs must succeed. Findings that can be skipped are reported as unevaluated, never as passed: a proof still pending; a ledger reference no source supplies; and, for a restricted deposit, every file-recomputing finding for a reader without access (Section 3.5). A finding thus ends in exactly one of three states: passed, failed, or unevaluated. A finding with several parts takes the worst state among them (failed over unevaluated over passed), its reason code naming the part, so two implementations print the same verdict. A timestamp still awaiting public confirmation is a form of unevaluated (reported as pending), never a pass or a failure. Absence of P^* or of any piece of that evidence is failure.

What a pass means. A pass means exactly what was chosen. Without the pinned pair θ the check fails; with it, a pass establishes record consistency. With $k > 0$ it establishes that and k steps replicated (or, at T2, re-executed against the per-output disclosure with no verdict; steps unevaluated for environment mismatch count toward neither side). With $\omega = \text{online}$ it adds whether π is still pending or now confirmed. Every finding reports its state with a reason code, and a failure names its cause—a missing piece or a mismatch, and which file—so a half-downloaded evidence set is never reported as tampering. The certificate software fetches the whole certificate bundle by the version DOI (Appendix B) and re-fetches once before reporting a mismatch.

4.1. Part 1: recompute the certificate (static checks)

Part 1 recomputes every value in the certificate from the deposited files and asks whether they match. It runs no analysis, covers every step rather than a sample, and is fast enough to be routine.

Cost. Recomputing the certificate is fast, and that asymmetry is what makes routine checking practical. The static checks read each file once and hash it. Hashing runs at roughly a gigabyte per second on an ordinary laptop, so a replication dataset at the one-gigabyte ceiling of Section 2.2 is hashed in about a second. Checking the handful of Ed25519 signatures takes well under a millisecond. Rebuilding Q and the Merkle roots is likewise linear in what was recorded. A complete static check is therefore a matter of seconds to a few minutes, dominated by reading the disk, while the analysis it certifies may have taken hours or weeks of estimation. That reading includes the evidence set: the stored blobs can exceed the deposit itself, so the bound is linear in what was stored, not in the one-gigabyte ceiling. Re-executing steps (Section 4.2) is the expensive part, and it is optional: $k = 0$ still settles record consistency for every step.

Static checks ($k = 0$). Static checks read the certificate and the files; they do not run the analysis. The ten findings come in two groups, in this order. First, recompute from the deposited files: do they match the certificate? Finally, check the archive’s signatures and the public record: did the archive bind this certificate, do its dates cohere with the public record, and does the ledger hold it? The numbering orders the report, not the execution: an implementation runs finding 5’s audit first, since finding 1’s partition consumes its result (Section 2.1). Recompute every value; do not read it off ρ .

From the deposited files.

1. Recompute the hashes for the files and for the environment capsule, and the hash of those hashes together. h_c, h_d, h_u, h_e from P^* ; h_x from x (taking h_o from ρ pending finding 3); h_e recomputed as the group hash of the environment group, whose one member is κ (so a swapped capsule changes h_e). The capture-code record coheres: its after-hashes match the deposited files, and h_c^- recomputed from its before-hashes matches its recorded value. The capture code is the schema’s: every file in the record’s added list is byte-identical to the schema’s canonical capture file for its language. Each touched target’s after-bytes equal its before-bytes plus exactly the schema’s loading line at the schema’s insertion point. The closure transformation is thus recomputed rather than trusted; a doctored hook would otherwise survive every finding and reproduce its own counterfeit on re-execution.
2. Recompute the Merkle root of the step list, and check that every listed script finished and saw the seed. R_T from the step records; each stored manifest, M_0 through M_n , reproduces its recorded root. M_0 equals, entry for entry, the manifest recomputed from the P^* in hand plus the declared dependencies’ (path, $H(\text{bytes})$) pairs from the declaration file inside P^* , with no entry besides (Section 3.2). Each r_i ’s written-file list equals the difference of M_i and M_{i+1} , and no manifest or write-set names a path on the schema’s

runtime-cache list (Section 4.2). n equals the record count; every step passed the schema's per-language success test (a zero exit, or for Stata the log's error test, Section 3.2); every target appears in T . s reached every target: the transcript of every target's step carries exactly one framed seed banner (the nested banners of launched processes aside) naming that step's s_i , derived from the s of ρ that finding 6 recomputes (Sections 2.2 and 3.2). A runtime's own log header may precede the banner. Re-execution coverage recomputed from which manifests' files are all available as bytes (blobs or unchanged P^* files, Section 3.2) equals the recorded (n_c, n) .

3. Recompute the outputs from the manifests, their hashes raw and normalized, and their consistency with the step list. o recomputed as the files of M_n new or changed against M_0 ; raw h_o over all of o and normalized h_o over its blob-stored members (Section 3.2). Every stored output file; each output's hash in M_n equal to the last after-hash any step recorded for that path; the output inventory by the schema's kind map.
4. Rebuild the captured numbers from the capture files the run wrote, and require an exact match. Q rebuilt from the stored capture files (Section 2.2) hashes to the $H(Q)$ in ρ . A claim that no estimates were captured must rebuild to an empty Q .
5. Recompute the determinism tier, re-search the scripts for outside files, and confirm that process observations are present. τ recomputed from the closure report, whose hash is the closure hash D signs (finding 6) and which names this deposit's $H(P^*)$ and s_0 , equal to the deposit's. The per-estimate seed sensitivity and its summary—including state, coverage counts, standard-error availability, and largest standard-error-scaled movement—recomputed from the report's values under s_0 and the Q rebuilt in finding 4 equal the ones recorded in ρ (Section 2.2). Absence of the closure report is failure: τ is then unmeasured. The dependency audit re-run on P^* reports no undeclared external reference. The fetch record written by the certificate software's own resolution of declared dependencies (Section 2.3) lists exactly the pairs of the declaration file inside P^* , hashes matching. Process observations are present and labeled observed. The audit re-run covers the image in hand when κ names one, and its artifact manifest recomputed from the image equals ρ 's (unevaluated for the image half when the image is not in hand). The exception mark recorded in ρ equals the schema's function of the recomputed tier, audit record, and process observations (Section 3.6), so the mark a journal reads is one a reader re-derives.

From the archive's signatures and the public record.

6. Recheck the nonce, the seed, and the receipt signature, and that they all name the same deposit. s recomputed from ν by the public rule; Vrfy on D under the key its key id names, valid at the receipt's confirmed time t_D (finding 9, under the key log of finding 7; unevaluated while the proof is pending). D 's recognized build $(v_{\text{tool}}, h_{\text{tool}})$ equals κ 's and is recognized at t_D by the key log. $H(P^*)$ recomputed by packing the P^* in hand (Section 2.1) equals the $H(P^*)$ in D ; the deposit identifier recomputed from $(H(P^*), v)$ equals D 's id ; D 's four group hashes h_c, h_d, h_u, h_e equal the ones recomputed in finding 1; the closure hash in D equals H of the closure report in hand. The D inside ρ , with the signature σ_D stored beside it, is the one whose ledger document hash the ledger names, and ρ 's ν and s are D 's. D and ρ agree on v and on h_{schema} , which equals the printed

version-1 hash (Appendix A); and ρ 's predecessor equals the one signed in D , or both record absence.

7. Check the witness W under the key the archive's key log names, which the pinned trust root vouches for—not a key that arrived with the files. $\text{Vrfy}(k_{\text{pub}}, \text{enc}(W), \sigma_W) = 1$, where σ_W is the archive's signature on W and k_{pub} is the key the archive's key log names as valid at the confirmed time of the head under which W 's ledger line falls (finding 10), the public date that covers the signature itself (Appendix B). W binds $H(\text{enc}(\rho))$, and through it everything ρ holds; W 's (v, h_{schema}) equal D 's and ρ 's, its key id free to differ, since a rotation may fall between deposit and seal; and W records the account that submitted ρ . Absence of a trust root is failure; that the root is genuine is the reader's duty (Section 4), which no finding can test.
8. Check the public time mark on the certificate's exact bytes. π is for the exact bytes of $\text{enc}(\rho)$. If $\omega = \text{online}$, look up confirmation again. **pending** is not failure; mismatch is failure; with several calendars (Appendix A.2), t_π (and t_D likewise) is the earliest confirmed time; and while π is pending, the confirmed time of the head covering W 's ledger line (finding 10), once that head has confirmed, serves as t_π .
9. Check that the dates cohere with the public record. The pulse β is real: the block at the named height carries the named hash in the stored header chain; write t_β for its recorded time. That chain passes through the block hash pinned beside the trust root and, beyond it, meets the chain's own difficulty rule header by header (Appendix A.2 and section 5.1.4); a pass is provisional for blocks beyond the pin (Appendix A.2). The receipt's timestamp proof is present. Once it is confirmed at a time t_D , the arrival time is sandwiched: $t_\beta - 2\text{h} \leq t_{\text{arr}} \leq t_D + 2\text{h}$ (the allowance relaxes each endpoint outward, covering Bitcoin's clock skew, Appendix A.2), and $t_D - t_\beta \leq C_{\text{time}}$, the window itself taking no allowance (Section 3.1). When D 's supersedes field is set, the dead receipt and its proof are in hand, its line is in the ledger, its *id*, uploader, and predecessor equal the successor's, no other successor names it, and its own $t_D - t_\beta$ exceeds C_{time} , else failure. With $\omega = \text{online}$, a pending proof is looked up again; one still pending is reported as unevaluated (pending)—never passed, never failed—exactly as π 's pending is not failure.
10. Check that the archive's ledger holds this certificate. From the stored documents: D 's entry and W 's entry each carry an inclusion proof under a signed head whose timestamp proof is confirmed (the head's signature checked as in finding 7, at the head's own confirmed time). The entries' document hashes are those of the D and W in hand, and each entry's position is the one its document names (Section 3.3.3). Then, against a reference head, a consistency proof shows that the reference extends the certificate's heads. That head is the newest ledger snapshot the reader holds, obtained through its DOI or a mirror rather than from the archive it checks (Appendix B) or, with $\omega = \text{online}$, the archive's current signed head. This half passes only against a reference at least as large as those heads, a smaller one leaving it unevaluated; a head of the same size with a different root is a split and fails. With a snapshot in hand the tree is rebuilt at each stored head's size from the snapshot's entries, a root mismatch being a split. A certificate the ledger never issued—one produced with a stolen signing key, say (Section 5.1)—fails here whenever a reference head newer than its own is available, and nowhere else. This finding exists

because a key-holder can satisfy every other finding with genuine signatures and real-time proofs. Proofs the archive has not yet attached are pending, as π 's is; without any reference head the consistency half is unevaluated, never passed.

4.2. Part 2: re-execute sampled steps

After publication, anyone may run some of the recorded steps again and see whether they still produce the recorded outputs. This is the replication claim of Section 1.2.1, and it is the reader's work alone: the author's two closure runs happened before deposit and measured τ (Section 2.2), and the journal only checks that the certificate exists (Section 4.3). Part 1 already covered every step without running anything, so what re-execution adds is evidence that the certificate describes real computation. It is also cheap in the way the certified run was for the author: the deposited environment and the recorded pre-state make re-running a step one command. The reader thus pays the step's own running time on the reader's hardware, not the days of rebuilding that Section 1.1 counted. Where licensing kept the environment out of the deposit, the reader first restores it from κ 's recipe (Section 2.1). The reader chooses how many steps to run, trading time for confidence. Because re-execution is a sample, it deters and spot-checks fabrication rather than proving honesty.

Sampled re-execution. Re-executing a step means rebuilding the exact file tree the step started from, running it again under the same settings and seed, and comparing what it wrote and what it captured against the record, at the tier the author earned. Four parts follow: first, the sample; second, the rebuild; third, the run and its comparison; and finally, the one precondition on the verdict.

First, the sample. Draw k distinct steps from the n_c complete ones (Section 3.2), *uniformly without replacement* from the reader's own randomness, never from anything in the certificate, so different readers' draws are independent. Each remaining step is equally likely, no step is drawn twice, and if $k \geq n_c$, draw all. An incomplete step has nothing to rebuild from. The one exception is a step whose only missing bytes are a declared dependency: a reader may re-execute it outside the sample by re-fetching the dependency under its declared hash, as the rebuild rule below allows. The sample, and Equation (8), count only the n_c . A drawn step the reader declines to run is reported unevaluated and is not replaced: redrawing would make the sample non-uniform and Equation (8) an overstatement. The per-step durations in T tell an author which step a budget-limited reader is least likely to run; the formula is therefore the power of a reader who draws before looking at them. Each declined step counts as a miss, so Equation (8) with the nominal k is an upper bound. A reader may also re-execute a drawn step under the closure seed derived from s_0 and compare its records to the closure report's. This is allowed when every entry of M_i is an entry of M_0 , a step whose inputs no earlier step wrote, or after re-running the steps before it under s_0 from a fresh copy of P^* . A mismatch then shows the report is not a run of these files, except for a file a later step's recorded write-set rewrote, whose closure value is the end-of-run one and which is reported unevaluated.

Second, the rebuild. For each drawn i , reconstruct the tree from M_i : for every (path, $H(\text{bytes})$) pair in the manifest, place the bytes that hash to that value at that path, creating folders as needed. The parent folder of every path in the step's recorded write-set is included, so

a folder an earlier step created and left empty is present. The bytes come from the stored blobs, from P^* itself for files unchanged since deposit (which the reader already holds), or, for a declared external dependency, by re-fetching its (locator, $H(\text{bytes})$) pair and requiring the fetched bytes to hash as declared. The hash, not the locator, is the identity, so a re-fetch that matches rebuilds the step as a blob would, though only a stored blob counts toward coverage (Section 3.2). A dataset too large for the ceilings (Section 2.2) does not forfeit re-execution merely by being declared, though the step is outside the sample and coverage says so.

Third, the run and its comparison. Run the recorded script exactly as at certification: its own process, its s_i handed through the schema's setting, frozen settings in force. Compare every written file to the recorded output under τ ; the recorded output is the stored blob's bytes, and at the numeric tiers the comparison parses those bytes, which a hash alone could not support. The set of paths the re-run wrote must equal the recorded write-set (M_{i+1} against M_i , deletions included). A recorded path the re-run did not write, or a written path the record lacks, fails the step, so a file planted into a later step's pre-state by a doctored build is caught when its step is drawn. A path on the schema's list of runtime caches is treated asymmetrically. If the re-run writes it, it is ignored, since a reader's environment may write such caches. If the record holds it, finding 2 fails, since the recognized build's frozen settings write none; closing deletes any path on the list from the closed copy before hashing (a vendored library's byte-code cache, say), so none is in P^* . A written file above C_{blob} is adjudicated byte-for-byte only: equality passes at any tier; inequality fails at T0 and is otherwise reported unevaluated for that file, since the bytes a looser tier would need were never stored. The re-run fires the capture code as well. Compare the records it produces with Q 's records for that step, matched by position within the step, at that same tier. The record counts must match first, and a differing count is itself a failed replication for that step: a step whose files agree while its coefficients do not has not replicated. At T0 only byte identity passes. At T2 each file and record the closure pair agreed on is judged at the loosest verdict tier, T1-reported where the schema parses the file and by normalized hash otherwise, since a T2 deposit has shown an instability the pair may have missed elsewhere. The write-set and record-count rules still apply. Only for an output the pair disagreed on does the check return no pass or fail. A disclosed gap is one draw of a random quantity, and a fresh draw, measured against the certified output, exceeds any fixed multiple of it at a computable rate (about 30% for twice the gap under Gaussian noise), so a verdict would fail honest outputs by design. The check instead reports the fresh gap beside the two disclosed values of Section 2.2, and a structural difference the disclosure did not name with its fresh hash beside the two disclosed ones; the result is reported as *re-executed against the disclosure*, neither *replicated* nor *failed*. If τ is unmeasured, this is not a replication check and the check has already failed.

Finally, the precondition on the verdict. The tier was measured as two runs on the author's machine, so it is a same-environment quantity: the numerical library, not the analysis, is what moves last digits between machines (Section 2.2). A re-executed step is therefore adjudicated at τ only when the re-executing environment matches κ on the schema's numeric-critical fields (Section 2.1). They are operating-system family, processor architecture, language runtime version, numerical-library implementation and version, the C mathematics library, and the numerical library's dispatch setting where it has one. The architecture and

the dispatch setting are listed because numerical libraries run different code paths on different processors, so identical versions can still differ in last digits. For an output the schema does not parse, adjudicated by hash at any tier, the precondition also requires the versions κ records for every add-on library the writing step loaded and the runtime’s exact build; on any other environment such an output is reported *unevaluated* (environment mismatch), never failed, while the parsed outputs and the Q records of the same step are adjudicated as above. On any other environment the step is reported *unevaluated* (environment mismatch), never failed, the same discipline every other unevaluable finding follows (Section 4.1). Without this rule, an honest T0 certificate re-run on different linear algebra would be declared *not replicated* for a reason the tier never measured, creating the false disputes this protocol exists to end. The container image that readiness asks for (Section 2.1) is how an author keeps κ matchable years later, and the single measure that does most to keep this part of the check usable. When one is deposited, the closure runs and the certified run execute inside it, and κ ’s numeric-critical fields record the image’s content digest, so matching means running the same image. A declarative build recipe in the Guix style goes further still: an environment a future reader can rebuild rather than merely restore (Vallet, Michonneau and Tournier, 2022). A proprietary runtime—Stata—cannot travel in one, so matching κ there depends on the reader’s own license.

Detection probability. How much a sample buys depends on how much was faked, so a reader can choose k knowingly. The chance a random sample includes a faked step follows the hypergeometric distribution—sampling without replacement, the setup familiar from survey sampling. Write m for the number of steps among the n_c replayable ones whose re-execution would fail an adjudicable comparison, one the tier can judge (a reuse of the letter: elsewhere m is a signed message, Appendix A.2, and a group’s file count, Section 2.1). An adjudicable comparison excludes an output kept only as a hash above C_{blob} at any tier but T0, and at T2 an output the closure pair disagreed on (Section 4.2). A fake hidden in a step that is *not* replayable escapes sampling altogether, which is what the coverage fraction n_c/n discloses. The chance of drawing k steps that are all among the $n_c - m$ honest ones is $\binom{n_c-m}{k}/\binom{n_c}{k}$ when $k \leq n_c - m$ (and 0 otherwise). So if $k \leq n_c - m$ the probability of drawing at least one misrepresented step is

$$1 - \frac{\binom{n_c-m}{k}}{\binom{n_c}{k}}, \quad (8)$$

and is 1 if $k > n_c - m$ (the sample is larger than the honest remainder, so it must include a fake). A single altered step ($m = 1$) is caught with probability k/n_c . (A replication dataset driven by one master script has $n_c = 1$, and re-execution is then all or nothing; the sampling arithmetic matters for multi-script deposits.)

Isolated fabrication is hard to catch; broad fabrication is not. With $n_c = 20$ replayable steps and one misrepresented, re-running $k = 4$ steps catches it with probability $4/20 = 0.2$, and even $k = 10$ gives only 0.5; certainty of drawing a single doctored step requires re-running everything ($k = n_c$). With $m = 5$ of 20 steps misrepresented, $k = 4$ already catches at least one with probability $1 - \binom{15}{4}/\binom{20}{4} \approx 0.72$. The deterrent does not depend on any particular k : an author deciding whether to fake a step cannot know which steps a later reader will draw, or how many, or whether anyone will draw at all. Equation (8) is the chance that the

sample *hits* such a step. The hit is a detection only when the drawn step is adjudicated: the reader’s environment matches κ and the compared outputs were stored as bytes. The formula is thus the power of one reader’s audit, conditional on the audit happening, not an estimate of how much fraud exists or of how many readers check; with $n_c = 0$ it is zero.

4.3. Part 3: what the journal checks (existence, not replication)

The certificate exists so that the journal need not replicate anything, and the journal’s work is correspondingly short. In order: the journal’s duties, which are look-ups; the citation the article prints; and finally the rules for comparing printed numbers with certified ones, for a journal that chooses to.

The journal’s duties. The journal keeps the two duties it already had and gains one look-up. (1) Tentatively accept the paper, subject to the author depositing the replication dataset in the archive and citing it in the article. (2) Confirm that both were done: the published version exists and the printed citation names it. If the journal requires certification, confirm also that the cited version carries one certification metadata field, written by the archive itself, linking to a certificate with no exception mark (Section 3.6) that the archive shows as sealed or confirmed, never failed (Appendix B). The archive must be one whose trust root the recognized certificate software carries (Appendix A.2); a look-up on any other archive’s page counts for nothing. Then formally accept. That is all. The journal runs no code and chooses no tier, since the protocol fixes what a certificate must meet (Section 2.2), and each step is a look-up on the archive’s page; a counterfeit page is caught by running *Verify*, which any doubting reader can (Section 4.1). What the look-up buys is exactly Section 1.2.1. A journal that never places the manuscript’s numbers beside the certificate’s is taking the author’s word for which table was certified, which is why the comparison below, though optional, is the one step we suggest journals take. Two choices remain the journal’s, each with a default that needs no written policy. First, whether to assign the number comparison below to an editor (reading, not computing), together with the curation look-ups no re-run ever covered, such as raw-data provenance and rights (Christian et al., 2018). The default is to assign it, since it is the one step this paper suggests. Finally, the journal reads the seed-sensitivity diagnostic as evidence about movement and coverage, not as a mechanical send-back rule. Version 1 adopts no threshold at which cross-seed movement refuses certification or automatically returns a manuscript. The metadata field therefore shows the diagnostic’s state, scope, coverage counts, and largest movement in baseline-standard-error units where defined (Section 2.2 and Appendix B). Invalid coefficients, unpaired records, ambiguous repeated identities, and unavailable standard-error scales are shown as incomplete coverage, not as zero movement. A journal that wants the sensitivity evidence to bear only on printed results uses the number comparison below and asks a person to decide whether the reported values and notes are adequate. The look-up should also note the landing page’s deposit counters (Appendix B): a certificate that is the second or later deposit of near-identical bytes is the trail Section 5.1 describes, and a journal may ask why.

Citation. The article prints the usual replication-data citation, in the form the archive already generates; only the final word is new:

Smith, Jane, 2027, “Replication Data for: The Effect of X on Y,” <https://doi.org/10.7910/DVN/AB1CD2>, Harvard Dataverse, V1. Certified.

The citation is version-specific: the DOI with “V1” names one frozen version—the version of record—which holds P^* and the metadata field linking to ρ (the field records $H(\text{enc}(\rho))$; the article does not print that hash, though the check still uses it internally— W binds it). Dataverse issues one DOI per dataset and selects a version explicitly, so the check resolves the pair, DOI and version, through the archive’s interface, never the bare DOI (Appendix B). A replication-dataset-level DOI that always resolves to the latest version is not this citation: a later version can link a different certificate. The final word “Certified” is a convenience for readers, not a cryptographic statement: the authoritative record is the metadata field on the cited version, which anyone confirms by following the DOI.

The number comparison, for a journal that chooses it. The certificate binds code and data to output files and to captured estimates, not to the printed article. Authors must take reported numbers from the certified run (Section 3.2), and whoever checks that places certified values beside printed values, number by number, citing the certified record for each match. Tables are compared as numbers under ordinary rounding to the digits the paper printed. Figures are compared through the data that drew them: those files enter o when the code writes them, or Q through the author-callable capture command (Section 2.2), and a figure drawn from data never written has no certified counterpart. A disagreement is a question for a person, never a failed finding, and comparing pictures by a similarity score is outside this protocol.

5. What certification guarantees

A certificate guarantees exactly what its check verifies, and this section states that by its edges. First, what is not claimed: what version 1 can certify, the attacks it does not address, and the risks that remain (Section 5.1). Second, what evading the protocol costs an attacker, computed rather than described, so that the surviving attacks can be priced (Section 5.2). Finally, what the guarantee adds over every alternative design: the tools it borrows, the one thing prior work leaves open, and our contributions (Section 5.3).

5.1. What is not claimed

The certificate does not catch every way a result can be wrong. This subsection states, first, what version 1 can certify; second, the attacks it does not address; third, the weak spots an implementation must state rather than leave a reader to discover; fourth, how the certificate’s claims age; and finally, one attack that beats the static checks but not re-execution. A future reader redoing the study from scratch remains a failsafe.

5.1.1. Version-1 scope

The constants and lists of version 1 bound what can be certified, and their intersection should be read together rather than discovered piecewise. The tree must fit in a gigabyte (C_{size} , Section 2.2). Code is recognized by the schema’s extension list, complete as given, so a replication dataset whose analysis lives in a language off that list cannot have that analysis certified. The author-supplied order of Section 2.2 rescues unusual *structure*, not unlisted *languages*. Analyses dependent on graphics processors (Section 2.2) will often measure T2 and stop on the ordinary route (Section 3.6). Capture covers the schema’s estimation-command lists, so hand-written estimators yield a thin Q , which the record count in ρ discloses (Section 3.2); a journal may ask for justification when that count falls far below the article’s table count. Data an archive may not hold has no route (Section 3.5). Many replication datasets deposited today would not clear these bounds as they stand—most do not even run (Section 1.1)—but that is a fact about today’s practice, not about the price of certification. Authors build replication datasets to whatever standard publication requires. The rungs of Section 1.1’s ladder that journals enforce with a check are deposit as a condition of publication, and re-execution; those were met once required, while the rung enforced by no check, data on request, was not. The failures on record are the fixable kind (automated path-and-library cleaning alone lifted one corpus’s execution rate from 25% to 40%, [Trisovic et al., 2022](#)). Closure is a diagnostic as much as a gate: preflight, the audit, and the closure report name exactly what to fix, on the author’s own machine, before anyone else’s time is spent. A replication dataset built to certify is a replication dataset built to run, which is the practice journals already ask for; certification makes the standard mechanical and gives authors the tool that checks it.

Table 2. Implemented execution profiles in the current prototype.

Profile	Status in reproai-cert-v5
run-seed-v1	Default profile. The run seed is seated at each target, and the current continuous sensitivity diagnostic uses the first closure run as its baseline.
step-seed-q-v1	Explicit opt-in profile for per-step seeds and per-step Q artifacts. It is not a default migration.
python-child-hooks-v1	Explicit opt-in historical Python child-hook profile.
mixed-child-hooks-v1	Explicit opt-in bounded profile for Python, R, and Stata child hooks and process-level Q capture.
Full version-1 target	Human-readable target specification described in this supplement; not a real implementation profile identifier and not fully implemented by the current prototype.

5.1.2. Out of scope

No claim is made against the following. First, a fabricated result written into the recorded scripts; it replicates on every re-run. Second, a specification that does not answer the pa-

per’s question, including *p-hacking* (trying many analyses and reporting only the favorable one) inside an honest run. Third, a *privileged computer* (a computer where someone has administrator access, a malicious *kernel*—the core of the operating system—or a compromised *boot chain*—the software that starts the computer). Fourth, a hardware adversary (*direct memory access*: a device that reads memory without the operating system; *firmware*: software baked into hardware; physical memory). Fifth, a file path constructed only at run time: the static audit cannot see it; process observations and re-execution are the remaining evidence. Finally, *confinement* of the run to an isolated execution environment (a sandbox that cannot reach undeclared files). *Cross-platform isolation* (the same sandbox on every operating system) without administrator rights does not exist in a form that behaves the same on the major desktop systems. A guarantee available to only some authors would make certificate strength depend on the author’s operating system.

5.1.3. Residual risks

Each item is a gap between what the certificate records and what happened, left open. The five groups, in order, are what recording can miss; what the author still influences; what the clock allows; the seed’s paid loophole; and what identifiers and the ledger do not promise.

What recording can miss. The dependency audit models languages by *patterns* (fixed text shapes that look like file reads), so *encoding* (how characters are stored as bytes: UTF-8 versus Latin-1 can make the same letters different files) can make audit and runtime disagree. So can *case-folding* (treating “File” and “file” as the same name on some disks), symbolic links, *macros* (text that expands into other text before the program runs), or generated code. Process observation is sampled, and a process that detaches from the step’s tree escapes it. Nothing here freezes the order in which a filesystem lists a folder, and nothing outside the author’s code can. So an analysis that iterates over a folder’s contents without sorting them can differ between machines even at a tier the closure runs earned on one, in reproducible-builds practice the most common single cause of same-code-different-output (Lamb and Zacchioli, 2022). The audit-to-run window (the gap between searching the scripts and running them) is narrowed by recorded hashes, per-step file lists, and a re-run of the audit, but not closed. Depositor identity is exactly as strong as the archive’s account registration, and a deposit proves custody of bytes, not authorship.

What the author still influences. Six parts follow, one per lever the author still holds, the last holding two: first, the tier; second, the seed; third, the seed’s floor; fourth, the capture channel; fifth, what sampling reaches; and finally, the declared environment and the target list. Every lever but one is disclosed by the certificate or bounded by it; the exception is named where it arises. First, the tier. The determinism tier is measured on the author’s own machine before deposit, so an author who writes deliberately unstable code earns a looser tolerance for later re-execution. The kept pair is fixed at deposit, before the seed exists (Section 3.1), and the closure report carries the count of pairs the software saw attempted, a self-report the author can reset. Retrying buys the author one thing: a pair that agrees by chance certifies an analysis the tier overstates. So the recorded tier is the best of the pairs attempted, not a measurement of one pair, and the later reader’s re-execution

is where that surfaces. The ordinary route forecloses only the loosest tier, T2, and what remains is disclosed by the tier itself. Below T0, residual nondeterminism under the one seed also lets an author run the certified run’s scripts privately more than once and submit the attempt they prefer. The certificate software does not select among attempts (Section 3.2), but nothing watches the author’s machine; the exposure is bounded by the disclosed tier.

Second, the seed. The archive’s seed can be overridden where the re-mapping of Section 2.3 does not reach, and the outputs are then independent of s , reopening seed shopping. Stata’s built-in `set seed` and its resampling options are disclosed and marked: for them the audit makes the override visible rather than impossible. A seeding idiom off the schema’s list is not disclosed. Its only clue is a stochastic estimate that did not move between the seeds, a person’s diagnostic read off values the author reported; a re-execution under s_0 against the closure report turns that clue into evidence for a step whose inputs no earlier step wrote (Section 4.2). Both clues hold only while the code does not distinguish the two runs, which a comparison against a remembered draw does without matching any pattern; beyond that the idiom is caught by reading the code. Off the list too is code carried into the run by a data file a script deserializes, or by a compiled artifact the audit cannot read. So ρ lists every such artifact against the registry digest it matches or none, and a launched program the hook cannot reach marks the certificate (Sections 2.3 and 3.2). An external dependency is bound at deposit by its declared hash, at the price that its bytes are not in the archive’s custody.

Third, the seed’s floor. An analysis that consumes no randomness gains no floor from the seed (Appendix A.2): a fully deterministic pipeline could have been run before deposit and replayed as the certified run, so there the window dates the record, not the computation. Even with randomness, an analysis cheap enough to run about two billion times can be precomputed under every seed of the $2^{31} - 1$ range before deposit and replayed after. That buys the author a replay, not a choice, since the seed is still the archive’s; it too dates the record rather than the computation.

Fourth, the capture channel. Capture-file mimicry, author code writing into a capture file’s path, is disclosed by the audit’s pattern family (Section 2.2). What survives is a write the patterns miss, and a counterfeit record so planted reproduces on every re-execution, which collapses it into the script fabrication that is out of scope.

Fifth, what sampling reaches. Files above C_{blob} are adjudicated only byte-for-byte (Section 4.2), and a step left incomplete escapes sampled re-execution; the coverage fraction and the oversize list in ρ are the disclosure.

Finally, the declared environment and the target list. κ hands the author one lever the check cannot price: the capsule is declared evidence, so a κ naming an exotic or false environment drives every later re-execution to unevaluated (environment mismatch), thinning the scrutiny Section 5.2 counts on (Section 2.1). That is why κ ’s numeric-critical fields sit on the certificate’s face, where a journal or reader can treat an unmatchable environment as they treat a loose tier (Section 4.3). The target list is the author’s too (Section 2.2), and with it n_c , the denominator of every detection probability in Section 4.2: a single master script makes re-execution all or nothing, and a list padded with trivial steps thins a reader’s fixed budget. The per-step durations in T disclose both.

What the clock allows. The floor is a day before the receipt’s proof, not the pulse’s own time (Section 3.1), and while that proof is pending the floor rests on duty rather than evidence.

The seed’s paid loophole. Seed shopping is closed in its free form and open in its paid one. Unchanged bytes under an unchanged predecessor claim replay the same seed, so a fresh draw requires *perturbing* the replication dataset (changing it slightly) or declaring a different predecessor under a later pulse (Section 3.4). A predecessor must be an earlier deposit of the same uploader, so that route gives at most one draw per earlier deposit. Either is a new deposit event the ledger records, and an author can sample perturbations until one gives a congenial draw. Each such deposit is individually honest (every finding passes), so the remaining defenses sit outside the check. The ledger records the trail of near-identical deposits, though no single certificate discloses its abandoned siblings, and *rate limits* (how often an author may deposit) are an operational matter outside this protocol.

What identifiers and the ledger do not promise. *id* is a full hash and gives no way to forge a deposit, because signed documents also bind $H(P^*)$. The archive’s online secrets are its signing key and the seed key k_{arch} ; the root key stays offline (Appendix B). They fail differently; neither failure is a break of standard cryptography. A stolen signing key produces internally consistent certificates whose one clue is absence from the ledger; finding 10, with the key log’s validity intervals (Appendix B), is the exposure. A leaked k_{arch} makes future seeds predictable, reviving free seed shopping with no artifact. Rotation through the key log bounds how long either leak lives. The worst case is a compromised signing service (Appendix A.2) that holds the keys *and* the ledger writer, so it can append its forgeries and pass finding 10 too. What even that compromise cannot do is backdate: the timestamp proofs it needs can only be obtained at real time, so its certificates carry compromise-era dates. What it cannot undo is its own trail: anchored entries cannot be quietly removed. Recovery is disclosure, a key-log rotation, and a public accounting of the exposure window, with every certificate from that window re-judged against the log’s intervals. The ledger’s two limits are stated with the anchor (Section 3.3.3): a split view is visible only against a head from a source the archive cannot revise, and a fork abandoned inside one anchoring interval is caught only when a certificate issued on it is checked. A replication-dataset-level DOI that always resolves to the latest version can name a later certificate than the one the article meant, so the citation is the version-specific DOI.

5.1.4. Obsolescence

The two claims of Section 1.2.1 age at different rates, and the record is built so that the durable one carries the perishable one. Record consistency is arithmetic over stored bytes, so the check works the same in fifty years as today (Section 1.2). A reader can still establish exactly what ran: every input file, every command, the environment down to library versions in κ , every estimate to full digits in Q . The dates rest on the public Bitcoin record rather than on anyone’s memory. Replication decays, because re-running the recorded steps needs the recorded environment, and operating systems, statistical packages, and eventually processors go out of service. Two things blunt the decay, and one gap survives them. First,

the record turns archaeology into translation. What defeats a decades-later replication is usually not dead hardware but not knowing what was done, and a certified deposit is a complete description of the computation with machine-readable expected answers. A later reader rewrites the analysis in whatever software then exists and compares it against Q at the certified tier, the comparison the numeric tiers were built for (Section 2.2). The certified seed does not travel with the translation: seeded streams are not portable even across versions of one runtime, so translation compares deterministic quantities exactly and treats stochastic ones as reimplementations, not replay. Finally, *emulation* (software that imitates one computer on another) outlives hardware; programs from the 1960s run in web browsers today. The frozen settings remove exactly the timing and core-count dependence that makes emulated replay unfaithful, while κ names what to emulate. The gap neither answer covers: a proprietary statistical package whose license and installer are decades gone is an archival and legal gap no record fills. There the failsafe of this subsection’s opening, redoing the study from scratch, remains, and the record still serves it by preserving what the answer was.

Two archival duties complete the aging story. First, the hashes themselves age: if SHA-256 is ever retired, every binding here retires with it. So the archive periodically re-timestamps its stored evidence under the successor hash while the incumbent still stands, and into a successor public record should Bitcoin’s stop being maintained. That is the standard renewal machinery of the Evidence Record Syntax (Gondrom, Brandner and Pordes, 2007), in the same lineage as Haber and Stornetta (1991). Finally, storing the Bitcoin block headers beside the proofs removes the last living dependency. The whole header chain is 80 bytes per block, under a hundred megabytes today and growing about four a year, stored once for all certificates. A lone header’s proof-of-work proves only that one block’s work was once done, so it is the stored chain, each header naming its predecessor, that lets the fifty-year check run with no network. A stored chain is evidence only through the block hash the reader pins beside the trust root (Appendix A.2): a chain forged at minimum difficulty is cheap, and the pinned block is what a forgery cannot pass through. Signatures age differently: Ed25519, unlike SHA-256, falls to a large quantum computer, and a confirmed π that predates the break is the mitigation, since it shows the signature existed while forgery was infeasible.

5.1.5. In-run memory edit

A value *planted* in the computer’s working memory during the one certified run—without changing the files—can pass the static checks and the article comparison, because that run really did write the planted number. Re-execution from the recorded pre-state replicates the honest value. Beating that requires repeating the plant on every later re-run of a public, signed record, under a seed that did not exist until after P^* was hashed.

5.2. The attacker’s calculus

Fraud is the rare case. The failures certification removes every day are error: moved files, absolute paths, missing libraries, the “silly” mistakes that make replication “all but impossible” (Gentzkow and Shapiro, 2014); closure removes them before deposit. Automatic path-and-

library cleaning alone raised one corpus’s execution rate from 25% to 40% (Trisovic et al., 2022) (file-level success rates, excluding files stopped at the time limit; the 74% of Section 1.1 is the initial-execution failure share over all R files). What remains is deliberate: Section 5.1 lists what a determined author can still do; this subsection prices it. The instrument is one inequality from the economics of crime: an offense is worth committing only when its payoff exceeds its cost plus the chance of being caught times the penalty (Becker, 1968). The protocol cannot set the payoff or the penalty—those are facts about careers and institutions—but it sets the other two terms, and they can be computed rather than described. Three parts follow: first, the deterrence inequality and the claim it carries; second, the surviving attacks priced in one table, with the two facts the table cannot show, that detection never depreciates and that it compounds across readers. Finally, what stays qualitative and whose lever it is.

The deterrence inequality. Write V for the value to the author of the fraudulent result standing published, S for the sanction if the fraud is exposed, $\text{cost}(a)$ for the price of mounting attack a , and $\text{det}(a)$ for the probability that a is ever detected. The *deterrence inequality* says mounting a is rational only when

$$V - \text{cost}(a) > \text{det}(a) S. \quad (9)$$

V and S are outside the protocol. Everything it controls lives in the other two terms, and this subsection claims that for every attack Section 5.1 leaves open, one of three *arms* holds. Either $\text{cost}(a)$ has a floor stated in the record’s own units, or $\text{det}(a)$ rises with every reader’s replay at a computable rate, or the attack’s reach is bounded by a quantity the certificate displays, so that a reader discounts it in advance. The displayed quantities are the tier, the coverage fraction, the oversize list, and κ ’s fields. One residual sits on no arm: a seeding idiom off the audit’s list, which detaches the outputs from s undisclosed. It is caught only by the zero-movement clue, read off values the author reported; by a reader’s re-execution of a drawn step under s_0 against the closure report’s records where the step’s inputs allow it; or by reading the code (Sections 4.2 and 5.1). With every other attack on an arm, the vagueness in “not worth it” is confined to the two symbols that were never the protocol’s to set. (If exposure also forfeits V —retraction—the rational-offense condition becomes $(1 - \text{det}(a)) V - \text{cost}(a) > \text{det}(a) S$, harder to satisfy than Equation (9); the form of Equation (9) is thus conservative, in the attacker’s favor.)

The surviving attacks, priced. Table 3 puts each surviving attack on its arm. The frauds the record forecloses outright are precisely the cheap ones, so they are not in the table. The edited output table, the swapped data file, and the quietly re-run script each today cost nothing and break record consistency here; the retyped manuscript number is caught by the comparison of Section 4.3. Every surviving attack short of fabrication either costs about as much as the honest work and leaves a ledger trail, or is bounded by a quantity the certificate displays, with the one residual on neither arm. Two rows need their arithmetic stated. Seed shopping pays per draw. If the congenial event arrives with probability q under one honest seed, and each fresh seed requires a fresh deposit (Section 5.1), the number of deposits until success is geometric with mean $1/q$. An author shopping for an outcome honest randomness

delivers one time in ten expects ten deposits, every one a permanent entry in a public ledger under the author’s pseudonym (Sections 3.1 and 3.3.3). Each deposit costs the seeded remainder’s running time plus a pulse interval on the predecessor route or a perturbation on the other, since deterministic work can be deposited as data and the closure report is a self-report that adds no floor. Detecting the shopping is counting, on the landing page and from the snapshots (Appendix B). Fabrication written into the scripts is out of scope for the certificate (Section 5.1), and it must be priced honestly. The check does not catch it, so its $\text{det}(a)$ under this protocol is zero, and what it saves is the science itself, the data collection a fabricator skips. What the record adds is that the fabricated file’s provenance and date are fixed on the day it ran, so the later dispute is a comparison of records rather than of recollections, and that every cheaper fraud now breaks record consistency. The line the protocol draws is therefore not that fraud is irrational but that the only fraud which survives is the one that was always outside a replication check.

Table 3. The surviving attacks, priced. Each has a cost floor in the record’s own units, a detection probability that rises with readers’ replays, or a bound the certificate displays; one residual has none.

Attack	Cost floor	Detection	Displayed
Paid seed shopping: perturb the files, or declare a new predecessor under a later pulse	one seeded run per draw, plus a perturbation or a pulse interval; $1/q$ draws expected; the predecessor route gives at most one draw per earlier deposit	every draw is a ledger line; counted, from the snapshots	the landing page’s deposit counters
Private selection among attempts below T0	extra runs	none by the check	the tier
A seeding idiom off the audit’s list	none	the zero-movement clue; re-execution under s_0 where the step’s inputs allow; reading the code	none: the one residual
In-run memory edit, a planted record, or a machine-level startup file	one run	$1 - (1 - k/n_c)^N$, compounding across N readers	coverage n_c/n
An unmatchable environment κ	none	re-execution unevaluated	κ ’s numeric-critical fields
A fake in an incomplete or oversize step	none	escapes sampling	coverage n_c/n ; the oversize list
Fabrication written into the code	saves the science	zero under the check	out of scope; provenance and date fixed
A stolen signing key	subverting the signing service (Appendix A.2)	finding 10, absence from the ledger, unless the ledger writer is held too	the key log’s intervals
An archive choosing its pulse to steer the seed	collusion	none; the coherence window’s candidates, re-draws counted (below)	the superseded-receipt count

Detection never depreciates, and compounds. The table cannot show two facts. First, without certification the chance that a future check catches computational fraud falls as the paper ages: environments drift, deposited code stops running (Trisovic et al., 2022), provenance fades from memory, and the dishonest author’s risk depreciates, so patience retires it. The certificate reverses that trend: its record and check are permanent, self-contained arithmetic (Section 1.2), the durable half of the obsolescence argument (Section 5.1.4). So for every attack that record consistency catches, the chance that a check next year catches it is what it was this year: a liability with no statute of limitations that costs its enforcers nothing to maintain. That chance is the hazard, not the cumulative probability, which is non-decreasing in any system. For the attacks only re-execution catches, $\text{det}(a)$ compounds while the recorded environment survives and then decays with it (Section 5.1.4; the translation route preserves the comparison, not the automatic check). Second, the compounding: the attacks cheap to mount are the ones re-execution catches, and the record makes re-execution accumulate. Equation (8) gives the chance that one reader’s sample of k among the n_c replayable steps hits one of m misrepresented ones; the record is public and permanent, so one reader’s sample is not the attacker’s whole exposure. Write N for the number of readers who ever run a sampled re-execution (Section 4.2). Samples are drawn independently across them, so a fake in a single step ($m = 1$) survives them all with probability $(1 - k/n_c)^N$, which decays geometrically in N : with $n_c = 20$ and $k = 4$, one reader misses it four times in five, but ten readers together miss it with probability $0.8^{10} \approx 0.11$. Each of those checks costs its reader the step’s running time and nothing else, needs no coordination with the others, and can happen at any time in the record’s unbounded life. For re-execution the compounding is front-loaded, though: N counts readers whose environments match κ (Section 4.2), commonest in the years just after publication (Section 5.1.4). The in-run memory edit (Section 5.1) is the limiting case. Its plant costs one run, but the honest value returns on *every* re-execution of the planted step, so its $\text{det}(a)$ is this same compounding probability and nothing the author later does can lower it. A κ chosen to be unmatchable lowers it in advance, the lever Section 5.1 names. Scrutiny also scales with the prize: readers re-execute the results that matter, so N grows with the result’s importance, the same importance that makes V large, and under Equation (9) raising the payoff raises the probability of paying S . That last inference is a claim about community behavior, not a protocol property. The certificate makes each reader’s check cheap, independent, and cumulative, but $N \geq 1$ still requires someone to check, and the evidence of Section 1.1 is that unorganized readers rarely re-run anything. The protocol’s contribution to N is to remove the check’s cost floor and give its result a public place to be recorded: an archive that accepts signed re-execution reports beside ρ turns private detections into public ones, a version-2 direction, not a version-1 rule.

What stays qualitative. What stays qualitative is V and S , deliberately, and one choice of the archive’s: which of the day’s blocks it names as the pulse in γ (Section 3.1), *shading* the seed, a choice no finding detects. Seed issuance is the one thing the archive is trusted for that the check cannot recompute (Appendix A). Three things bound that trust. The coherence window leaves a shading archive about 144 candidate pulses, hence 144 candidate seeds, per receipt (Section 3.1), and a reissue for cause, a further draw, leaves a dead receipt the ledger and the landing page attribute to the archive, so re-draws are counted rather than

bounded. Shopping among them serves only an author the archive is already colluding with, whose cooperation would buy far more than a seed (Section 5.1). And a reader who doubts a draw can re-run the recorded steps under seeds of their own, the hashes of blocks mined after the certificate’s timestamp proof, and read the certified value against their spread. The yardstick is the seed sensitivity the certificate already reports (Section 2.2): a diagnostic for a person, since the tier says nothing across seeds, not a verification of the archive’s derivation. A verifiable random function would let a reader check the derivation itself in a later version (Appendix A.2). Whether a caught fabrication ends a career is set by journals, universities, and funders, not by a hash, and a protocol that claimed to know those values would be pretending. Equation (9) is the interface: the protocol publishes $\text{cost}(a)$ floors and $\text{det}(a)$ rates computed from its own parameters (q, k, n_c , the ledger); policy supplies the two remaining numbers and reads off whether the inequality can hold. No claim is made that fraud is irrational—that claim needs V and S —only that every term the protocol controls is now a formula in published parameters, or a quantity the certificate displays, rather than an adjective.

5.3. Prior work and our contributions

Our protocol builds on standard cryptography wherever possible. This subsection separates what it borrows from what it adds, in four parts. First, the alternative designs for the same problem, ordered by the assurance they buy, and where the certificate sits among them; second, the tools it borrows. Third, the one thing every alternative and every tool leaves open, a lower bound on when a run happened; and finally, our contributions.

Alternatives, ordered by assurance. Approaches to the same problem form a ladder, each rung buying more assurance at more cost. Three parts climb it: first, the rungs below certification, from file fingerprints to attesting platforms; second, TRACE, the nearest design, and how its trust decision differs from ours; and finally, the rungs above, and where the certificate sits. First, the lower rungs. A file fingerprint—Dataverse’s Universal Numerical Fingerprint, a hash of a data file’s normalized content that is the same whatever format stores it (The Dataverse Project, 2026)—shows whether a file changed and nothing about what running it produces. Signed provenance adds a record of how files are claimed to have been produced. The software *supply chain* is the sequence of build, test, and packaging steps between a developer’s source code and the program a user installs. It has a framework, SLSA (Supply-chain Levels for Software Artifacts), that grades such records by how trustworthy the platform that generated them is; at every level above the first the reader trusts that platform (Open Source Security Foundation, 2026). Environment-capturing platforms sit one rung higher: “Whole Tale” packages the running environment with the analysis so a later reader can re-execute it (Brinckman et al., 2019), evidence still conditional on the platform that did the packaging.

Second, TRACE. The TRACE program (a name, not an acronym) has a similar goal to ours, to certify the original execution so that others can trust results without repeating them (Vilhuber et al., 2023; McPhillips et al., 2019). But it answers it by vesting trust in an attesting platform: a “transparent research system,” a controlled computing environment

that runs the workflow and signs a record that it did. TRACE’s platform signs as an eye-witness because it ran the workflow itself; our witness W signs as a notary, attesting only to the certificate it sealed, not to a run it never observed. The two designs differ in what the trusted party vouches for. TRACE vouches for the substance, that the computation happened as recorded, and its attestation is the evidence, with the hashes, signatures, and timestamps of that attestation exposed for anyone to check; the record’s assurance is therefore the platform’s continued integrity. What no reader can do is re-derive the substance without trusting the platform. The author works inside it, so that model reaches settings this one cannot, such as enclave-held data (Section 3.5). Here no party vouches for the substance. That the run happened as recorded rests on arithmetic any reader can compute and steps any reader can re-run. The archive is a notary, never an observer (Section 1.2.2; Appendix A states the seed trust, Section 5.1 the key-custody exposure). The seed bounds the run from below in time, which no platform’s word supplies. The trust decision is also an economic one. A platform that attests by executing pays for a full run of every analysis it certifies, on hardware provisioned for whatever authors bring: the data editor’s cost curve, which grows with the computation authors choose to run. An archive in our protocol that only hashes and signs pays per certificate the cost of hashing and storing what was deposited and recorded, and running the findings it can evaluate at seal. When the timestamp calendars lag (Appendix A.2), it pays one transaction fee shared across that day’s receipts. The runs themselves are paid for by the author on the author’s own machine. So an existing archive can take the role without running any analysis (Section 1.2; contribution 10).

Finally, the upper rungs. Independent re-execution has another party run the analysis: the data editors of Section 1.1, CODECHECK’s named checkers (Nüst and Eglén, 2021), and an accredited agency, cascadi, a French certification agency whose pre-publication certificates some journals accept and which serves confidential data through accredited staff (Pérignon et al., 2019; Hurlin and Pérignon, 2024). That is direct evidence at the cost of a qualified person per paper. Trusted hardware signs a record binding named code to output produced inside a protected processor, evidence conditional on trusting the chip’s maker (Costan and Devadas, 2016). Cryptographic proofs of computation remove even that trust (Goldwasser, Micali and Rackoff, 1989), but remain far too costly for general pipelines in R, Stata, and Python today, though proofs of training exist for small models (Walfish and Blumberg, 2015; Garg et al., 2023). The certificate sits between provenance and re-execution: more than a signed claim, because the archive’s seed fixes the run after the deposit and re-execution is sampled rather than skipped. It is less than a cryptographic proof of computation, because it certifies a run that only the author performed.

Borrowed tools. The Merkle tree that folds the trace into one root is due to Merkle (1980). The hash tree itself, with its 0x00/0x01 domain-separation tags, and the Merkle log the ledger uses come from Certificate Transparency, the public-log system by which web browsers audit the certificates behind `https` (Laurie, Langley and Kasper, 2013). The deployed system closest to the archive’s role here, signed artifacts entered in a public transparency log whose tree heads independent witnesses cosign, is Sigstore, whose Rekor log plays the ledger’s part (Newman, Meyers and Torres-Arias, 2022; Sigstore Project, 2026). That triple has since become an internet standard for the software supply chain (Birkholz et al., 2026): a signed

statement about an artifact, a transparency service, and the receipt the service returns. A certificate could be exported into that envelope form (an in-toto statement under a DSSE signature) for supply-chain tooling, but version 1 defines no such export, and the canonical record is the flat enc record. The keyed hash that binds the nonce to one deposit is HMAC (Bellare, Canetti and Krawczyk, 1996), and signatures are Ed25519 (Bernstein et al., 2012). Publicly verifiable timestamping originates with Haber and Stornetta (1991) and is realized here through OpenTimestamps (Todd, 2016). Anchoring research materials this way is established practice: Bitcoin-backed timestamping was built for researchers a decade ago (Gipp, Meuschke and Gernandt, 2015), and an archive consortium now records hashes of deposited research data on a blockchain as a service to its members (Wittek et al., 2020). What such services establish is existence at a time, and nothing about a run.

The practice of recording a run is also borrowed. Freezing clocks, locales, and embedded dates, then running twice to see whether the outputs agree, is the practice of the reproducible-builds community (Lamb and Zacchioli, 2022). Recording each step of a computation as a signed record carrying the hashes of its inputs and outputs, its console output, and its exit status is the design of *in-toto*, a framework for securing the software supply chain (Torres-Arias et al., 2019). Workflow engines keep the same kind of per-step provenance for their own runs—Snakemake tracks the code, parameters, inputs, and software of every job it executes (Mölder et al., 2021). Capturing a run’s true dependencies by tracing its system calls, then packing them for replay, is ReproZip (Chirigati et al., 2016); the static audit of Section 2.3 trades that completeness for a mechanism that needs no system-call tracing, which is not available on every platform. Re-executing a random sample of recorded steps from their stored pre-states, and bounding the chance that a fabricated step escapes the sample, is due to Monrose, Wyckoff and Rubin (1999), with the spot-check deterrence it prices going back to the uncheatable-computation line of Golle and Mironov (2001). The same logged-run-plus-sampled-replay shape verifies machine-learning training in proofs of learning (Jia et al., 2021). Hashing a frozen analysis pipeline, signing it, timestamping it publicly, and summarizing the run in a Merkle tree has been done for astrophysics (Casas and Fidler, 2024). A cryptographic near-namesake, “proof of replication,” is an unrelated storage primitive, proving that data is held in dedicated replicas, as in Filecoin (Fisch, 2018), and has nothing to do with re-computation.

What prior work leaves open. None of this bounds a run from below: a signature, a transparency log, and a timestamp each attest that bytes existed no later than some moment. Acceptance by certificate needs evidence that the certified run happened *after* the deposit was fixed, which Section 3.3.2 supplies. The in-toto designers name what follows a “fake-check”: an attestation fabricated for a step that produced nothing, which they treat as a consequence of a compromised signing key (Torres-Arias et al., 2019). A signature over a record of work is only evidence that someone with the key said the work happened.

Two lines of work come close. The first is the randomness beacon (Section 3.1): a public service that emits an unpredictable value on a fixed schedule, so that anyone who commits to a plan beforehand and then seeds it from a later pulse has shown the work came afterwards (Rabin, 1983; Kelsey et al., 2019). Distributed beacons are now deployed at scale (Syta et al., 2017; Drand and The League of Entropy, 2026). That is the right shape, and a beacon can

be tied to named materials by hashing the pulse together with a description of them, as the reference beacon’s guidance describes (Kelsey et al., 2019). What it leaves to the user is the commitment itself: publishing the plan and the materials somewhere the pulse can be shown to postdate. That is the deposit this protocol already requires, so here the archive’s receipt plays the commitment’s part and the pulse dates it. The second line puts the unpredictable value inside the experiment, seeding it from a hash of the preregistered protocol so that the data themselves depend on the plan (Mazor, Mazor and Mukamel, 2019). But the author computes that hash, with no other party involved, so nothing stops them adjusting the plan and deriving a new one. Blind analysis in physics (working under a hidden perturbation until the analysis is frozen) removes the analyst’s ability to condition on the realized draw by the same logic, enforced by discipline rather than by a second party (MacCoun and Perlmutter, 2015). Enforcement in hardware is the one route that closes both gaps for scientific work, by sealing the analysis inside a trusted enclave and running it there exactly once (Meißner et al., 2021). The price is conducting the entire study on that platform and making independent re-execution impossible by construction.

Contributions. Our proposed protocol makes ten contributions: seven that follow from the design, and finally three, (8) to (10), that make the journal’s shortcut sound—none new cryptography, each a new use of standard methods.

(1) *Acceptance by existence.* A journal can accept a paper by confirming that a certificate exists (Section 4.3), rather than by re-running the analysis. The certified run already did that work, on the terms of Section 1.2.1: the manuscript’s numbers are the certified numbers, and the record can be replayed by anyone. The immediate advantage is cost and convenience. A journal that verifies today does so by hiring and paying a data editor to re-run every tentatively accepted paper, typically delaying publication for months. Certification removes the re-execution outright and moves the repair work re-execution triggers to the author before submission, where it is paid once and most cheaply. A data editor’s elapsed months include the authors’ own repairs: the *American Journal of Political Science* process’s author-response time averages a third of its seventy-five days (Jacoby, 2018). The saving extends past the journal: a later researcher, and especially anyone replicating at scale across many papers, can settle record consistency in seconds of hashing instead of re-running every analysis. What certification does not remove is the comparison of manuscript to certified record; it makes that comparison mechanical and leaves it, with the curation look-ups, to a much smaller data-editor role (Section 4.3).

(2) *A shared record.* With the certificate—the record Section 1.1 ends on—a failed replication becomes a comparison of computations rather than a contest of reputations.

(3) *A record that outlives its tooling.* A verifying data editor leaves behind a verdict; the certificate leaves the computation itself, still checkable—and still a reference for reimplementing—after the software stack that ran it dies (Section 5.1.4).

(4) *Computable deterrence.* The security argument is arithmetic rather than adjective: every cost and detection term the protocol controls is a formula in published parameters, or a quantity the certificate displays, with one residual named as neither. That holds conditional on a reader checking, which the protocol makes cheap and cannot compel, and leaves policy only the two symbols that were never a protocol’s to set (Section 5.2).

(5) *The estimates themselves are bound.* Capture code copies every listed estimate out of the statistical runtime, its memory in R and Python and its log in Stata (Section 3.2), into the signed record (Section 2.2). So the certificate binds the estimates themselves, not only the files that produced them. The article’s numbers are bound to those estimates by the rule that authors report certified values and by whoever compares the two (Section 4.3).

(6) *A closed specification.* Nothing that affects what a check computes is left to implementer choice, so independent implementations must agree—same hashes, same seed, same tier—and must accept each other’s certificates (Appendix A).

(7) *Split evidence that travels inside the certificates.* Certificate Transparency catches a log that shows different parties different histories through dedicated monitors and, in its design literature, gossip among clients (Chuat et al., 2015; Meiklejohn et al., 2020). Here the proof travels inside the signed objects: each certificate stores its own inclusion proofs under signed, timestamped heads. So an archive’s equivocation is documented afterward by any two certificates whose heads conflict, without asking the archive for anything (Section 3.3.3).

(8) *Seed shopping made visible.* An author must not be able to shop quietly for a random draw—to run an analysis repeatedly under different seeds and report whichever one looks best—a specification search that preregistration does not fix, because the analysis never changes. Certification does not forbid it; it makes it visible. The archive hashes what arrives and only then issues the seed, a challenge the author could not predict, so the certified run cannot have produced its draws beforehand. A fresh draw requires a fresh deposit that the ledger records (Sections 3.1 and 3.4). The author’s own reseeds are re-mapped through s where the language allows and disclosed where it does not, for every reseed in the text the audit reads (Section 2.3); an idiom off that list detaches the outputs undisclosed, and that residual is stated (Section 5.1). Two ingredients do different jobs. The archive’s secret makes the seed specific to these bytes. The public pulse beside it, confined by the check to the day before the receipt’s timestamp proof, makes the receipt undatable to any earlier moment, so the floor holds even against the archive (Section 3.3.2).

(9) *The earned tier.* The test a later re-run must pass needs a threshold nobody chose for their own benefit, since an author who set it would be choosing the bar they must later clear. Measurement supplies it: two closure runs before deposit earn the tier (Section 2.2), the closure report is stored and hashed, and the check re-derives the tier from it rather than reading it off the certificate (Section 4.1). That sits within the same self-report boundary as the capture code: a modified certificate-software build is not caught (Appendix C and section 5.1). What the author keeps is influence rather than choice: code written to be unstable earns a looser tier. But that trade is visible on the certificate: T2 by the ordinary route stops certification (Section 3.6), and a T1-reported certificate says on its face that the analysis reproduces only to within one unit in the fourth significant figure. Badging schemes, by contrast, record a mark without any measurement of agreement. The recommended practice that standardizes them places “the process for verifying results” outside its scope (National Information Standards Organization, 2021), and the largest deployed regime, the Association for Computing Machinery’s artifact review, vests verification in conference committees (Association for Computing Machinery, 2020).

(10) *The open check.* The journal’s shortcut is safe only if the deeper check is open to everyone, so checking requires no privileged access. A reader who doubts a certificate runs the same calculation as the archive that signed it, needing only the deposited files and a

trust root and one block hash obtained independently (Appendix A.2). An existence check suffices because anyone who doubts the certificate can run the full one. It lets an archive offer certification without running the analysis, which makes adoption by an existing archive practical (Appendix B). This property is standard in software supply-chain systems and is inherited from them rather than invented here. What is new is having it in a research certificate, since prior systems that verify a published analysis vest the check in a designated person, agency, or witnessing platform (Nüst and Eglen, 2021; Pérignon et al., 2019; Vilhuber et al., 2023) and hand the reader a verdict instead. It holds for every deposit—for a restricted-access deposit, for every approved reader (Section 3.5).

A. Cryptographic tools used throughout

This appendix expands Section 1.3, which named each tool in a sentence; here each is defined exactly, with its byte spellings and the security property it supplies. Producing a certificate (Section 3) and checking one (Section 4) use the same small toolkit, in three groups: tools that name bytes (Appendix A.1), tools that bind a person or a time to those bytes (Appendix A.2), and a tool that folds a list into one hash (Appendix A.3). Two conventions govern all of them, and every section of the paper, so they come first: a machine-readable schema that fixes byte-level spellings, and the protocol version that names which edition of the rules—and which schema—every signed object was built under. The preliminaries close with what sits outside the protocol and the assumptions every security claim rests on.

Schema. A *schema* is the machine-readable half of this specification: one small file tree fixing every byte-level spelling whose meaning the prose explains. It fixes the key names and field order of each signed object, the grammars of the result capture and of the closure report, the normalization byte rules, the dependency-audit patterns, the capture code itself, and the per-language estimation-command lists. The objects themselves are defined in Sections 2 and 3; the schema fixes their spellings, this paper their meaning. Spellings must live in a file because implementations consume them exactly and a PDF is typeset, not parsed: whether the arrival time’s key is `t_arr` or `arrival_unix` changes every hash in the protocol, since the canonical encoding (Appendix A.1) writes fields sorted by key name. The schema is named the way every object here is named—by its hash, $h_{\text{schema}} = H(\text{pack}(\cdot))$ over its tree (Section 2.1)—and is deposited in the archive under a version-specific DOI, preserved by the same infrastructure the protocol relies on. The published version of this paper prints that hash here: $h_{\text{schema}} = \langle \text{printed at publication} \rangle$.

Protocol version v . A version number says which edition of these rules a certificate was built under, like a codebook edition: later rules do not apply retroactively. Three things follow: first, how the pair of version and schema hash binds every object; second, what it means that version 1 is closed; and finally, what is outside the protocol. First, the binding. Every signed object carries the pair (v, h_{schema}) in its protocol context. A reader applies the rules of the declared v and never those of a later one, and the check requires the pair to match across the receipt, witness, and certificate that name one deposit (Section 4.1). A certificate claiming $v = 1$ with any other schema hash is not a version-1 certificate. Each

artifact is thus fixed one level up: the schema by h_{schema} (Appendix A), the paper by its version of record, and every certificate by the pair it names.

Second, closure. This paper is version 1, and version 1 is closed in this text. Every rule that affects what a check computes is fixed here or in the schema, whose hash is printed at publication (Appendix A); no value is left for an implementer to pick. Until that hash is printed the schema is a draft, and what follows is a claim about the published pair, tested by the conformance vectors the schema supplies. Two independent implementations of version 1 must then compute the same hashes, the same roots, the same seed, and the same tier from the same replication dataset, and each must accept the other’s certificates. Where this specification names a constant, that value *is* the rule, not a suggested default: C_{size} , C_{blob} , C_{time} , the anchoring interval, Δ , the domain strings, the seed range, the tolerance at T1, the two-hour allowance, the six-confirmation depth, and the build-recognition overlap. The rest are the attestation deadline of Section 3.1, the four significant figures of T1-reported (Section 2.2), the sensitivity diagnostic’s schema name and coverage fields (Section 2.2), and the fixed build date of Appendix C. The domain strings are Λ , Σ , the manifest string of Section 3.2, and the ledger’s string of Section 3.3; the schema fixes the complete list, one per construction. Where it names a fixed list (the tiers; the fields of D , W , and κ), the list is complete as given, and every byte-level spelling those lists leave open is fixed in the version-1 schema (Appendix A, which enumerates what it fixes). And where it names an outside service (the timestamp service of Appendix A.2; the source of the freshness pulse of Section 3.1), the named one is the one a version 1 certificate is issued and checked against. Changing any of them is a new v , not a local option. Each is defined where its object is introduced; Figure 2 maps the objects and their sections.

Finally, what is outside. Three things a check might be expected to inspect are outside the protocol, and a check never inspects any of them. They are how often an archive lets an account deposit (rate limits, Section 5.1); the archive’s backups, succession arrangements, and landing-page design (Appendix B); and any aid a person uses when comparing an output to the printed article (Section 4.3). Each is named as outside so that silence is not mistaken for latitude; the archive’s terms of deposit, the recognized-build list, and the payoff and sanction of Section 5.2 are outside it too, named where they arise.

Assumptions. Every security claim in this paper reduces to four standard assumptions and one trust assumption, collected here for simplicity. No one can find SHA-256 collisions or preimages (Appendix A.1). HMAC-SHA-256 outputs are unpredictable without the key (the pseudorandom-function property, Appendix A.2). Ed25519 signatures cannot be forged without the secret key (Appendix A.2). And Bitcoin’s blockchain is immutable at depth, its block times approximately honest, and its block headers available (stored beside the proofs, Section 5.1.4; Appendix A.2 and section 3.1). Two operational premises sit beneath all four: the archive’s keys were drawn from a well-seeded cryptographically secure generator (a random-number source an outsider cannot predict), and the reader pinned the genuine trust root and one Bitcoin block hash out of band (Appendix A.2). One trust assumption stands beside them, and Section 5.2 states what it admits and what bounds it: the archive computes ν as specified and does not shop the one field of γ it chooses (Section 3.1). The argument is informal throughout—each claim reduces to these assumptions in prose, with no

formal security games—and where a claim needs less, the text says so; where an assumption ages, Section 5.1.4 says what the archive does about it.

A.1. Naming bytes: hash and canonical encoding

Hash H . A *hash* is a function that computes a fingerprint of a file’s bytes: change even one character and the hash changes completely. The file cannot be recovered from the hash, except by guessing it and checking (a channel Section 3.5 closes for restricted files). No one can find a file that hashes to a given value. And no one can usefully construct *two* different files sharing one hash (*collision resistance*—here the property that carries the most weight, because the author, who supplies the bytes being hashed, is also the party the record must bind). Let H be SHA-256, a standard cryptographic hash: any run of bytes in, always 32 bytes out, written as 64 lowercase *hexadecimal* characters (digits 0–9 and letters a–f; two characters per byte). Let $H(z)$ be the hash of object z .

One use of H recurs throughout and deserves its name at once: a *commitment* is a hash—or a short bundle of hashes—stored inside a later signed or hashed object, so swapping the committed bytes later makes a stored hash fail to match. The receipt, the witness, and the commitment x of Section 3 all bind bytes this way.

Canonical encoding $\text{enc}(\cdot)$. The canonical encoding turns a record with named fields into one run of bytes that every implementation spells identically. Three parts: first, why a rule is needed and what enc is; second, what may not appear in a signed object; and finally, how this encoding relates to the standard one. First, the rule. What H takes is one run of bytes, from a first byte to a last. A single file is already that, so hashing a file needs no preparation. Anything else does. A record with named fields, and a folder holding many files, are not runs of bytes until somebody writes down a rule for laying them out in one order and spelling every part the same way each time. If two implementations pick different rules, they compute different hashes for identical material. For records, that rule is the canonical encoding defined here; for file trees it is *pack* (Section 2.1), which waits until the closed replication dataset it lays out has been built. Let $\text{enc}(\cdot)$ convert a structured object to bytes as JSON (a text format of named fields, like `{"seed": 42, "tier": "T0"}`). Keys are sorted by Unicode character number (the standard numbering of characters; ordinary English letters sort as in the alphabet). No extra whitespace appears (a colon and a comma with nothing around them). And any character outside ASCII (the basic English letters, digits, and punctuation) is written as a `\u` code, so every implementation spells it the same way.

Second, what may not appear. A *floating-point* value is a number stored in binary scientific notation, which two computers may print with different digits. None may appear in any signed object or in a hash chain (a list where each item carries the hash of the previous one). A signature covers bytes, not the number those bytes were meant to represent, and two implementations that agree on a number may still disagree on a float’s shortest spelling. Times are integer *Unix seconds* (whole seconds since 1970-01-01 00:00:00 UTC—Coordinated Universal Time, one shared clock—a fixed origin like a Stata date origin); durations are integer *milliseconds*; and any ratio is stored as the two integers from which it is computed. Array order, unlike key order, is part of a value and is never sorted: the commitment’s

five hashes (Section 2), the nonce context’s four fields (Section 3.1), and the ledger entry’s four fields (Section 3.3) mean what they mean by position. The finished document is pure ASCII, so its byte encoding needs no separate rule. Every finer spelling detail (escape rules, hexadecimal case, integer format) is the schema’s (Appendix A).

Finally, the standard. This encoding is a deliberately stricter sibling of the standard JSON Canonicalization Scheme (Rundgren, Jordan and Erdtman, 2020): that scheme admits floating-point numbers, spelled by a shortest-round-trip rule, and this protocol forbids them outright. Because every key in a version-1 object is schema-fixed ASCII, the two schemes’ different treatments of non-ASCII keys never arise. An implementer must build enc to this paragraph and the schema, not to an off-the-shelf canonicalizer.

A.2. Binding a person or a time: signatures, nonce, seed, timestamp

Signatures **Sign and Vrfy.** A *digital signature* is a mark that only the holder of a secret key can make, and that anyone holding the matching public key can check—like a specimen signature on file, except that the mathematics makes forgery infeasible. Three parts: first, the two operations; second, where the archive’s keys live; and finally, how a reader obtains the public key. First, the operations. A signature is not a picture of a name but a number: under Ed25519, the standard signature scheme used here, a signature is 64 bytes, ordinarily written as 128 hexadecimal characters. Two operations define it. $\text{Sign}(k_{\text{priv}}, m)$ returns the signature of a message m (the bytes being signed) under the secret key k_{priv} . $\text{Vrfy}(k_{\text{pub}}, m, \sigma) = 1$ exactly when the signature σ is valid for m under the public key k_{pub} , and 0 otherwise.³

Second, the keys. The archive’s keys are made on the signing service and never travel (rotation is Appendix B’s). The archive’s *signing service*—the one running copy of its software, on the archive’s own server, that issues receipts and witnesses—generates a *key pair* (a secret half k_{priv} and a matching public half k_{pub}) the first time it starts. It stores k_{priv} where only it can read the key and never copies that half anywhere else: producing the archive’s signatures then requires subverting that one service or stealing its stored key, not merely working at the archive.

Finally, the trust root. The reader must obtain the archive’s key from somewhere the certificate cannot influence. Anyone checking a signature pins the archive’s *trust root*: a root key, kept offline and used for nothing else, in the rotation discipline of software-update systems (Samuel et al., 2010). That key signs the archive’s key log, the ledger-borne list naming each signing key, its validity interval, and its successor (Appendix B). The reader settles in advance on that root, obtains it out of band, and holds it fixed. The recognized certificate software carries the roots of the archives it recognizes and is itself checked against its published hash. For a reader who runs no such software, the root’s public key is deposited under a version-specific DOI beside the key log and the ledger snapshots. A copy of a key shipped alongside the files being checked does not suffice, because a substituted archive would ship the substitute’s key, and its forged signatures would then appear valid. One more value is pinned the same way, beside the root: the hash of one Bitcoin block, as recent

³Ed25519 verifiers are not all alike: published libraries disagree on edge cases (non-canonical point encodings; cofactored versus cofactorless verification). The disagreement is enough that one library can accept a signature another rejects (Chalkias, Garillot and Nikolaenko, 2020). The schema therefore pins the verification criteria (Appendix A), so two version-1 implementations return the same verdict on the same bytes.

as the certificate software that carries it. The stored header chain must pass through it and, beyond it, meet the chain’s own difficulty rule header by header (Section 5.1.4), so that a forged chain would cost real proof of work rather than the minimum difficulty a forger could otherwise choose. Beyond the pinned block the difficulty rule proves work spent, not that the suffix is the public chain, so the pin should postdate the proofs being checked. Each recognized build ships a fresh one. A reader checking proofs newer than their pin treats the date findings as provisional until the online check, or a later build’s pin, confirms the suffix (Section 4.1).

Nonce ν . A *nonce* is a number the author could not have known in advance; the archive makes it with a secret of its own, and the run seed is then computed from it. The term ordinarily stresses that such a number is used once. Uniqueness here follows from the hash and pulse in the nonce context γ (Section 3.1). The property that works is unpredictability: ν is a *challenge*, a value the archive issues that the author could not have predicted, so that any answer containing it must have come afterwards. The tool that makes it is a *keyed hash*: a hash that also takes a secret key, so only someone holding that key can produce the output (ordinary H anyone can recompute; a keyed hash only the archive can). Let $\text{HMAC}(k_{\text{arch}}, m)$ be the keyed hash of a message m under the archive’s secret key k_{arch} , computed with HMAC-SHA-256, the standard keyed hash built from SHA-256, whose name abbreviates *hash-based message authentication code*. The signing service draws k_{arch} , 32 bytes long, from a cryptographically secure generator (Appendix A) when it first starts, and again at each rotation the key log records (Appendix B). The key k_{arch} is used for this HMAC and for nothing else, so no other use of the key exists to collide with it. The nonce is

$$\nu = \text{HMAC}(k_{\text{arch}}, \text{enc}(\gamma)), \quad (10)$$

where γ is a four-field record that identifies one deposit (defined when the receipt is issued, Section 3.1). The value ν is 32 bytes: written as 64 lowercase hexadecimal characters when stored in a JSON document, used as the 32 raw bytes when concatenated for hashing, as in the seed rule that follows. A reader cannot recompute ν , lacking k_{arch} , and instead checks the archive’s signature over the receipt that contains ν . (A verifiable random function—RFC 9381’s construction on the edwards25519 key type the archive already holds—would let a reader check the derivation as well (Goldberg et al., 2023). Version 1 keeps the HMAC and states the trust it implies, Appendix A.)

Seed s . The *seed* is the number that starts the random draws in an analysis: the simulations, bootstraps, and random splits whose results depend on where the draws begin. The author does not choose it. Choosing would allow seed shopping (Section 1.2), running the analysis under many seeds and reporting the one that looks best, which Section 5.3’s contribution 8 is built to make visible. Three parts: first, who fixes the seed and who can check it; second, the range it must fall in; and finally, the public rule that produces it. First, who fixes it. The archive’s nonce ν fixes it, and the secret and public halves of that arrangement do different jobs. Making ν takes the archive’s secret k_{arch} , which is why the author could not have known the seed before depositing. Turning ν into s takes no secret at all: the derivation rule is public, so any reader holding ν can recompute s and check it against the seed the

certificate reports. The reader obtains ν from the archive’s signed receipt rather than by recomputing it (Section 3.1). An analysis that consumes no randomness gains nothing from the seed; its outputs then depend only on code and data the archive has hashed.

Second, the range. What the rule must produce is an integer s in $\{1, 2, \dots, 2^{31} - 1\}$: a range that every statistical *runtime* on the version-1 list—the programs that execute analysis code, such as R, Stata, and Python—accepts, the widest range every one of them takes, and fixed as a version-1 constant.

Finally, the rule. It hashes the nonce and cuts the result down to that range. Write Λ for the fixed ASCII *domain string* `certified-runtime-seed-v1|` (lowercase letters, digits, hyphens, and a final vertical bar): a string included solely so that one use of a hash cannot collide with any other use of the same input. Write $a \parallel b$ for the bytes of a followed immediately by the bytes of b , and $a \bmod n$ for the remainder after dividing a by n (the usual integer remainder, in $\{0, 1, \dots, n - 1\}$). Concatenate Λ and ν , hash that sequence— Λ as ASCII bytes, ν as the 32 raw bytes HMAC returned—and take the first eight of the 32 output bytes. Write BE_{64} for reading those eight bytes as a *big-endian unsigned* integer: a non-negative whole number whose left-hand byte carries the high-order 8 bits, the same convention as writing 1234 with the thousands digit first. Then

$$s = 1 + (\text{BE}_{64}(H(\Lambda \parallel \nu)[0..7]) \bmod (2^{31} - 1)).$$

The remainder maps the 64-bit integer into $\{0, 1, \dots, 2^{31} - 2\}$, and adding 1 shifts it into $\{1, 2, \dots, 2^{31} - 1\}$. Reducing a uniform 64-bit integer modulo $2^{31} - 1$ leaves a deviation from uniformity below one part in 2^{33} , negligible for any use made of the seed. A reader recomputes s from the signed ν and requires it to equal the seed recorded in ρ ; a seed that is merely reported is not accepted. Derived seeds use the same reduction: a step’s seed s_i (Section 3.2) and an author’s re-mapped reseed (Section 2.3) are $1 + (\text{BE}_{64}(H(\Lambda' \parallel \text{BE}_{64}(s) \parallel \text{BE}_{64}(i))[0..7]) \bmod (2^{31} - 1))$ under the schema’s per-step and per-reseed domain strings Λ' , with i the step index or the requested seed.

Timestamp proof π . The timestamp proof stops one particular fraud: manufacturing a certificate after the fact and passing it off as contemporaneous with the research. Why stopping that fraud takes a public record no party controls, and the two-edged window the proof closes around the certified run, are stated with the certificate (Section 3.3.2); this unit is the mechanism. Three parts follow: first, the proof object and its three outcomes; second, the service that supplies it, and why nobody need trust that service; and finally, how precise its times are.

First, the object. A *timestamp proof* is a public mark that these exact bytes already existed by a known moment. It says nothing about how much earlier they may have existed. A proof π binds the exact bytes of $\text{enc}(\rho)$ at issue. Checking π returns one of three outcomes. **pending**: the mark is recorded but not yet confirmed on the public record (in version 1, Bitcoin’s blockchain). **confirmed(t_π)**: a time t_π on that list, so $\text{enc}(\rho)$ existed no later than t_π ; confirmed only once the block carrying it is six confirmations deep (the block itself and five after it). The third is **fail**. This specification adds that depth rule on top of the timestamp service’s own semantics, the same rule as the freshness pulse (Section 3.1), so a proof cannot be confirmed one hour and *orphaned*—displaced from the chain by a reorganization—the

next. Pending is not failure; a hash of ρ that disagrees with π is always failure. Confirmation must not require trusting the archive or the author.

Second, the service. Those three outcomes are an interface, and version 1 fills it with one service: OpenTimestamps into Bitcoin (Todd, 2016). Timestamping itself is much older (Haber and Stornetta, 1991). The commercial arrangement is a *timestamping authority*, a company that signs a statement saying it saw your document at a stated moment (Adams et al., 2001); that only moves the trust, since such a signature is worth exactly what the company is. Bitcoin replaces the company with a public record no one owns, but space in that record must be bought and the network accepts only a few entries a second. OpenTimestamps closes that gap by batching strangers together. The archive sends $H(\text{enc}(\rho))$ to a public *calendar* server, which collects the hashes reaching it from many unrelated users over a short interval and folds them into one Merkle tree of its own (an operation list rather than the RFC 6962 tree of Appendix A.3). Periodically the calendar writes that tree’s root into a Bitcoin transaction. The root then belongs to a block that thousands of independent computers keep a copy of. Altering that block means redoing its proof of work faster than the rest of the network combined; proof of work is the deliberately expensive computation that had to be performed to publish the block. The proof π is a small file holding the chain of hash operations that climbs from $H(\text{enc}(\rho))$ to that root, with the height of the block whose header carries it. Checking it means recomputing the climb, reading the named block’s header from the stored header chain (Section 5.1.4) or from any Bitcoin node, confirming the root appears in it, and reading the block’s recorded time. No step of that consults the archive or the author. Sharing the tree is what makes this affordable and harmless. One fee and one transaction per interval cover every document submitted in it: the calendar writes one 32-byte root whatever the crowd’s size, so a document’s share is a fraction of a cent. The public calendars charge nothing to submit, and an archive running a calendar of its own would pay once an interval rather than once a certificate. All that survives of the neighbors in π is a handful of sibling hashes, which disclose nothing about them and demand nothing of whoever submitted them. The proof grows by about one hash per doubling of the crowd, so a million documents in one interval add about twenty. The calendar never becomes a party anyone must trust. It cannot backdate a document, since a climb to a root already in Bitcoin cannot be constructed without breaking H . It cannot revoke one, since π is self-contained and never consults the calendar again, so its proofs survive its shutdown. The worst it can do is decline to include a document, which is obvious at once. A calendar that dies before anchoring would leave a proof permanently pending, *stranded*, so the archive submits every hash to several independent calendars, as the standard client does; one calendar’s death strands nothing. An archive that will not wait for the calendars (the attestation deadline of Section 3.1) commits the pending root itself: it broadcasts a transaction carrying that root, or a purchased transaction that does, and the proof’s climb then ends in that transaction’s block instead.

Finally, precision. The proof is **pending** between submission and the moment the block is mined, normally a matter of hours, which is why pending is an ordinary outcome rather than a failure. And Bitcoin block times are only approximate. The network’s timestamp rules are asymmetric. A node rejects a block stamped more than two hours ahead of its own clock, a rule of node policy, not of consensus. And a block’s stamp need only exceed the median of the previous eleven blocks’ stamps, an original consensus rule (*median time past*,

the quantity [Harding and Todd, 2015](#) defines for transaction lock-times), which trails real time by roughly an hour. Neither rule ties a stamp to real time by itself (the median lags real time by more than two hours for about one block in fifty, [Szalachowski, 2018](#)). So read t_π , and every block time the protocol uses, as a moment good to an hour or two, not to the second. The one finding that sets a block time beside a clock reading, finding 9, allows two hours at each end for this (Section 4.1). Neither imprecision matters for the use made of block times here, where the question is whether a certificate existed before some later event, not exactly when.

A.3. Folding a list into one hash: the Merkle root

Merkle root. A *Merkle tree* folds a long ordered list into one hash, the *Merkle root*, so that changing any item, or the order of items, changes that one hash: pair items, hash each pair, repeat until one value remains. The certified run later builds one such list of step records; its Merkle root is written R_T once that list T is defined (Section 3.2). For an ordered list of records $r_0, \dots, r_{n-1}$, form *leaves* (one hash per record) by $H(0x00 \parallel \text{enc}(r_i))$, where $0x00$ is a single byte of value 0. Then, as long as more than one hash remains, pair them left to right: each pair (a, b) becomes an *interior node* $H(0x01 \parallel a \parallel b)$, where $0x01$ is a single byte of value 1. A leftover hash with no partner is *promoted* (kept as-is into the next round), never copied to fill a pair. The one-byte tags are *domain-separation* prefixes taken from RFC 6962, a published internet standard for hash trees of this kind. They keep a leaf from being mistaken for an interior node: a match across the two prefixes would require two different inputs with one hash, which no one can usefully find (Appendix A.1). The tree is RFC 6962’s exactly. Pairing left to right and promoting the leftover builds, for every list length, the same tree the standard defines by splitting a list at the largest power of two below its length. So a Certificate Transparency library (the public-log system of Section 5.3) computes the same top hash, and the standard’s inclusion and consistency proofs apply to every tree in this protocol unchanged. What this specification adds is the wrap that follows. Write t for the tree’s top hash after pairing is finished (bare t always means this top hash; subscripted t ’s are times), n for the leaf count, spelled as the schema fixes (Appendix A), and Σ for the fixed ASCII domain string `certified-merkle-v1|`. Like Λ , Σ ends in a vertical bar that appears nowhere else in any domain string, so that no domain string can be a byte-prefix of another; it does for whole constructions what the one-byte tags do for tree levels. The Merkle root is H of three pieces concatenated: the bytes of Σ , then those digits, then the 32 bytes of t ,

$$H(\Sigma \parallel n \parallel t).$$

(An empty list has no t ; its root is $H(\Sigma \parallel 0)$, defined for completeness, since the preflight of Section 2.2 keeps a certified run’s step list (Section 3.2) from ever being empty.) Promotion plus the one-byte tags prevents a list and a padded twin (the same list with a dummy extra item made by copying the last leaf) from sharing a root. That is not a hypothetical hazard: Bitcoin’s early duplicate-last-leaf rule let distinct transaction lists share a root, a disclosed flaw its implementation still guards against ([MITRE Corporation, 2012](#)).

B. Requirements of the archive

This appendix collects every duty of the archive role, so that an archive can be built from it, in the order the duties are met: keys once, at installation. Then, per deposit: the draft, what follows the certified run, storage, and the ledger with its proofs. Then the standing machinery (renewal, crash recovery, the interface, and operator change with the key log), restricted files, and revisions. It is a checklist: it names objects rather than redefining them, each defined where it was introduced (Appendix A and sections 2, 3.1 and 3.3). Dataverse already stores replication datasets, issues version-specific DOIs, and freezes a version on publish; certification uses that, and everything else on this list is what Dataverse (or any archive taking the role) adds. The later check of ρ inspects only the protocol items; it does not inspect backups, succession, or the landing page.

Keys and the trust root. Generate one Ed25519 signing key pair and one 32-byte HMAC secret the first time the signing service starts, and again at each rotation the key log records. Generate the archive’s root key once, offline, for signing key-log entries and nothing else. Never copy a private half or the HMAC secret off the device that holds it. Keep both in the most hardened storage the deployment has (a hardware security module where one exists): one compromise of the signing service takes both secrets at once. Section 5.1 states how each failure shows. Publish the root through the recognized certificate software, and deposit the root’s public key, the currently pinned block hash, and the key log under version-specific DOIs (Operator change, below); readers pin the root, never a key that arrives with files.

Draft until the certificate is linked. Accept a closed replication dataset P^* into an unpublished draft; refuse an unrecognized or lapsed certificate-software build (Section 3.1). Hash the bytes received— P^* and the closure report—never taking an author-computed hash, and run the dependency audit on P^* , whose partition the group hashes require (Section 2.1). Read the freshness pulse β —the newest Bitcoin block with six confirmations (Section 3.1)—and record it in γ ; refuse the deposit if no Bitcoin node can be reached. Only then sign the receipt D , including ν , s , *identity_assurance* from the uploader account, and the ledger position its line takes. Append that line, submit $H(\text{enc}(D))$ to the timestamp service, store the returned proof beside D , and serve the two together. Broadcast a transaction of the archive’s own, or a purchased one, at the attestation deadline (Section 3.1). Watch the proof until it confirms, raising the fee if the transaction lags. Reissue for cause a receipt whose proof confirms outside the window (Section 3.1). Keep the draft unpublished.

After the certified run. Accept one ρ per receipt: a second ρ citing a D already witnessed is refused (below T0 an author could otherwise submit several honest attempts; the choice among them is a residual risk Section 5.1 states). Before signing, run every finding that can be evaluated at that moment, and refuse on any failure, so that no ρ is signed that the reader’s check would fail. They are findings 1 through 6 over the deposited files and the submitted ρ , and finding 9 whole, which means waiting for D ’s proof to confirm inside the coherence window. The wait is the calendar’s confirmation time, ordinarily some hours; a run of that length or longer absorbs it, a shorter one does not, and no deadline runs against the

author meanwhile. Recompute the exception mark as finding 5 does, and write that value, never the recorder's, onto the metadata field. Findings 7, 8, and 10 concern the witness, the timestamp proof, and the ledger proofs, which do not yet exist; the pass that gathers π 's confirmation evaluates them and records the result beside ρ . A submission is therefore in one of five states the archive exposes on the metadata field: submitted, refused (with the failing finding), sealed (W signed, π pending), confirmed, and failed (naming the finding). The state *confirmed* reports π 's own outcome of the same name. The failed state is written by the later pass when a proof or a ledger proof fails. For an unchanged submission retry, the archive returns the persisted original response and evidence for the same W ; it does not mint a routine successor witness. If the archive itself made the original evidence unusable, the separate reissue-for-cause mechanism may issue a distinct receipt or witness that names the failed archival object and keeps both objects in the bundle. That recovery path is for archive-caused failure, not for ordinary author retransmission, and it is not a general promise that every interrupted case is automatically reconstructed. The witness W has D 's durability rule (Section 3.1, *Crash recovery* below). Accept a ρ only from the account that deposited its D , or from one that account has authorized for this deposit (a coauthor's, or a replicator's certifying public bytes); W records the submitter. Sign W . Place the timestamp proof π on $\text{enc}(\rho)$. Then store, link, and publish as *Storage*, next, specifies; the linked ρ must be the one W signed.

Storage. The archive keeps the replication dataset under content addressing (Section 2.1): the file's name on disk is its hash, so a later swap would have the wrong name. Store P^* under $H(P^*)$. Write to a temporary file, *flush* to durable storage (force the disk to finish writing), re-read and re-hash, then move into place read-only. A mismatch is a storage-integrity failure and stops the deposit. Before serving, re-hash; a file whose bytes no longer match the name it is stored under is *quarantined* (moved out of the live store, retained for inspection). Serve P^* as the packed byte run itself, one opaque file exempt from ingest (Section 2.1). Store a deposited container image content-addressed by its digest beside P^* , outside C_{size} , and serve it to readers who re-execute (Section 2.1).

After W is signed, store ρ in the archive's certificate store, content-addressed by $H(\text{enc}(\rho))$ —not as a file in the replication dataset. Store beside it, addressed the same way, everything the check reads: D with its timestamp proof, W , π , the certificate's ledger proofs, the key-log entry governing the signing key, and the evidence set (Section 3.6). The ledger proofs are the inclusion proofs and signed, anchored heads of Section 3.3.3, attached in the pass that gathers π 's confirmation. The blobs are the archive's storage exposure: many steps, each writing files up to C_{blob} , can together dwarf C_{size} . An archive prices that exposure in its terms of deposit, which are outside the protocol. The certification metadata field is written by the signing service and is read-only to depositors and curators. On the replication-dataset version about to be published it links to the stored ρ and records the 64-character hexadecimal of $H(\text{enc}(\rho))$. It also records the tier, the exception mark, the seed-sensitivity summary, the count of records in Q (Section 3.2), the counts of records excluded from the summary as loops and of records the author-callable command emitted (Appendix C), the count of compiled artifacts matching no registry digest (Section 2.3), the re-execution coverage n_c/n and the count of oversize outputs (Section 3.2), and the state of *After the certified*

run, above. The seed-sensitivity summary gives the diagnostic state, coverage counts, and largest movement between the two seeds in baseline-standard-error units, where that scale is available (Section 2.2). The version is published only after the archive has checked that the linked ρ 's D names this version's $H(P^*)$, a match the landing page states—so a passing certificate cannot be pointed at a version whose files it did not cover. Publish only after P^* , the stored ρ , and that field are in place, so the version-specific citation (the DOI with its version selector, where the archive issues one DOI per dataset, Section 4.3) names a frozen set.

Ledger, proofs, and snapshots. Append D 's line when the receipt is issued, W 's when the certificate is sealed, and P 's when the version is published, each a leaf of the tree of Appendix A.3; P 's document is the metadata field's contents as at publication, the mutable state excluded, under statement type **publication**. At the anchoring interval (Section 3.3.3), sign a head naming the tree's size and root, submit its hash to the timestamp service, and retain the proof. Serve signed heads with their proofs, an inclusion proof for any entry, and a consistency proof between any two heads; attach each certificate's own proofs to its stored documents in the pass that gathers π 's confirmation. At least daily, deposit a snapshot of the whole ledger, signed by the signing key with the protocol context, as an ordinary archival object under its own version-specific DOI. The snapshot holds every leaf with the document its hash names (D , W , K , or P , each with its signature), every signed head with its proof, and the key log. Other archives mirror the snapshot, and the mirror, not this archive's copy, is the source it cannot later revise, so that finding 10's reference head survives this archive. The same (P^*, v, id^-) replays the same D and does not append a second D , unless that D was superseded for cause, in which case the successor is served. A certificate signed under a different v is treated as absent, so the replication dataset is re-deposited under the current v , naming the earlier deposit as predecessor; the re-issue is then a dated link rather than a fresh draw without a trail. Do not rewrite a published version, and never withdraw one that carries a certification field to a bare tombstone: a withdrawn version's page keeps serving the certificate bundle and states the withdrawal, so a doctored successor cannot hide the version it replaced. A landing page should also display four counts from the ledger. They are an uploader's deposits naming the same predecessor, the uploader's receipts never followed by a witness, the receipts the archive superseded for cause, and the uploader's witnessed certificates never followed by a publication line. The superseded receipts are shown, not excluded, so an archive's re-draws are counted. Beside them, from the snapshot, it shows the number of receipts across all uploaders sharing this deposit's bytes or any of its group hashes, and the days between chained versions. The seed-shopping trail of Section 5.1 is then a display rather than an investigation. The counters attach to one account at one archive; only ORCID assurance ties accounts to a person. An honest run that failed under s and was fixed leaves the same abandoned receipt a shopper does, so a count is read against an author's total, never as a verdict.

Renewal. Two duties keep old certificates checkable as the cryptography ages (Section 5.1.4 gives the argument). Periodically re-timestamp the stored evidence under the successor hash while SHA-256 still stands, and into a successor public record should Bitcoin's stop being

maintained. And store the Bitcoin block headers—the whole header chain, once—beside the proofs, so that the check runs with no network at all.

Crash recovery. If the archive’s computer stops halfway through taking a deposit, finish what was completed and throw away what was not. Every deposit writes a work-log entry (a note of work in progress) before making the record permanent, and writes the finished record so it survives a crash. On startup, rebuild the lookup table in working memory from finished records, then compare unfinished notes to the ledger. An entry whose deposit the ledger already has is leftover and is cleared; an entry with no ledger record is unfinished and its upload is discarded. Return D to the author only after its ledger line is durable, so a receipt the author holds is never one a crash could orphan; and refuse a ρ citing a D the ledger does not hold.

Interface. The interface is the small set of requests the archive answers. Two writes that require a login: accept a deposit and return D ; accept a finished ρ and return W . Public reads: the issued D with its timestamp proof; the certificate bundle by the version DOI; and the signed heads with their proofs. Further public reads: an inclusion proof for any entry, a consistency proof between any two heads, the key log, and *liveness* (whether the archive is responding). The certificate bundle is ρ with every document stored beside it, returned in one read, so a reader never assembles it by hand. Uploads use a secret of at least sixteen characters that resolves to an uploader account; what the archive knows about that account is written into D as *identity_assurance*. Because these reads need no login, the archive cannot use credentials to decide who may run the check on a published deposit; for restricted deposits, access control governs who holds the files (Section 3.5), never what the check computes from them.

Operator change and the key log. When a new archive takes over from an old one, nothing already signed is signed again. A certificate is judged under the key that signed it, and never under another key. What a reader pins across key changes is the archive’s root, which signs the *key log*: entries of kind K on the ledger. Each entry carries a public key with its key id ($H(k_{\text{pub}})$; the hash alone would not reconstruct the key an offline reader needs), its validity interval, its successor, and the recognized certificate-software builds with their recognition intervals (Section 3.1). The key id stamped into every D and W names which entry applies. Validity is judged at the public dates the check reads, never at the archive’s own asserted times. Those dates are the time of the block carrying the receipt’s proof for D (Section 3.1), the confirmed time of the head covering W ’s line for the witness, and a head’s own confirmed time for a head. A certificate whose dates fall outside its key’s interval is judged by the interval, not by the signature alone. A rotation is itself a ledger line, anchored and timestamped like every other, so a thief cannot backdate one. The log’s entries cover the HMAC secret’s generations on the same terms, so k_{arch} rotates with the signing key rather than living forever. A retiring key’s interval closes no earlier than the latest confirmed public date of any statement signed under it (a witness’s covering head, a receipt’s proof, a head’s own proof), so that a calendar’s delay never fails an honest certificate. A snapshot’s weight is its DOI deposit and the anchored heads inside it, not its signature’s date. A rotation

cross-signed by a stolen key is one a thief can sign too, so the root, not the cross-signature, is what a reader trusts. The root key stays offline and is used for nothing else, and its own succession is a new recognized checker release and a new DOI deposit. Every verdict is relative to the newest key log the reader holds: an interval closed after that log was taken is invisible offline, and the online check fetches the current log first. The ledger either continues under one key or begins a new tree whose first entry names the preceding head. A renderer of ρ (Section 3.1) shows archive identity, which archive now holds the files, and $H(\text{enc}(\rho))$; it must produce the same page from the same fields, so anyone can regenerate the page from the downloaded files.

Restricted files. Files the archive may hold but not publish are a deposit under restricted access (Section 3.5): store P^* and the evidence set, serve them to approved readers only, and publish ρ and its documents as usual. Files the archive may not hold at all have no route in version 1.

Revisions. A later, different replication dataset is a new draft, a new D , a new seed, a new ρ , a new metadata link, and a new published version. Unchanged bytes under an unchanged predecessor claim replay the same D and are not a new version. The predecessor link is the earlier version’s *id*, signed into the new D . Files beside the packed P^* in the same version (a README, the article) are outside the certificate: the renderer lists them as uncertified, and changing them is not a revision.

C. Byte-level rules of closing and deposit

This appendix collects the byte-level rules whose point the body states and whose spelling only an implementer needs. Each unit names the body unit it serves, and the version-1 schema (Appendix A) holds the same spellings in machine-readable form. In the body’s order: paths; the environment capsule’s fields; the preflight list; the frozen settings by name; the capture code’s mechanism; and the dependency audit’s mechanism. Then the seed sensitivity’s grouping rules; the normalization list; the receipt’s fields; the capture file’s number spelling; and finally salting for restricted deposits.

Paths. The rules for a path in the style of Section 2.1. Write paths as UTF-8 (the standard byte encoding of Unicode text). Put them in Unicode normalization form C first, the standard single spelling for accented characters, so that the same name typed on macOS and on Linux gives the same bytes. Otherwise an accented character can be stored either as one character or as a letter plus a combining mark. Then compare as bytes, which is why letter case matters. No component may be $.$, $\dots$, or empty. No path may contain a control character (a byte below 0x20, the range that includes tab and newline): the group hash (h_G , Section 2.1) writes one path per line, and a path holding a newline could make two different sets of files hash alike. Ordinary spaces are allowed, since every line ends in a fixed-width hash. Finally, a path names a regular file: symbolic links (names that point at another file, like a shortcut) are not followed and are not listed. A link whose target lies outside the

replication dataset is therefore an undeclared external reference, and it stops certification through the dependency audit (Section 2.3) rather than passing as a file of its own.

Environment capsule fields. The capsule κ (Section 2.1) is encoded with `enc` and stored at the fixed path `environment.json`. It contains the operating system name and version, the processor architecture, and, for every language the targets will run (Section 2.2), the language name and its exact version string. For every add-on library the dependency audit (Section 2.3) finds the scripts loading, it contains the name and the exact version (vendored inside P^* , or in the deposited image, Section 2.3). It contains the numerical library that performs the linear algebra: the implementation and version of the code that multiplies and inverts matrices, which makes two machines disagree in the last digits. It contains the C mathematics library the runtime links, the source of exponentials, powers, and logarithms, whose results also move across versions. It also contains the frozen settings actually in force (Section 2.2), the container image’s content digest when one is deposited (Section 2.1), and the certificate software’s own version and source hash, the pair $(v_{\text{tool}}, h_{\text{tool}})$ of Section 3.1 (tool abbreviating the certificate software in every subscript).

Preflight list. Before closure (Section 2.2), reject on any of the following. The replication dataset is not a directory (a folder). There is no empty, writable place for the scratch copy, the run’s temporary working copy (Section 3.2). A code group has no file (nothing to run). Two paths differ only in letter case (a tree that cannot be unpacked on the common case-insensitive filesystems). A directory is empty: packing carries only files, so a run that writes into a pre-existing empty folder would fail on re-execution. A runtime required by the scripts present is missing (R is missing when the replication dataset has `.R` files, and likewise for Stata and Python). The tree is larger than the size ceiling C_{size} , fixed at one gigabyte (10^9 bytes) before encoding. The archive is unreachable.

Frozen settings, by name. The settings of Section 2.2, as the schema names them. Clocks and formatting: on P^* set the process timezone to UTC (Appendix A.1) and the process locale (the settings that choose date format and decimal mark) to `C` (the plain C-language default: a period for decimals, not a comma; dates in one fixed spelling). Set `SOURCE_DATE_EPOCH` to the schema’s fixed value—the Unix second for 2024-06-01 00:00:00 UTC—a standard process setting that freezes the “date of this build,” so output files do not embed today’s date. Cores: every setting on the schema’s threading list is set to 1, and where a numerical library dispatches different code by instruction set, its reproducibility switch (Intel’s `MKL_CBWR`) is set to its fixed branch. Hash randomization: the per-language setting (`PYTHONHASHSEED` for Python) is on the schema’s frozen-settings list and is frozen to the step’s seed s_i (Section 3.2). The list also disables Python’s bytecode cache (`PYTHONDONTWRITEBYTECODE`) and R’s saved workspace and history, whose files would otherwise enter o carrying a copy time.

Capture code: mechanism. This unit is the mechanism behind Section 2.2, in three parts: first, how the code arrives and what its loading line does; second, how the capture channel is protected; and finally, what capture code may and may not do.

First, how the code arrives. It arrives as files, not as edits to the author’s statements: capture code writes its own files into the code group and inserts a single loading line at the top of each target (Section 2.2). The line goes at the first position the language permits, after any prefix the language itself mandates come first, as the schema’s per-language rule fixes. Such a prefix is a shebang (the `#!` first line naming a script’s interpreter), an encoding declaration, or Python’s `__future__` imports. The added files register hooks (small routines the language itself calls at set moments) that fire after each estimation command in every language the replication dataset uses. The loading line does four things before any author code executes. First, it applies the step’s seed s_i (Section 3.2). Second, it masks the language’s set-seed call, so that the author’s later reseeds are re-mapped through s (Section 2.3). Third, it sets the determinism state that no process setting of Section 2.2 reaches, the schema’s per-language list. Stata alone needs `set processors 1`, since Stata/MP, the multiprocessor edition, ignores the threading settings and the command exists only there; and it needs `set sortseed` to s_i , since its `sort` otherwise breaks ties at random. Finally, it prints the seed banner, a schema-fixed line naming that seed, into the step’s transcript, which later makes seed delivery checkable (Section 4.1). Nothing of the author’s is rewritten, which keeps the difference between their code and the deposited code small enough to audit. The text of that loading line and the contents of the added files are fixed in the version-1 schema (Appendix A), not composed afresh for each deposit. They have to be, since otherwise two implementations would build different closed copies from the same P and compute different h_c for it. The division of labor is the usual one. This paper fixes what the capture code must do: after each estimation command on the schema’s per-language list, read the runtime’s stored results and emit one record. The schema fixes how each runtime is read and how the digits are formatted.

Second, how the channel is protected. Because a capture file is written by code running in the author’s process, author code could write into it too. Its schema-named path is therefore on the dependency audit’s write-pattern family (Section 2.3). Closing stops before deposit when a match is found (Section 3.6), while the author can still rename the output, rather than at a reader’s check years on. Only the seed banner and the re-map lines (one framed line per re-mapped reseed, naming the request and the seated seed) travel in the transcript, each framed by a schema-fixed delimiter, so seed delivery stays checkable from the log (finding 2, Section 4.1). The check also verifies that the capture code deposited is the schema’s. Finding 1 compares every added file to the schema’s canonical file for its language, and each touched target to its original plus exactly the loading line (Section 4.1), so a doctored hook cannot enter the record. The schema also fixes one author-callable capture command—a registered emit-to- Q call a hand-written estimator may invoke, its records labeled author-emitted in Q —so an analysis whose estimator matches no listed command can still bind its estimates instead of leaving Q thin (Section 5.1).

Finally, what capture code may and may not do. After the loading line’s four duties, capture code may only read and print. It must not change what the author’s code computes, since a run of altered code would not be a run of the author’s analysis, and the determinism tier would be measuring something else. Read-and-print states the goal, not the mechanism, and the mechanism differs by language; the schema names each. R and Python register hooks (a masked function binding; an import hook), which alters interpreter state the audit exempts by rule. Stata has no post-command hook and resolves its built-in commands ahead

of any user path. There, the capture code therefore reads each estimation's results back from the log the run writes, a parse the closure runs exercise, at the digits the log prints and only for estimations the log shows. Section 3.2 states the limit and its remedy. The constraint that matters is that none of it may alter the numbers the author's code computes. Beyond finding 1's byte comparison, nothing in the later check inspects what the added lines do. What carries the constraint is the certificate-software build: the archive signs only a build it recognizes by $(v_{\text{tool}}, h_{\text{tool}})$ (Section 3.1), so the code doing the inserting is published and held fixed rather than assembled per deposit. That defense has the limit of any self-report, that a modified build which returns the expected hash is not caught, and the limit is stated where the build check is. The startup hook of Section 3.2 repeats the loading line's duties for a process a target launches (`system`, `shell`, `subprocess`, and `kin`). It is installed in the run's environment outside the tree (R's profile, Python's site hook, Stata's system ado-path) and runs the child under the caller's s_i . The hook acts only when the schema's process setting shows a loading line already ran: in the target's own process it fires before the loading line and does nothing, and every process launched after the loading line inherits the setting and is seeded, masked, and captured. Its banner is always framed as nested; only the loading line prints the counted frame, so finding 2 counts the target's own, and the banner is printed from the value actually seated, not from a separate computation. A launch that suppresses the hook or clears the environment would leave the child unseeded and uncaptured, so closure names such a line and stops; R's `-vanilla` and `-no-init-file`, Python's `-I`, `-S`, and `-E`, and a launch with a fresh environment are the cases. Every observed process is recorded in the step's process observations with its executable's path and hash. An executable is exempt from the mark when it is a hooked interpreter with a hook record (below), a text file the audit read, a file whose hash matches an artifacts-manifest entry with a registry digest, or a file under a runtime root recorded in κ ; from every other observed executable finding 5 derives the exception mark *unhooked process* (Section 3.6). ρ lists each observed executable with its manifest status, so a utility the run called (a document converter, a compressor) is exempt by its registry digest while an author's own binary, vendored or built into a custom image layer, is marked. The hook does not take its seed from the environment, which author code owns: the certificate software writes the hook file for each run with s , the scratch root, and a record path baked in, and the hook appends one record per process (process id, parent process id, seated seed) to a schema-named file outside the tree, kept in the evidence set (Section 3.6). Finding 5 requires, for every observed process whose executable is a hooked interpreter, a hook record naming the enclosing step's s_i ; an interpreter observed with no record, or with another seed, yields the unhooked-process mark. Family 1 lists any reference in executed code to the two seed settings or to the variables that carry the hook, and a match marks the certificate rather than disclosing it. Every target, and every child through the environment the hook carries, is launched with the flags that skip the machine's user startup files: Python's `-s`, R's `-no-environ` with `R_ENVIRON_USER` and `R_LIBS_USER` empty, and Stata's `S_ADO` pointing its personal and plus directories at empty ones. The hook lives in the run's environment rather than in P^* , so it is fixed by the same build and compared by no finding.

Dependency audit: mechanism. This unit is the mechanism behind Section 2.3, in three parts: first, the fetch layer and the library root; second, the reseed masks; and finally, what is audited where.

First, the fetch layer and the library root. A reference is resolved from the tree root, the working directory of every step (Section 3.2). The fetch layer is the only channel through which a locator is resolved, and it does no version resolution of its own. It fetches bytes and requires them to match the declared hash. It places each file at the path the declaration names under the *dependency root*, the schema-named directory inside the scratch copy that holds fetched files. It writes into ρ the record of what was reached and its hash, so that the record the check reads (finding 5, Section 4.1) has a defined producer. A declared file must not carry a code extension, no target may invoke a file under the dependency root, and P^* holds nothing under that root. Which files are the runtime, exempt from the audit and recorded in κ , and which are add-ons is fixed per language by the schema: the runtime’s own installation roots are the runtime, and everything under a user library path is an add-on. A vendored library is a project-local library under the schema-named library root; a *lockfile* (`renv`’s, or Python’s hash-checking install) is how an author reproduces one exactly. It costs space against C_{size} , a price stated rather than hidden. Its files are other inputs, never targets, and exempt from the read patterns (Sections 2.1 and 2.2) but not from the reseed and capture-path families. A reseed inside a vendored package is disclosed like one in a script. A vendored file whose bytes differ from the public registry’s record for that package is named in ρ , and a vendored file that names a capture file’s path stops certification. When a container image is deposited its text files are audited like the root’s, and the image is part of the evidence set (Section 3.6). The artifact manifest of Section 2.3 covers the image’s and the root’s compiled and byte-compiled files. The registry is the schema’s per-language package index (CRAN, PyPI, the image registry). A match means the file’s hash equals the one that index publishes for the same package, version, and platform, and a file the index does not list by hash is “none”. Finding 5 recomputes the manifest from the image in hand and reports the image half unevaluated when the image is not.

Second, the masks. The set-seed mask is installed where package code resolves the call—R’s base namespace binding and Python’s module attributes—not only in the script’s own scope, so a reseed inside a package is re-mapped too. The mask family covers the call every keyword-argument seed reaches (Python’s `RandomState`, which `scikit-learn`’s and `SciPy`’s `random_state` arguments construct) and R’s direct assignment to the generator’s state. Two requests collide in the seed range with probability about $B^2/2^{32}$ for B distinct requests, negligible below a few thousand; a non-integer or absent argument is seated as the schema fixes and recorded as such. The capture files carry their own hash implementation and load nothing from the author’s tree, so the re-mapping arithmetic is the schema’s, not the author’s.

Finally, what is audited where. The read patterns run over the scripts; compiled artifacts are unaudited and listed. The launch of a separate process is an arrow of the invocation graph of Section 2.2, so the launched file is reachable for the families and is not itself a target (Appendix C).

Seed sensitivity: grouping and pairing. The rules behind the sensitivity summary of Section 2.2. The schema pairs only unique captured command identities in the initial supported scope. A candidate identity consists of step, script, source line or recorded call, command, dependent variable, and the set of coefficient names. A baseline identity and certified-run identity must each occur exactly once, and the identity must contain enough semantic information to distinguish the estimation. Repeated identities are ambiguous; they are not paired positionally and not reported as measured zero movement. Records present under one seed only are unpaired. Coefficients with invalid numbers, non-finite movement, missing comparison values, or no positive finite baseline standard error are disclosed through the invalid, paired, and standardized coverage counts rather than folded into a flip or zero convention. The first closure run is the baseline.

Normalization list. Before comparison (Section 2.2), strip PDF creation timestamps and producer strings (the name of the program that wrote the PDF), log-header stamps, document date stamps, and the header timestamp every Stata .dta file carries. Strip the per-entry times inside zip-based office formats (.xlsx, .docx) and PDF dates wherever the writer places them (compressed object streams and metadata packets included). Strip runtime-metric lines in logs (elapsed time, memory) and uninitialized metadata padding in binary data files (unused leftover bytes a program did not set). Whether a stripped region is removed or zero-filled is fixed per category in the schema.

The receipt's fields. For the receipt D (Section 3.1), the archive signs $\text{enc}(D)$ where D contains the fields below, in this order. First: statement type **deposit**; archive name; id ; $H(P^*)$; h_c, h_d, h_u, h_e ; t_{arr} ; and the uploader account identifier, a stable pseudonym the archive assigns. The pseudonym is resolved to a person only on the landing page of a published version, so the daily snapshot names no one who has not yet published. The pseudonym is stable because the counters need it, so once any version is published every deposit under it is attributable. That a deposit exists, with its predecessor link, is public from the day of the receipt, published or not. Second: $identity_assurance \in \{\text{shared}, \text{account}, \text{orcid}\}$ (a shared login, a person registered with the archive, or a person signed in through ORCID); the predecessor id^- when the author declared one, else a recorded absence (Section 3.4); and protocol context (v , the schema hash h_{schema} , and the archive key id, Section 3.1). Finally: the recognized certificate-software build ($v_{\text{tool}}, h_{\text{tool}}$); the ledger position the receipt's line takes; the freshness pulse β (Section 3.1); ν ; s ; the closure hash (Section 2.2); and, for a receipt reissued for cause, the superseded receipt's ledger document hash and confirmed time, else a recorded absence (Section 3.1).

The capture file's number spelling. Each record of the result capture Q (Section 3.2) is a single JSON object per estimation, carrying no floating-point values. Each number is written in one canonical hexadecimal floating-point spelling, or as a pair of integers if it is a ratio. The spelling is the form Python's `float.hex()` prints: a sign, `0x1.`, thirteen hexadecimal digits, `p`, and a signed decimal exponent, with the forms it prints for zero and subnormal values. The capture code converts into that form what C's `%a` and Stata's `%21x` print, since the three formatters spell one value three ways. That form is an exact, byte-fixed

spelling of the binary value, so no formatter’s rounding enters. Non-finite values are spelled as the schema fixes. The per-estimate values and the summary’s movement in ρ are spelled the same way.

Salting for restricted deposits. For a restricted deposit (Section 3.5), every per-file hash a public document carries is salted. Those hashes are the per-file lines over which the group hashes are computed (Section 2.1), the per-file output hashes, the step records’ written-file, transcript, and capture-file hashes in ρ , the declared dependencies’ hashes, and a redacted Q value. A redacted Q value is salted under its path, position, and field, so equal withheld values hash differently. A restricted deposit’s group hashes, and the D that signs them, are therefore computed over salted lines. Each published file hash is $H(\text{salt} \parallel \text{bytes})$ under a per-file random 32-byte salt. The salt is HMAC-SHA-256, the standard keyed hash built from SHA-256 (Bellare, Canetti and Krawczyk, 1996), of the file’s path under the deposit’s random file R named next. One secret made where the hashing happens thus yields an independent salt per file and per certificate. The salt is stored with the restricted evidence set, which the file-recomputing findings consume for approved readers, so an approved reader still verifies while an outsider cannot test a guess. The whole-object hashes cannot be salted, because $H(P^*)$, the group hashes, and h_x are the protocol’s own equations. A restricted deposit therefore adds one schema-named file R of 32 random bytes to the other-inputs group u before closing (a reuse of the letter: R_T and R_{M_i} are Merkle roots). Every hash over the tree then commits to bytes no outsider can guess: the whole-tree and per-group confirmation oracles close for the price of one file. Every manifest root is covered the same way because R appears in every manifest.

References

- Adams, Carlisle, Pat Cain, Denis Pinkas and Robert Zuccherato. 2001. “Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP).” RFC 3161, RFC Editor. <https://www.rfc-editor.org/rfc/rfc3161>
- Association for Computing Machinery. 2020. “Artifact Review and Badging, Version 1.1.”. <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- Becker, Gary S. 1968. “Crime and Punishment: An Economic Approach.” *Journal of Political Economy* 76(2):169–217. <https://doi.org/10.1086/259394>
- Bellare, Mihir, Ran Canetti and Hugo Krawczyk. 1996. Keying Hash Functions for Message Authentication. In *Advances in Cryptology — CRYPTO ’96*. Vol. 1109 of *Lecture Notes in Computer Science* Springer pp. 1–15. <https://cseweb.ucsd.edu/~mihir/papers/kmd5.pdf>
- Bernstein, Daniel J., Niels Duif, Tanja Lange, Peter Schwabe and Bo-Yin Yang. 2012. “High-Speed High-Security Signatures.” *Journal of Cryptographic Engineering* 2(2):77–89. <https://ed25519.cr.yp.to/ed25519-20110926.pdf>
- Birkholz, Henk, Antoine Delignat-Lavaud, Cédric Fournet, Yogesh Deshpande and Steve Lasker. 2026. “An Architecture for Trustworthy and Transparent Digital Supply Chains.” RFC 9943, RFC Editor. <https://www.rfc-editor.org/rfc/rfc9943>

- Bonneau, Joseph, Jeremy Clark and Steven Goldfeder. 2015. “On Bitcoin as a Public Randomness Source.” IACR Cryptology ePrint Archive, Report 2015/1015. <https://eprint.iacr.org/2015/1015>
- Brinckman, Adam, Kyle Chard, Niall Gaffney, Mihael Hategan, Matthew B. Jones, Kacper Kowalik, Sivakumar Kulasekaran, Bertram Ludäscher, Bryce D. Mecum, Jarek Nabrzyski, Victoria Stodden, Ian J. Taylor, Matthew J. Turk and Kandace Turner. 2019. “Computing Environments for Reproducibility: Capturing the “Whole Tale”.” *Future Generation Computer Systems* 94:854–867. <https://doi.org/10.1016/j.future.2017.12.029>
- Brodeur, Abel, Derek Mikola, Nikolai Cook et al. 2024. Mass Reproducibility and Replicability: A New Hope. Discussion Paper 16912 IZA Institute of Labor Economics. <https://docs.iza.org/dp16912.pdf>
- Brodeur, Abel, Derek Mikola, Nikolai Cook et al. 2026. “Reproducibility and Robustness of Economics and Political Science Research.” *Nature* 652:151–156. <https://doi.org/10.1038/s41586-026-10251-x>
- Broockman, David, Joshua Kalla and Peter Aronow. 2015. “Irregularities in LaCour (2014).”. https://stanford.edu/~dbroock/broockman_kalla_aronow_lg_irregularities.pdf
- Casas, Santiago and Christian Fidler. 2024. “TOPO: Time-Ordered Provable Outputs.” arXiv:2411.00072 [astro-ph.IM]. <https://arxiv.org/pdf/2411.00072>
- Chalkias, Konstantinos, François Garillot and Valeria Nikolaenko. 2020. Taming the Many EdDSAs. In *Security Standardisation Research (SSR 2020)*. Springer pp. 67–90. <https://eprint.iacr.org/2020/1244>
- Chang, Andrew C. and Phillip Li. 2022. “Is Economics Research Replicable? Sixty Published Papers from Thirteen Journals Say ‘Often Not’.” *Critical Finance Review* 11(1):185–206. <https://doi.org/10.1561/104.000000053>
- Chirigati, Fernando, Rémi Rampin, Dennis Shasha and Juliana Freire. 2016. ReproZip: Computational Reproducibility With Ease. In *Proceedings of the 2016 International Conference on Management of Data (SIGMOD)*. pp. 2085–2088. <https://doi.org/10.1145/2882903.2899401>
- Christensen, Garret and Edward Miguel. 2018. “Transparency, Reproducibility, and the Credibility of Economics Research.” *Journal of Economic Literature* 56(3):920–980. <https://doi.org/10.1257/jel.20171350>
- Christian, Thu-Mai, Sophia Lafferty-Hess, William G. Jacoby and Thomas M. Carsey. 2018. “Operationalizing the Replication Standard.” *International Journal of Digital Curation* 13(1):114–124. <https://doi.org/10.2218/ijdc.v13i1.555>
- Chuat, Laurent, Pawel Szalachowski, Adrian Perrig, Ben Laurie and Eran Messeri. 2015. Efficient Gossip Protocols for Verifying the Consistency of Certificate Logs. In *2015 IEEE Conference on Communications and Network Security (CNS)*. pp. 415–423. <https://doi.org/10.1109/CNS.2015.7346853>

- Costan, Victor and Srinivas Devadas. 2016. “Intel SGX Explained.” IACR Cryptology ePrint Archive, Report 2016/086. <https://eprint.iacr.org/2016/086>
- Data Access and Research Transparency (DA-RT). 2015. “Data Access and Research Transparency (DA-RT): A Joint Statement by Political Science Journal Editors.” *State Politics & Policy Quarterly* 15(4):423–424. <https://doi.org/10.1177/1532440015598318>
- Dewald, William G., Jerry G. Thursby and Richard G. Anderson. 1986. “Replication in Empirical Economics: The Journal of Money, Credit and Banking Project.” *American Economic Review* 76(4):587–603. <https://www.jstor.org/stable/1806061>
- Di Cosmo, Roberto. 2020. Archiving and Referencing Source Code with Software Heritage. In *Mathematical Software – ICMS 2020*. Vol. 12097 of *Lecture Notes in Computer Science* Springer pp. 362–373. https://doi.org/10.1007/978-3-030-52200-1_36
- Drand and The League of Entropy. 2026. “drand: A Distributed Randomness Beacon Daemon.” <https://drand.love/>.
- Dwork, Cynthia and Moni Naor. 1993. Pricing via Processing or Combatting Junk Mail. In *Advances in Cryptology—CRYPTO ’92*. Vol. 740 of *Lecture Notes in Computer Science* Springer pp. 139–147. https://doi.org/10.1007/3-540-48071-4_10
- Eubank, Nicholas. 2016. “Lessons from a Decade of Replications at the Quarterly Journal of Political Science.” *PS: Political Science & Politics* 49(2):273–276. https://www.nickeubank.com/wp-content/uploads/2019/10/Eubank_LessonsFromADecadeOfReplications.pdf
- Ferguson, Joel, Rebecca Littman, Garret Christensen, Elizabeth Levy Paluck, Nicholas Swanson, Zenan Wang, Edward Miguel, David Birke and John-Henry Pezzuto. 2023. “Survey of Open Science Practices and Attitudes in the Social Sciences.” *Nature Communications* 14:5401. <https://doi.org/10.1038/s41467-023-41111-1>
- Fišar, Miloš, Ben Greiner, Christoph Huber, Elena Katok, Ali I. Ožkeš and the Management Science Reproducibility Collaboration. 2024. “Reproducibility in Management Science.” *Management Science* 70(3):1343–1356. <https://doi.org/10.1287/mnsc.2023.03556>
- Fisch, Ben. 2018. “PoReps: Proofs of Space on Useful Data.” Cryptology ePrint Archive, Paper 2018/678. <https://eprint.iacr.org/2018/678>
- Garg, Sanjam, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmood, Guru-Vamsi Policharla and Mingyuan Wang. 2023. Experimenting with Zero-Knowledge Proofs of Training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. pp. 1880–1894. <https://doi.org/10.1145/3576915.3623202>
- Gentzkow, Matthew and Jesse M. Shapiro. 2014. “Code and Data for the Social Sciences: A Practitioner’s Guide.” University of Chicago. <https://web.stanford.edu/~gentzkow/research/CodeAndData.pdf>

- Gipp, Bela, Norman Meuschke and André Gernandt. 2015. Decentralized Trusted Timestamping using the Crypto Currency Bitcoin. In *Proceedings of the iConference 2015*. Newport Beach, CA: . <https://arxiv.org/pdf/1502.04015>
- Goldberg, Sharon, Leonid Reyzin, Dimitrios Papadopoulos and Jan Včelák. 2023. “Verifiable Random Functions (VRFs).” RFC 9381, RFC Editor. <https://www.rfc-editor.org/rfc/rfc9381>
- Goldwasser, Shafi, Silvio Micali and Charles Rackoff. 1989. “The Knowledge Complexity of Interactive Proof Systems.” *SIAM Journal on Computing* 18(1):186–208. <https://doi.org/10.1137/0218012>
- Golle, Philippe and Ilya Mironov. 2001. Uncheatable Distributed Computations. In *Topics in Cryptology — CT-RSA 2001*. Springer pp. 425–440. https://doi.org/10.1007/3-540-45353-9_31
- Gondrom, Tobias, Ralf Brandner and Ulrich Podesch. 2007. “Evidence Record Syntax (ERS).” RFC 4998, Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc4998>
- Haber, Stuart and W. Scott Stornetta. 1991. “How to Time-Stamp a Digital Document.” *Journal of Cryptology* 3(2):99–111. <https://nakamotoinstitute.org/library/time-stamp-digital-document/>
- Harding, Thomas and Peter Todd. 2015. “Median Time-Past as Endpoint for Locktime Calculations.” Bitcoin Improvement Proposal 113. <https://github.com/bitcoin/bips/blob/master/bip-0113.mediawiki>
- Hardwicke, Tom E., Maya B. Mathur, Kyle MacDonald, Gustav Nilsson, George C. Banks, Mallory C. Kidwell, Alicia Hofelich Mohr, Elizabeth Clayton, Erica J. Yoon, Michael Henry Tessler, Richie L. Lenne, Sara Altman, Bria Long and Michael C. Frank. 2018. “Data Availability, Reusability, and Analytic Reproducibility: Evaluating the Impact of a Mandatory Open Data Policy at the Journal Cognition.” *Royal Society Open Science* 5(8):180448. <https://doi.org/10.1098/rsos.180448>
- Harnik, Danny, Benny Pinkas and Alexandra Shulman-Peleg. 2010. “Side Channels in Cloud Services: Deduplication in Cloud Storage.” *IEEE Security & Privacy* 8(6):40–47. <https://doi.org/10.1109/MSP.2010.187>
- Heil, Benjamin J., Michael M. Hoffman, Florian Markowetz, Su-In Lee, Casey S. Greene and Stephanie C. Hicks. 2021. “Reproducibility standards for machine learning in the life sciences.” *Nature Methods* 18(10):1132–1135. <https://doi.org/10.1038/s41592-021-01256-7>
- Herndon, Thomas, Michael Ash and Robert Pollin. 2014. “Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff.” *Cambridge Journal of Economics* 38(2):257–279. <https://doi.org/10.1093/cje/bet075>

- Hurlin, Christophe and Christophe Pérignon. 2024. “The Role of Third-Party Verification in Research Reproducibility.” *Harvard Data Science Review* 6(2). <https://hdsr.mitpress.mit.edu/pub/nqw12udq>
- Ioannidis, John P. A., David B. Allison, Catherine A. Ball, Issa Coulibaly, Xiangqin Cui, Aedín C. Culhane, Mario Falchi, Cesare Furlanello, Laurence Game, Giuseppe Jurman, Jon Mangion, Tapan Mehta, Michael Nitzberg, Grier P. Page, Enrico Petretto and Vera van Noort. 2009. “Repeatability of published microarray gene expression analyses.” *Nature Genetics* 41(2):149–155. <https://doi.org/10.1038/ng.295>
- Jacoby, William G. 2018. “The American Journal of Political Science Replication and Verification Policy.” *American Journal of Political Science*. <https://ajps.org/wp-content/uploads/2019/03/ajps-replic-and-verif-policy-2-27-18.pdf>
- Jia, Hengrui, Mohammad Yaghini, Christopher A. Choquette-Choo, Natalie Dullerud, Anvith Thudi, Varun Chandrasekaran and Nicolas Papernot. 2021. Proof-of-Learning: Definitions and Practice. In *2021 IEEE Symposium on Security and Privacy (S&P)*. pp. 1039–1056. <https://arxiv.org/abs/2103.05633>
- Kahneman, Daniel. 2014. “A New Etiquette for Replication.” *Social Psychology* 45(4):310–311. <https://doi.org/10.1027/1864-9335/a000202>
- Kelsey, John, Luís T. A. N. Brandão, Rene Peralta and Harold Booth. 2019. A Reference for Randomness Beacons: Format and Protocol Version 2. Technical Report NIST IR 8213 (Initial Public Draft) National Institute of Standards and Technology. <https://nvlpubs.nist.gov/nistpubs/ir/2019/NIST.IR.8213-draft.pdf>
- King, Gary. 1995. “Replication, Replication.” *PS: Political Science & Politics* 28(3):444–452. <https://gking.harvard.edu/files/replication.pdf>
- King, Gary. 2007. “An Introduction to the Dataverse Network as an Infrastructure for Data Sharing.” *Sociological Methods & Research* 36(2):173–199. <https://gking.harvard.edu/files/dvn.pdf>
- Lamb, Chris and Stefano Zacchiroli. 2022. “Reproducible Builds: Increasing the Integrity of Software Supply Chains.” *IEEE Software* 39(2):62–70. <https://arxiv.org/pdf/2104.06020>
- Laurie, Ben, Adam Langley and Emilia Kasper. 2013. “Certificate Transparency.” RFC 6962, RFC Editor. <https://www.rfc-editor.org/rfc/rfc6962.txt>
- Lupia, Arthur and Colin Elman. 2014. “Openness in Political Science: Data Access and Research Transparency.” *PS: Political Science & Politics* 47(1):19–42. <https://doi.org/10.1017/S1049096513001716>
- Luttmer, Erzo F. P. 2026. “Report of the Editor, American Economic Review.” *AEA Papers and Proceedings* 116:742–760. <https://doi.org/10.1257/pandp.116.742>
- MacCoun, Robert and Saul Perlmutter. 2015. “Blind Analysis: Hide Results to Seek the Truth.” *Nature* 526:187–189. <https://doi.org/10.1038/526187a>

- Mazor, Matan, Noam Mazor and Roy Mukamel. 2019. “A Novel Tool for Time-Locking Study Plans to Results.” *European Journal of Neuroscience* 49(9):1149–1156. <https://matanmazor.github.io/files/papers/mazor2019novel.pdf>
- McPhillips, Timothy, Craig Willis, Michael R. Gryk, Santiago Núñez-Corrales and Bertram Ludäscher. 2019. Reproducibility by Other Means: Transparent Research Objects. In *2019 IEEE 15th International Conference on eScience*. pp. 502–509. <https://doi.org/10.1109/eScience.2019.00066>
- Meiklejohn, Sarah, Pavel Kalinnikov, Cindy S. Lin, Martin Hutchinson, Gary Belvin, Mariana Raykova and Al Cutter. 2020. “Think Global, Act Local: Gossip and Client Audits in Verifiable Data Structures.” arXiv:2011.04551. <https://arxiv.org/abs/2011.04551>
- Meißner, Echo, Felix Engelmann, Frank Kargl and Benjamin Erb. 2021. PeQES: A Platform for Privacy-Enhanced Quantitative Empirical Studies. In *Proceedings of the 36th ACM/SIGAPP Symposium on Applied Computing (SAC '21)*. pp. 1226–1234. <https://arxiv.org/pdf/2103.05544>
- Merkle, Ralph C. 1980. Protocols for Public Key Cryptosystems. In *Proceedings of the 1980 IEEE Symposium on Security and Privacy*. pp. 122–134. <https://www.ralphmerkle.com/papers/Protocols.pdf>
- Miske, Olivia et al. 2026. “Investigating the reproducibility of the social and behavioural sciences.” *Nature* 652(8108):126–134. <https://doi.org/10.1038/s41586-026-10203-5>
- MITRE Corporation. 2012. “CVE-2012-2459.” Common Vulnerabilities and Exposures. <https://nvd.nist.gov/vuln/detail/CVE-2012-2459>
- Mölder, Felix, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen and Johannes Köster. 2021. “Sustainable data analysis with Snakemake.” *F1000Research* 10:33. Version 2. <https://doi.org/10.12688/f1000research.29032.2>
- Monrose, Fabian, Peter Wyckoff and Aviel D. Rubin. 1999. Distributed Execution with Remote Audit. In *Proceedings of the Network and Distributed System Security Symposium (NDSS '99)*. <https://www.ndss-symposium.org/wp-content/uploads/2017/09/Distributed-Execution-with-Remote-Audit.pdf>
- National Academies of Sciences, Engineering, and Medicine. 2019. *Reproducibility and Replicability in Science*. Washington, DC: The National Academies Press. <https://doi.org/10.17226/25303>
- National Information Standards Organization. 2021. “Reproducibility Badging and Definitions.” NISO RP-31-2021. https://groups.niso.org/higherlogic/ws/public/download/24810/RP-31-2021_Reproducibility_Badging_and_Definitions.pdf

- Newman, Zachary, John Speed Meyers and Santiago Torres-Arias. 2022. Sigstore: Software Signing for Everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. pp. 2353–2367. <https://doi.org/10.1145/3548606.3560596>
- Nosek, Brian A., George Alter, George C. Banks et al. 2015. “Promoting an Open Research Culture.” *Science* 348(6242):1422–1425. <https://doi.org/10.1126/science.aab2374>
- Nüst, Daniel and Stephen J. Egle. 2021. “CODECHECK: an Open Science initiative for the independent execution of computations underlying research articles during peer review to improve reproducibility.” *F1000Research* 10:253. Version 2. <https://f1000research.com/articles/10-253/v2/pdf>
- Open Source Security Foundation. 2026. “SLSA: Supply-chain Levels for Software Artifacts.” <https://slsa.dev/>.
- Peng, Roger D. 2011. “Reproducible Research in Computational Science.” *Science* 334(6060):1226–1227. <https://doi.org/10.1126/science.1213847>
- Pérignon, Christophe, Kamel Gadouche, Christophe Hurlin, Roxane Silberman and Eric Debonnel. 2019. “Certify reproducibility with confidential data.” *Science* 365(6449):127–128. <https://doi.org/10.1126/science.aaw2825>
- Pimentel, João Felipe, Leonardo Murta, Vanessa Braganholo and Juliana Freire. 2019. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. pp. 507–517. <https://doi.org/10.1109/MSR.2019.00077>
- Rabin, Michael O. 1983. “Transaction Protection by Beacons.” *Journal of Computer and System Sciences* 27(2):256–267. [https://doi.org/10.1016/0022-0000\(83\)90042-9](https://doi.org/10.1016/0022-0000(83)90042-9)
- Rainey, Carlisle, Harley Roe, Qing Wang and Hao Zhou. 2025. “Data and Code Availability in Political Science Publications from 1995 to 2022.” *PS: Political Science & Politics* 58(2):339–345. <https://doi.org/10.1017/S1049096524001276>
- Reproducible Builds Project. 2026a. “reprotest: Build Software and Check It for Reproducibility.”. <https://salsa.debian.org/reproducible-builds/reprotest>
- Reproducible Builds Project. 2026b. “strip-nondeterminism: A Tool for Stripping Non-Deterministic Information from Files.”. <https://salsa.debian.org/reproducible-builds/strip-nondeterminism>
- Rundgren, Anders, Bret Jordan and Samuel Erdtman. 2020. “JSON Canonicalization Scheme (JCS).” RFC 8785, Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc8785>
- Samuel, Justin, Nick Mathewson, Justin Cappos and Roger Dingledine. 2010. Survivable Key Compromise in Software Update Systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*. pp. 61–72. <https://doi.org/10.1145/1866307.1866315>

- Samuel, Sheeba and Daniel Mitchen. 2024. “Computational reproducibility of Jupyter notebooks from biomedical publications.” *GigaScience* 13:giad113. <https://doi.org/10.1093/gigascience/giad113>
- Sigstore Project. 2026. “Rekor: Software Supply Chain Transparency Log.” <https://docs.sigstore.dev/logging/overview/>.
- Stodden, Victoria, Jennifer Seiler and Zhaokun Ma. 2018. “An Empirical Analysis of Journal Policy Effectiveness for Computational Reproducibility.” *Proceedings of the National Academy of Sciences* 115(11):2584–2589. <https://doi.org/10.1073/pnas.1708290115>
- Syta, Ewa, Philipp Jovanovic, Eleftherios Kokoris Kogias, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Michael J. Fischer and Bryan Ford. 2017. Scalable Bias-Resistant Distributed Randomness. In *2017 IEEE Symposium on Security and Privacy (S&P)*. pp. 444–460. <https://doi.org/10.1109/SP.2017.45>
- Szalachowski, Pawel. 2018. (Short Paper) Towards More Reliable Bitcoin Timestamps. In *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*. pp. 101–104. <https://arxiv.org/abs/1803.09028>
- The Dataverse Project. 2026. “Universal Numerical Fingerprint (UNF).” <https://guides.dataverse.org/en/latest/developers/unf/index.html>.
- Todd, Peter. 2016. “OpenTimestamps: Scalable, Trust-Minimized, Distributed Timestamping with Bitcoin.” <https://petertodd.org/2016/opentimestamps-announcement>.
- Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*. USENIX Association pp. 1393–1410. <https://www.usenix.org/system/files/sec19-torres-arias.pdf>
- Trisovic, Ana, Matthew K. Lau, Thomas Pasquier and Mercè Crosas. 2022. “A Large-Scale Study on Research Code Quality and Execution.” *Scientific Data* 9:60. <https://www.nature.com/articles/s41597-022-01143-6.pdf>
- Vallet, Nicolas, David Michonneau and Simon Tournier. 2022. “Toward Practical Transparent Verifiable and Long-Term Reproducible Research Using Guix.” *Scientific Data* 9:597. <https://doi.org/10.1038/s41597-022-01720-9>
- Vilhuber, Lars. 2020. “Reproducibility and Replicability in Economics.” *Harvard Data Science Review* 2(4). <https://hdsr.mitpress.mit.edu/pub/fgmpj11/download/pdf>
- Vilhuber, Lars. 2024. “Report of the AEA Data Editor.” *AEA Papers and Proceedings* 114:878–890. <https://doi.org/10.1257/pandp.114.878>
- Vilhuber, Lars, Thu-Mai Christian, Michał Kowalik, Bertram Ludäscher, Timothy McPhillips and Craig Willis. 2023. “TRACE: TRAnsparency CErtified — Trusting Computational Research Without Repeating It.” <https://doi.org/10.5281/zenodo.8000023>

- Walfish, Michael and Andrew J. Blumberg. 2015. “Verifying Computations Without Reexecuting Them.” *Communications of the ACM* 58(2):74–84. <https://dl.acm.org/doi/pdf/10.1145/2641562>
- Wittek, Kevin, Daniel Krakau, Neslihan Wittek, James Lawton and Norbert Pohlmann. 2020. “Integrating bloxberg’s Proof of Existence Service with MATLAB.” *Frontiers in Blockchain* 3:546264. <https://doi.org/10.3389/fbloc.2020.546264>
- Xu, Yiqing and Leo Yang Yang. 2026. “Scaling Reproducibility: An AI-Assisted Workflow for Large-Scale Replication and Reanalysis.” *arXiv preprint arXiv:2602.16733* . <https://arxiv.org/pdf/2602.16733>